Intro To ML
Prof. Mathieu Salzmann

Luigi Ieracitano

May 27, 2026

Contents

1 Introduction **5**

1.1 What is Machine Learning ? . . . 5

1.1.1 Data . . . 5

1.1.2 Insight (what we discover from the datas) . . . 5

1.1.3 Algorithms . . . 6

2 Supervised Learning **7**

2.1 Notation . . . 7

2.2 Linear Regression . . . 7

2.2.1 Training . . . 7

2.2.2 Prediction . . . 8

2.2.3 1D linear regression . . . 9

2.2.4 Two input dimensions . . . 9

2.2.5 Hyperplane . . . 9

2.2.6 Minimizing multivariate function . . . 10

2.2.7 Solution of the linear regression . . . 10

2.2.8 Test time . . . 11

2.3 Model evaluation . . . 11

2.3.1 Evaluation metrics for regression . . . 11

2.3.2 Linear regression with multiple output dimensions . . . 12

2.4 Linear classification . . . 13

2.4.1 Loss function . . . 14

2.4.2 Logistic regression . . . 14

2.4.3 Difference between linear models . . . 16

2.4.4 Support Vector Machine (SVM) . . . 16

2.4.5 Classification evaluation . . . 17

2.4.6 From binary to multi-class classification . . . 18

2.4.7 Decision boundaries: General Definition . . . 18

2.4.8 Classification evaluation for Multi-class . . . 19

2.4.9 Least-square classification . . . 19

2.4.10 Multi-class logistic regression . . . 21

2.4.11 Dealing with multiple classes . . . 22

2.4.12 Comparing linear classifiers . . . 22

2.4.13 Recap - and new challenge . . . 22

2.5 Non-linear Model . . . 23

2.5.1 Histograms . . . 23

2.5.2 k-Nearest neighbor method . . . 23

2.5.3 Feature expansion . . . 24
2.5.4 Model complexity . . . 25
2.5.5 Overfitting vs Underfitting . . . 25
2.5.6 Cross-validation . . . 26
2.5.7 Overfitting with a linear model . . . 29
2.5.8 Kernel function . . . 31
2.6 Data representation . . . 35
2.6.1 Why ? . . . 35
2.6.2 Attributes . . . 35
2.6.3 Bag of Words . . . 36
2.6.4 Bag of visual words . . . 37
2.6.5 Visual dictionary . . . 37
2.6.6 Data pre-processing . . . 37
2.6.7 Data normalization . . . 38
2.6.8 Imbalanced data . . . 39
2.7 Deep Learning . . . 42
2.7.1 Parametric feature expansion . . . 42
2.7.2 Hidden representation . . . 42
2.7.3 Multilayer Perceptron (MLP) . . . 44
2.7.4 Training an MLP . . . 45
2.7.5 Optimization . . . 46
2.7.6 Practical training issues . . . 47
2.7.7 From regression to classification . . . 48
2.7.8 Working with images . . . 49
2.7.9 Convolutional Neural Network (CNN) . . . 49
2.7.10 Semantic segmentation . . . 54
2.7.11 The keys to the success of Deep Learning . . . 56
2.7.12 Despite the success: Adversarial examples . . . 57
2.8 From images to text . . . 58
2.9 Representing text . . . 58
2.9.1 The heterogeneity problem . . . 58
2.9.2 Bag of Words: a document-level representation . . . 58
2.9.3 Word representations . . . 59
2.9.4 Words as vectors: the naive one-hot approach . . . 59
2.9.5 Word embeddings . . . 59
2.9.6 Why individual words are not enough . . . 61
2.10 Modeling sequences . . . 61
2.10.1 Sequence-to-sequence: encoder–decoder . . . 61
2.11 Recurrent Neural Networks (RNNs) . . . 62
2.11.1 Idea . . . 62
2.11.2 Unrolling . . . 62
2.11.3 Equations for the standard (Elman) RNN . . . 63
2.11.4 Three uses of an RNN . . . 63
2.11.5 RNN variants: LSTM and GRU . . . 64

2.11.6 Training an encoder–decoder RNN . . . 65
2.11.7 Limitations of RNNs . . . 65
2.12 Attention . . . 66
2.12.1 Motivation . . . 66
2.12.2 The attention function . . . 66
2.12.3 The attentive encoder–decoder . . . 67
2.12.4 Bonus: interpretability . . . 67
2.12.5 Recurrence as problem . . . 68
2.13 Transformers . . . 68
2.13.1 Self-attention . . . 68
2.13.2 Multi-head self-attention . . . 71
2.13.3 The transformer block . . . 71
2.13.4 Full transformer . . . 72
2.13.5 The decoder side: masked self-attention and cross-attention . . . 72
2.13.6 Position embeddings . . . 74
2.14 GPT: a decoder-only transformer . . . 74
2.14.1 (Pre)Training . . . 75
2.14.2 Fine-tuning and the paradigm shift . . . 76
2.14.3 Scale . . . 77
2.14.4 Other famous text models . . . 77
2.15 Transformers beyond text . . . 77
2.15.1 Transformers for images . . . 77
2.15.2 Vision Transformer (ViT) . . . 78

3 Unsupervised Learning **79**
3.1 From supervised to unsupervised learning . . . 79
3.2 Recall from Linear Algebra . . . 79
3.3 Dimensionality reduction . . . 79
3.3.1 Intuition: the data manifold . . . 80
3.3.2 Formal setting . . . 80
3.3.3 Principal Component Analysis (PCA) . . . 80
3.3.4 Kernel PCA . . . 83
3.3.5 Autoencoders . . . 86
3.4 Clustering: another unsupervised task . . . 89
3.5 K-means clustering . . . 90
3.5.1 Intuition . . . 90
3.5.2 Algorithm . . . 90
3.5.3 Properties and practical considerations . . . 91
3.5.4 Example: color-based image segmentation . . . 92
3.5.5 The Euclidean distance is not always appropriate . . . 92
3.6 Fisher Linear Discriminant Analysis (LDA) . . . 93
3.6.1 Clustering within each class: within-class scatter . . . 93
3.6.2 Separating the classes: between-class scatter . . . 94
3.6.3 Fisher LDA objective . . . 94

3.6.4 Example: visualizing MNIST . . . 95
3.6.5 Eigenfaces vs Fisherfaces . . . 95
3.7 Spectral clustering . . . 96
3.7.1 Why K-means and LDA can fail . . . 96
3.7.2 Connectivity as the right notion . . . 97
3.7.3 Building the graph . . . 97
3.7.4 Graph cut . . . 97
3.7.5 Graph terminology . . . 98
3.7.6 Normalized cut . . . 98
3.7.7 Transformed formulation . . . 98
3.7.8 Spectral relaxation . . . 99
3.7.9 K-way partitioning . . . 99
3.7.10 Beyond compactness and connectivity: density . . . 100

4 Review **101**
4.1 The machine learning pipeline . . . 101
4.2 Supervised learning . . . 101
4.2.1 Setting . . . 101
4.2.2 The linear model . . . 101
4.2.3 Linear regression . . . 102
4.2.4 Linear classification . . . 102
4.2.5 Dealing with nonlinear data . . . 103
4.2.6 Neural networks . . . 104
4.2.7 Convolutional neural networks . . . 104
4.2.8 Sequences and text . . . 104
4.2.9 k-Nearest Neighbors . . . 105
4.3 Unsupervised learning . . . 105
4.3.1 Clustering . . . 105
4.3.2 Dimensionality reduction . . . 105
4.4 Cross-cutting concepts . . . 106
4.4.1 Data representation . . . 106
4.4.2 Cross-validation . . . 106

Chapter 1
Introduction

1.1 What is Machine Learning ?

Machine learning = Data →Algorithms →Insight

1.1.1 Data

Data = attributes/tabular, text, speech, images, mixed.

Definition 1 (Data sample)**.** An individual observation. E.g., an image, or a list of attributes for one patient

Definition 2 (Data set)**.** A collection of multiple data sample. E.g., a collection of images, the attribute lists for multiple patients.

Unsupervised data : Each sample consists only of an observation, e.g., an image (without information about its content).
Supervised data : Each sample comes with additional annotations, e.g., category label.

1.1.2 Insight (what we discover from the datas)

What insight ?

- Precise, concrete prediction. E.g., the category depicted by an image.
- Better understanding of a phenomenon. E.g., identify the mother's charasteristics that lead to low birth weight.
- Better understanding of data. E.g., analyze the influences between different musical genres.
- **in this course** we mostly focus on precise insights. In particular we will study two classes of ML problems : **regression** and **classification**

Regression : Predict continuous value(s) for a given sample. E.g., predict the birth weight of a baby, human pose estimation : predict the 3D positions of human joints (multiple values).
Classification : Predict one discrete label for a given sample. E.g., binary classifications for movies (like vs not like; 1 or 0), muti-class image recognition (cat vs tree vs car; 0 vs 1 vs 2).

Order:
In regression, the values typically follow an order. E.g, predicting a weight of 3002g insted of 2977g is better that predicting 2500g.
In classification, the categories do not follow an order. E.g, predicting a "tree" instead of a "cat" is just as wrong as predicting "car". →here it's true or false.

1.1.3 Algorithms

What (classes of) algorithms ?

- Supervised learning :
  - Relies on supervised data
  - The annotations typically correspond to the desired insight
- Unsupervised learning :
  - Relies on unsupervised data
  - The goal is rather to analyze the observed data set

Unsupervised learning:

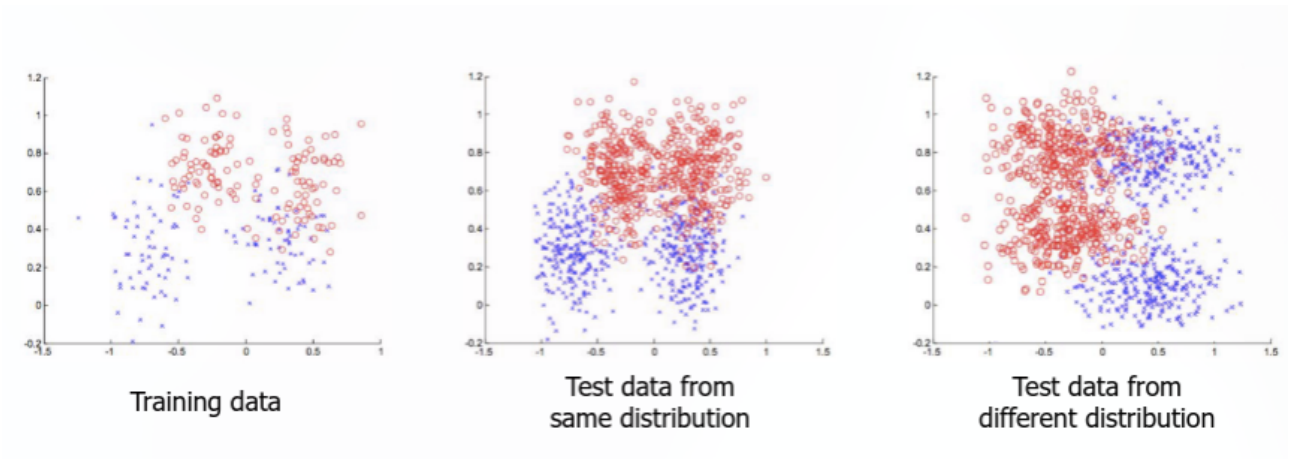
- Single stage: Analyze the whole data or transform it for further analysis.

Supervised learning:

- Stage 1: Training: Use data with ground-truth (info that is true) labels to optimize model parameters.
- Stage 2: Testing: Predict the output for a new data sample.
- Assumption : The annotations (correct responses) are the insight that we seek to obtain from the data. E.g.,: category label, human pose

Training set vs test set:

- The training set and the test set should always be **completely separate**.
- Never use the test annotations during training →the test set is not to train or to improve the performance of the model, but only to test his efficiency.
- Assumption: they are both drawn from the same statistical distribution. E.g., digit recognition: A model trained on MNIST will work poorly on Street View House Numbers



Chapter 2

Supervised Learning

2.1 Notation

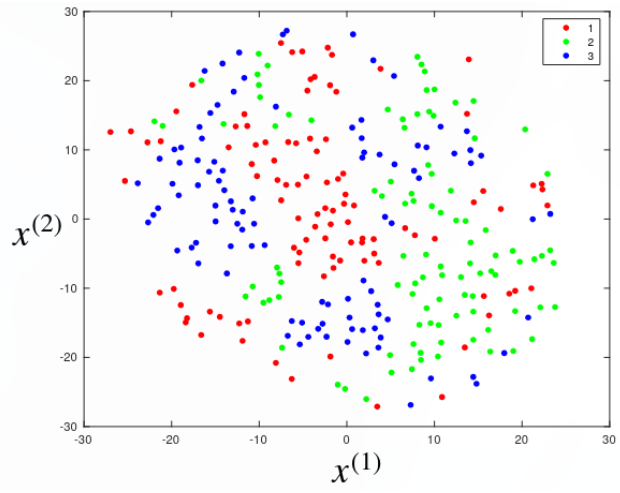
We denote the ith data sample (input) in the collection of N samples as

$$x_i \in \mathbb{R}^D,$$

a column vector of dimension D.
We denote the ith label (output) in the collection of N samples as y_i.
y_i :

- for classification represents the category the sample belongs to.
- for regression, it can be a single continuous value, or a column vector of dim C.

Example:



Here, x_i is one 2D point and thus $D = 2$, y_i is a discrete value indicating the class (color), i.e, 1, 2 or 3.

2.2 Linear Regression

2.2.1 Training

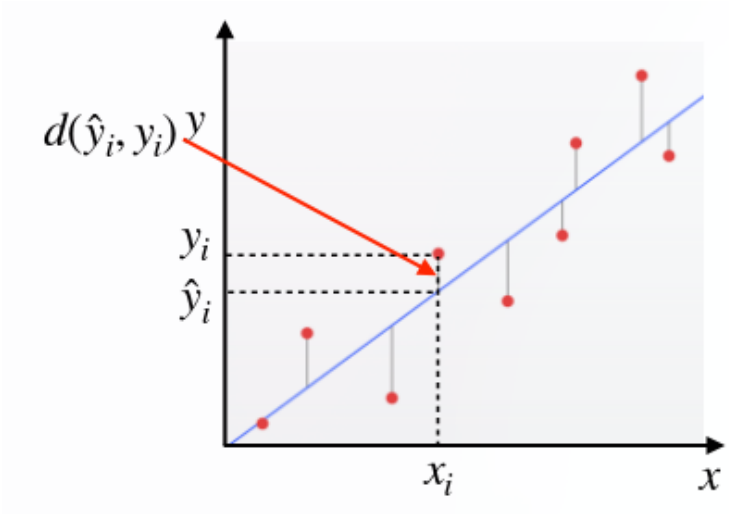
Given N annotated data samples (x_i, y_i) with measurements, we can plot the data, and we will approximate it with a line. Our goal is to find the line that bests fits these measurements →linear regression. It consists of finding the best line parameters $w^{(0)*}$ and $w^{(1)*}$ for some given data.
This corresponds to the training stage:
Given N training pairs (x_i, y_i), we aim to find $\left(w^{(0)}, w^{(1)}\right)$, s.t the predictions of the model

$$\hat{y_i} = w^{(0)} + w^{(1)} x_i$$

are close to the true values y_i.
The mesure of closeness/error (loss function) between y_i and $\hat{y}_i$ is represented by the Euclidean distance:

$$d\left(\hat{y}_i, y_i\right) = \sqrt{\left(\hat{y}_i - y_i\right)^2}$$

.



$\hat{y}_i - y_i$: often reffered to as the *residual*

In practice, one often prefers using the squared Euclidian distance:

$$d^2\left(\hat{y}_i, y_i\right) = \left(\hat{y}_i - y_i\right)^2$$

Training can then be expressed as the least-squared minimization problem:

$$R(w) = \min_{w^{(0)}, w^{(1)}} \frac{1}{N} \sum_{i=1}^{N} d^2(\hat{y}_i, y_i)$$

2.2.2 Prediction

Once the best line for the given N observations is found, we can use it to predict a y value for a new x.
Let $w^{(0)*}$ and $w^{(1)*}$ be the best line parameters given the observations.
Then

$$\hat{y} = w^{(0)*} + w^{(1)*} x$$

is the estimate value of y for any x.

2.2.3 1D linear regression

With 1D inputs, training a linear regressor aims to minimize

$$R\left(w^{(0)}, w^{(1)}\right) = \frac{1}{N}\sum_{i=1}^{N} d^2\left(\hat{y}_i, y_i\right) = \frac{1}{N}\sum_{i=1}^{N}\left(w^{(0)} + w^{(1)}x_i - y_i\right)^2$$

Equivalently, we can write

$$R\left(w\right) = \frac{1}{N}\sum_{i=1}^{N}\left(w^T x_i - y_i\right)^2 = \frac{1}{N}\sum_{i=1}^{N}\left(x_i^T w - y_i\right)^2$$

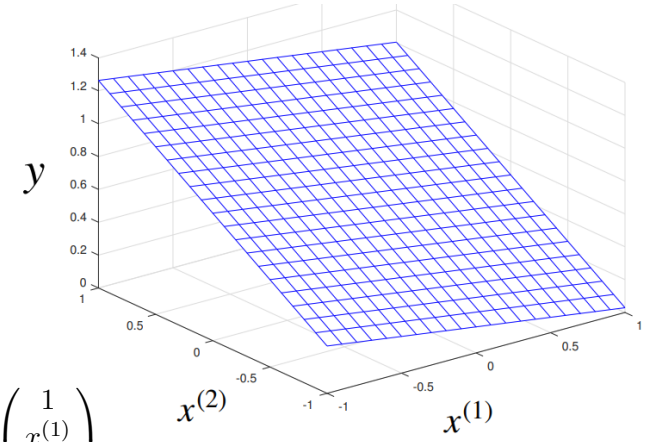
where

$$x_i = \begin{pmatrix} 1 \\ x_i \end{pmatrix} \in \mathbb{R}^2 \qquad w = \begin{pmatrix} w^{(0)} \\ w^{(1)} \end{pmatrix} \in \mathbb{R}^2$$

2.2.4 Two input dimensions

Here instead of a line, we can define a plane.
Expressed

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \end{pmatrix}$$



2.2.5 Hyperplane

This can be generalized to higher dimensions.
In dimension D we can write

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} + \ldots + w^{(D)}x^{(D)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{pmatrix}$$

Whatever the dimension, we can write

$$y = w^T x \quad x \in \mathbb{R}^{D+1}$$

, where the extra dimension contains a 1 to account for $w^{(0)}$ (bias).

Because the output remains 1D, we can still use the same least square loss function, and write training as

$$\min_w \frac{1}{N}\sum_{i=1}^{N} d^2\left(\hat{y}_i, y_i\right) \iff \min_w \frac{1}{N}\sum_{i=1}^{N}\left(x_i^T w - y_i\right)^2$$

Solving this minimization problem involves computing its derivatives (derivation and gradient) w.r.t the model parameters (w).

2.2.6 Minimizing multivariate function

As in the 1D case (derivative = 0), to minimize a function, we seek the point w^* where the gradient where the gradient vanishes, i.e,

$$\nabla_w R(w^*) = 0$$

2.2.7 Solution of the linear regression

With our least-square formulation, we seek to minimize

$$R(w) = \frac{1}{N}\sum_{i=1}^{N}(x_i^T w - y_i)^2$$

This is a convex function of w, which means that we search for parameters w^* s.t

$$\nabla_w R(w^*) = 0$$

Let us start by computing the gradient $\nabla_w R(w)$

Hint 1: $R(w)$ is an average over the N training samples

- As the gradient of an average is the average of the gradients, you can focus on computing the gradient

$$\nabla_w (x_i^T w - y_i)^2$$

Hint 2: Use the chain rule

- The function $(x_i^T w - y_i)^2$ has the form $f(g(w))$, with $f(g) = g^2$
- You can compute its gradient as $\frac{df(g)}{dg}\nabla_w g(w)$

Hint 3: To compute the gradient of g(w), you can make use of the following general vector derivative rule:

- Given 2 vectors (a, b) in $\mathbb{R}^D$

$$\nabla_b (a^T b) = a$$

So with the chain rule:

$$\nabla_w R = \frac{1}{N}\sum_{i=1}^{N}\frac{d(x_i^T w - y_i)^2}{d(x_i^T w - y_i)} \cdot \nabla_w (x_i^T w - y_i) = \frac{2}{N}\sum_{i=1}^{N}(x_i^T w - y_i)\cdot x_i$$

To obtain the solution, we search for w^* s.t

$$\nabla_w R(w^*) = \frac{2}{N}\sum_{i=1}^{N} x_i (x_i^T w^* - y_i) = 0$$

This means

$$(\sum_{i=1}^{N} x_i x_i^T) w^* = \sum_{i=1}^{N} x_i y_i$$

$(D+1)\times(D+1)$ matrix $\cdot$ $(D+1)\times 1$ vector = $(D+1)\times 1$ vector

Let us now group all x_i and all y_i in a matrix and a vector

$$X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(D)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(D)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_N^{(1)} & x_N^{(2)} & \dots & x_N^{(D)} \end{pmatrix} \in \mathbb{R}^{N\times(D+1)}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

Then we can re-write the solution as

$$X^T X w^* = X^T y$$

This finally give us

$$w^* = (X^T X)^{-1} X^T y = X^\dagger y$$

where $X^\dagger$ is known as the Moore-Penrose pseudo-inverse of X.

2.2.8 Test time

So now the predicted value $\hat{y_t}$ becomes

$$\hat{y_t} = \begin{pmatrix} 1 \\ x_t \end{pmatrix}^T w^* = (w^*)^T \begin{pmatrix} 1 \\ x_t \end{pmatrix}$$

2.3 Model evaluation

Once an ML model is trained, one would typically understand how well it performs on unseen test data

- At this stage, the parameters of the model are fixed
- Recall that the training and testing data must be separated!

During this evaluation, one compares the predictions of the model with the true annotations of the test data

- In contrast to the training stage, the model parameters are not updated
- The evaluation metric may directly be the loss function, but may also differ from it

2.3.1 Evaluation metrics for regression

Mean Squared Error (MSE): Same as the loss function but for N_t test samples.

$$\text{MSE} = \frac{1}{N_t} \sum_{i=1}^{N_t} (\hat{y_i} - y_i)^2$$

where $\hat{y_i}$ is the prediction for the test sample i and y_i the corresponding ground-truth value

Root Mean Squared Error (RMSE): Square-root of the MSE
Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N_t} \sum_{i=1}^{N_t} |\hat{y_i} - y_i|$$

Mean Absolute Percentage Error (MAPE)

$$\text{MAPE} = \frac{1}{N_t} \sum_{i=1}^{N_t} \left| \frac{\hat{y_i} - y_i}{y_i} \right|$$

2.3.2 Linear regression with multiple output dimensions

E.g., human pose estimation.

Model

The linear model has to be modified to output multiple values (not a vector w anymore because $w^T x_i$ output a single value)
So we use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$ s.t

$$\hat{y_i} = W^T x_i = \begin{pmatrix} w_{(1)}^T \\ w_{(2)}^T \\ \vdots \\ w_{(C)}^T \end{pmatrix} x_i$$

where each $w_{(j)}$ is a $(D+1)$-dimensional vector, used to predict one output dimension.
The multi-output linear regression is written

$$R(W) = \min_w \sum_{i=1}^{N} \left\| W^T x_i - y_i \right\|^2$$

where $\|a\| = \sqrt{\sum_{i=1}^{C} (a^{(i)})^2}$ indicates the norm of a vector (here $a \in \mathbb{R}^C$), it's the equivalent of the least square but for vectors.

To compute the gradient of this function, we make use of the following properties:
1. Norm and derivative:

$$\|a\|^2 = a^T a$$

and thus

$$\nabla_a \|a\|^2 = 2a$$

2. Matrix derivative:

$$\nabla_B (a^T B) = a$$

Thus using the chain rule we obtain the gradient

$$\nabla_W R = 2 \sum_{i=1}^{N} x_i (x_i^T W - y_i^T)$$

setting it to zero means that

$$(\sum_{i=1}^{N} x_i x_i^T) W^* = \sum_{i=1}^{N} x_i y_i^T$$

$(D+1) \times (D+1)$ matrix * $(D+1) \times C$ matrix = $(D+1) \times C$ matrix
We can again group the inputs in a matrix $\mathbf{X} \in \mathbb{R}^{N\times(D+1)}$, but the outputs now need to be grouped in a matrix (not a vector anymore), as

$$Y = \begin{pmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_N^T \end{pmatrix} = \begin{pmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(C)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(C)} \\ \vdots & \vdots & \vdots & \vdots \\ y_N^{(1)} & y_N^{(2)} & \cdots & y_N^{(C)} \end{pmatrix} \in \mathbb{R}^{N\times C}$$

Then, following the same strategy as before, we have

$$W^* = (X^T X)^{-1} X^T Y = X^\dagger Y$$

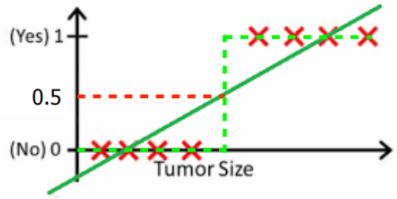
2.4 Linear classification

To predict one discrete label for a given sample e.g., binary classification or movies (like vs not), multi-class image recognition.
Classification can be tackle as a regression task (given a threshold we can classify the output in categories).

E.g. Classify a tumor as benign vs malignant

- 1D input (tumor size), 1D output (malignant/benign)
- Using a linear model, we can predict the label $\hat{y} = w^{(0)} + w^{(1)}x$

$$\text{label} = \begin{cases} 1 & \text{if } \hat{y}_t \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \qquad \text{with } \hat{y}_t = \mathbf{x}_t^T \mathbf{w}^*$$



To fit the linear model to training data, the same strategy as for linear regression can be used:

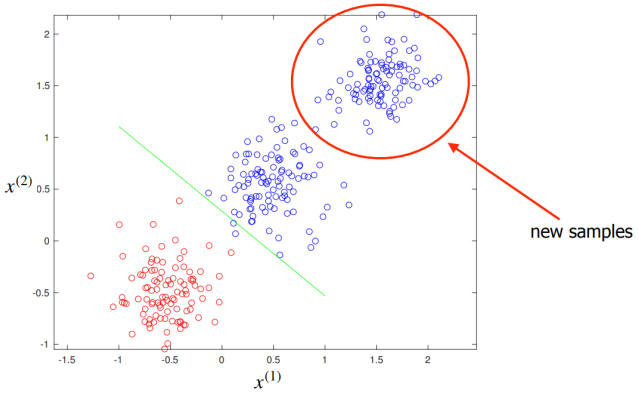
- Given N pairs (x_i, y_i), we can use the least-square loss to write

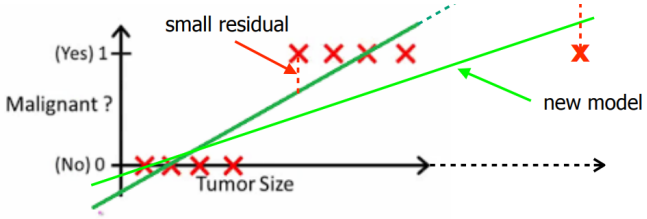
$$R(w) = \min_w \sum_{i=1}^{N} (x_i^T w - y_i)^2$$

So because the formulation is the same, the solution too

$$w^* = (X^T X)^{-1} X^T y$$

However, this model isn't that good because with some data, the **decision boundary** (threshold) will not react well and misclassified some samples.





So we will introduce an other model the **logistic regression**.

2.4.1 Loss function

Definition 3 (Loss function)**.** The loss function (general ML concept) : $\mathcal{L}(\hat{y}_i, yi)$ computes an error value between the prediction and the true value.

In general we use the empirical risk (the mean of the loss on the training or test data)

$$\hat{R}(w) = \frac{1}{N_t} \sum_{i=1}^{N_t} \mathcal{L}(y_i, \hat{y}_i)$$

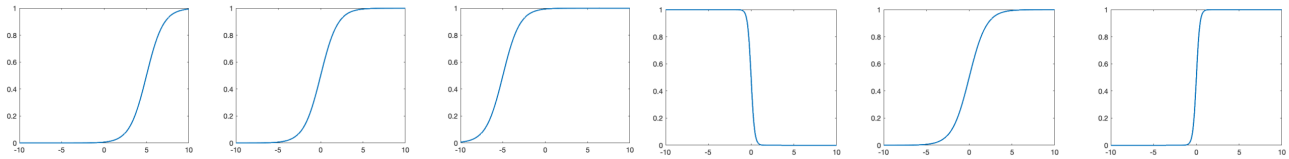
with a different loss function $\mathcal{L}(y_i, \hat{y}_i)$.

2.4.2 Logistic regression

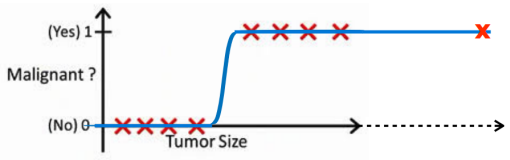
The idea of the logistic regression is that first we compute $y = w^T x$ and then we apply the sigmoid on y written $\sigma(y)$, to establish a probability $p(y = 1|x)$. Then the decision boundary is obtained when the probability equals (0.5).
Logistic function : $f(a) = \frac{1}{1+exp(-a)}$
Applying it to the output of a linear model gives a prediction:

$$\hat{y} = \frac{1}{1 + exp(-w^{(0)} - w^{(1)}x)}$$

This is a smooth approximation of the step function (avoid the gradient that is zero almost everywhere).



Then the data fit much better even with the presence of outliers.



In the binary case, with D dimensional inputs, we can write the prediction of the model (logistic regression) as

$$\hat{y} = \sigma(w^T x) = \frac{1}{1 + exp(-w^T x)}$$

it can be interpreted as the probability that **x** belongs to the positive class $\rightarrow 1 - \hat{y} =$ proba. negative class.
Notes, in the logistic regression:

$$y \in \{0, 1\} \text{ with } 0 \rightarrow \text{negative class}, 1 \rightarrow \text{positive class}$$

$$\text{The model computes: } \hat{y} = P(y = 1|x)$$

Even that the name of this model is logistic **regression**, this is truly a classification model, not a regression one.

Training

To learn the parameters w need a loss function that works well with probabilities, and keeps the problem convex:

Cross-entropy :

1D case

$$R(w) = -\sum_{i=1}^{N}(y_i \ln(\hat{y}_i) + (1-y_i)\ln(1-\hat{y}_i)) = -\sum_{i \in positivesamples} \ln(\hat{y}_i) - \sum_{i \in negativesamples} \ln(1-\hat{y}_i)$$

as y_i is either 0 or 1

It's a **convex** function, however, its minimization has no closed-form solution. So we optimize it via gradient descent.

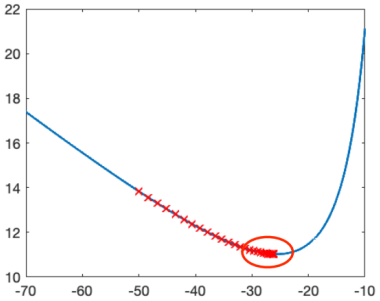
The **algorithm** for minimization is:

- 1. Initialize w_0 (e.g, randomly)
- 2. While not converged $\implies$ *update* $\mathrm{w}_k \leftarrow w_{k-1} - \eta \frac{dR(w_{k-1})}{dw}$ with η the step size of each iteration (learning rate)

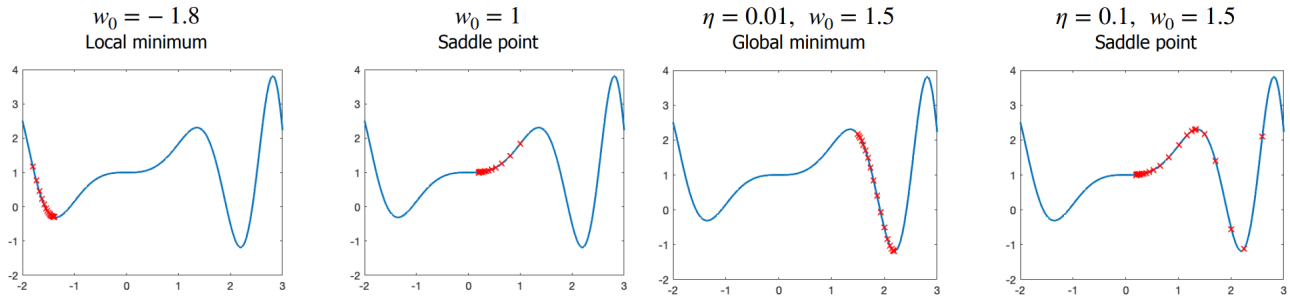
This algorithm converges because the derivative becomes smaller ($\frac{dR(w)}{dw} \to 0$ (∇w becomes smaller)).

The η influences the speed of the convergences (doing bigger or smaller steps), but having too large steps may make the algorithm to jumps between two points and never converge.

As the initial w_0 is the starting point, having a w_0 near the minimum will makes the algorithm converges faster.



Remark To minimize **non-convex** function the same algorithm can be used. However, different initializations yield to different results because there is more than one point with derivative equal to zero.



D dimensions case As in practice we have more than one parameter (i.e $\mathbf{w} \in \mathbb{R}^{D+1}$), we use the function gradient:

$$\nabla_w R(w) = \begin{pmatrix} \frac{\partial R}{\partial w^{(0)}} \\ \frac{\partial R}{\partial w^{(1)}} \\ \vdots \\ \frac{\partial R}{\partial w^{(D)}} \end{pmatrix} \in \mathbb{R}^D$$

Recap The gradient at a point w has the direction of the greatest increase of the function at w and its magnitude is the rate of increase in that direction.

So to decrease the function we therefore move in the direction opposite to the gradient.

Gradient descent is used to find the *argmin* of a function, i.e., we want to find the values of parameters x such that a function f is minimized :

$$x^* = \underset{x}{\operatorname{argmin}} f(x)$$

Algorithm - gradient descent

- 1. Initialize w_0 (e.g, randomly)
- 2. While not converged $\implies$ *update* $\mathrm{w}_k \leftarrow w_{k-1} - \eta \nabla_w R(w_{k-1})$

Cross-Entropy (multiple outputs):

$$R(w) = -\sum_{i=1}^{N} y_i \ln \sigma(w^T x_i) + (1 - y_i) \ln(1 - \sigma(w^T x_i))$$

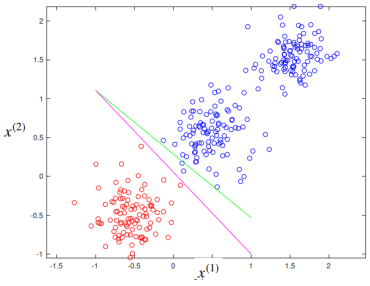
$$= - \sum_{i \in positivesamples} \ln \sigma(w^T x_i) - \sum_{i \in negativesamples} \ln(1 - \sigma(w^T x_i))$$

To minimize it via gradient descent we compute its gradient w.r.t w

$$\nabla_w R(w) = \sum_{i=1}^{N} (\hat{y}_i - y_i) x_i$$

where y_i is the true label for sample i, i.e., 1 for positive samples and 0 for negative samples.

And now, even with outliers, the boundary remains correct (Least-square classification in green, logistic regresion in magenta)



2.4.3 Difference between linear models

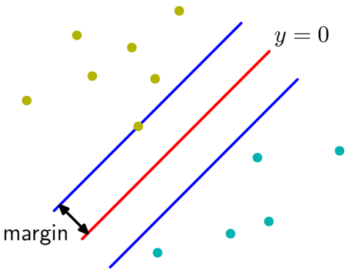
The real difference between linear models is the **loss function**, it has an impact on the resulting algorithm.

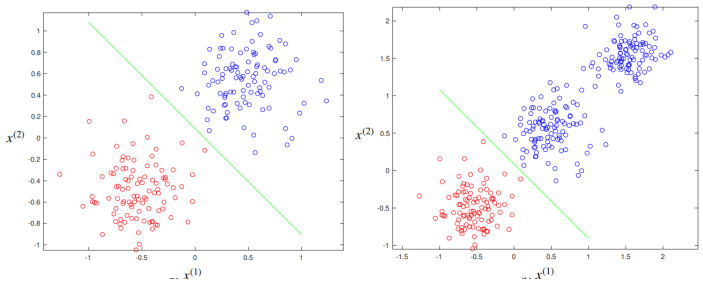
2.4.4 Support Vector Machine (SVM)

The SVM is a binary classifier based on the linear model as before:

$$\hat{y} = w^T x$$

It aims to maximize the margin (the distance between the decision boundary and the nearest sample) as a good decision boundary should maximize it, while enforcing a correct classification for each training samples.





So here we see that adding a new samples does not change the decision boundary as the margin stay the same.

2.4.5 Classification evaluation

For binary case we could use confusion matrix:
TP: True positive (number of samples correctly classified as positive)
FP: False positive (number of samples incorrectly classified as positive)
TN: True negative (number of samples correctly classified as negative)
FN: False negative (number of samples incorrectly classified as negative)
P: Total number of positive samples
N: Total number of negative samples

True class → / Hypothesized class ↓	Pos	Neg
Yes	TP	FP
No	FN	TN
	P=TP+FN	N=FP+TN

Definition 4 (Accuracy)**.** Percentage of correctly classified samples

$$Acc = \frac{TP + TN}{P + N}$$

However, the accuracy doesn't give the information about the behavior of a model, in fact models with same accuracy could have a totally different behavior(e.g, one performing poorly on positive recogniton and well on negative one and the other one the total contrary).
Thus we could use other metrics in complement:

Definition 5 (Precision)**.** Percentage of samples classified as positives that are truly positives

$$Precision = \frac{TP}{TP + FP}$$

Definition 6 (Recall)**.** Percentage of positive samples that are correctly classified as positives

$$Recall = \frac{TP}{P}$$

But, still two classifiers with the same precision and recall could have a very different behavior as before (checking for accuracy could clearly show this), so we could too look for :

Definition 7 (False positive rate)**.** Percentage of negative samples that are incorrectly classified as positives

$$FP\ rate = \frac{FP}{N}$$

Definition 8 (F1 score)**.** Single number that combines precision and recall

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

The F1 score is used as it provides a balanced measure between precision and recall, which is particularly useful when the classes are imbalanced.

Example Class A: 990 samples, Class B: 10 samples. With an accuracy of 99% we don't know if our model recognizes the class B or it predict every samples as class A. But with F1 score this would be detected (poor F1 score).

ROC curves

Many classifiers output a score/confidence for their predictions (e.g., $\hat{y}$ is a continuous value between 0 and 1 in logistic regression).
The decision between positive and negative can be computed by thresholding this score:

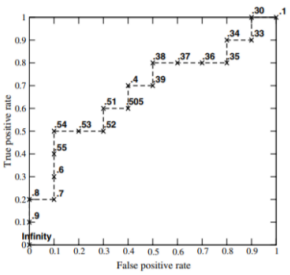
$$\text{label}_i = 1 \text{ if } \hat{y}_i \geq \tau$$

where τ is a given threshold

The Receiver Operating Characteristic (ROC) curve plots the true positive rate (recall) as a function of the false positive rate, obtained by varying the score threshold.

Inst#	Class	Score	Inst#	Class	Score
1	p	.9	11	p	.4
2	p	.8	12	n	.39
3	n	.7	13	p	.38
4	p	.6	14	n	.37
5	p	.55	15	n	.36
6	p	.54	16	n	.35
7	n	.53	17	p	.34
8	n	.52	18	n	.33
9	p	.51	19	p	.30
10	n	.505	20	n	.1

Samples sorted by score
(*Class* denotes the true label)



ROC curve

This model give a score (probability) to each observation (here Inst) and make the threshold vary, then compute the TP and FP rate, and plot the curves.
A good model as a plot near the top left corner (high TP rate and low FP rate) as we mesure the quality of the model with the Area under the curve **(AUC)**(goes from 0 to 1), 1.0 that the model is perfect, 0.5 is the result if the model makes random predictions, and less is worse than randomness.

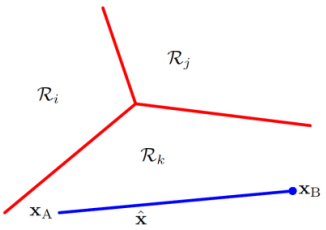
2.4.6 From binary to multi-class classification

What if we have more than two classes? How our three linear models can handle this scenario ?

2.4.7 Decision boundaries: General Definition

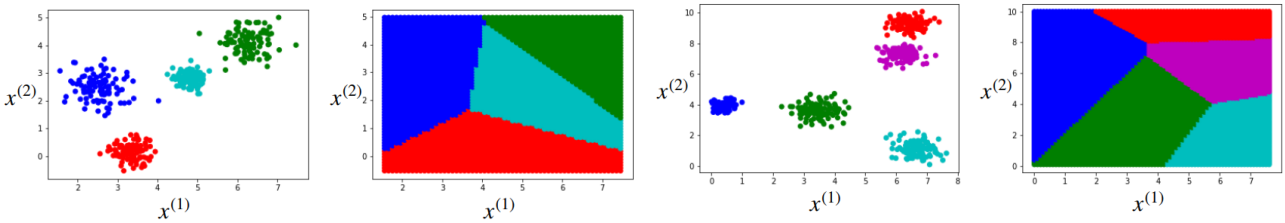
First let's update our knowledge on decision boundaries, as here we are not anymore treating binary label.
The decision boundary between two classes k and j is defined as the set of points x such that

$$\hat{y}^{(k)}(x) = \hat{y}^{(j)}(x)$$



This corresponds to the $(D-1)-$dimensional hyperplane.
The decision regions are **singly-connected and convex** (any point $\hat{x}$ between two points x_A and x_B in the same region (e.g, k) will also be in the same region)

Decision boundaries for C = 4 and C = 5 classes.



2.4.8 Classification evaluation for Multi-class

Confusion matrix for multi-class case.

		Predicted		
		Iris-setosa	*Iris-versicolor*	*Iris-virginica*
Actual	*Iris-setosa*	12	1	1
	Iris-versicolor	0	16	0
	Iris-virginica	0	1	16

Accuracy : A global accuracy can be obtained by summing the correct predictions for all classes (diagonal values of the confusion matrix), and dividing this by the total number of samples (sum of all entries in the confusion matrix).
Precision, recall, FP-rate and F1-score: The simplest approach to handling these metrics is to compute them for each individual class, and then average over the classes.
In this case, each individual class is in turn considered as the "positive" class while the others act as the "negative" class.
This does not account for class imbalance (some classes may have many more samples than others). This can be handled by weighting the metric for each class by a value depending on the proportion of samples from that class.

2.4.9 Least-square classification

The most common approach to modeling a multi-class problem consists of encoding the class label as a vector with a single 1 at the index corresponding to the category and 0s elsewhere.
Example in a 5-class problem, a sample in class 2 is represented as:

$$y = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

This is referred as **one-hot-encoding**.
As this is similar to making the model output multiple values (as for multi-output regression), we therefore again use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$, s.t

$$\hat{y_i} = W^T x_i = \begin{pmatrix} {w_{(1)}}^T \\ {w_{(2)}}^T \\ \vdots \\ {w_{(C)}}^T \end{pmatrix} x_i$$

where each $w_{(j)}$ is a $(D+1)-$dimensional vector, used to predict one output dimension, i.e., the **score** for one class.

Training

We can then write training as

$$\min_w \sum_{i=1}^{N} \left\| W^T x_i - y_i \right\|^2$$

where each y_i is now a $C-$dimensional vector with a single one at the index corresponding to the class label and zeroes everywhere else.

Solution

The inputs and the outputs are two matrices:

$$X = \begin{pmatrix} x_i^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix} \in \mathbb{R}^{N\times(D+1)} \quad Y = \begin{pmatrix} y_i^T \\ y_2^T \\ \vdots \\ y_N^T \end{pmatrix} \in \mathbb{R}^{N\times C}$$

X = data $\quad Y$ = True classes
N = number of samples $\quad D$ = number of features (+1 for bias) $\quad C$ = number of classes
Then, the solution has the same form as in the regression case (obtained by zeroing out the gradient w.r.t. W), i.e.,

$$W^* = (X^T X)^{-1} X^T Y = X^\dagger Y$$

W = weight matrix (e.g, $W = [W_0, W_1]$, $\quad W_0, W_1$ weight classes 0 and 1)

Prediction

Given the optimal parameters, the prediction for a new sample is computed as

$$\hat{y} = (W^*)^T x$$

The sample is then assigned to class k if

$$\hat{y}^{(k)} > \hat{y}^{(j)}, \quad \forall j \neq k$$

In other words the final label k is obtained as

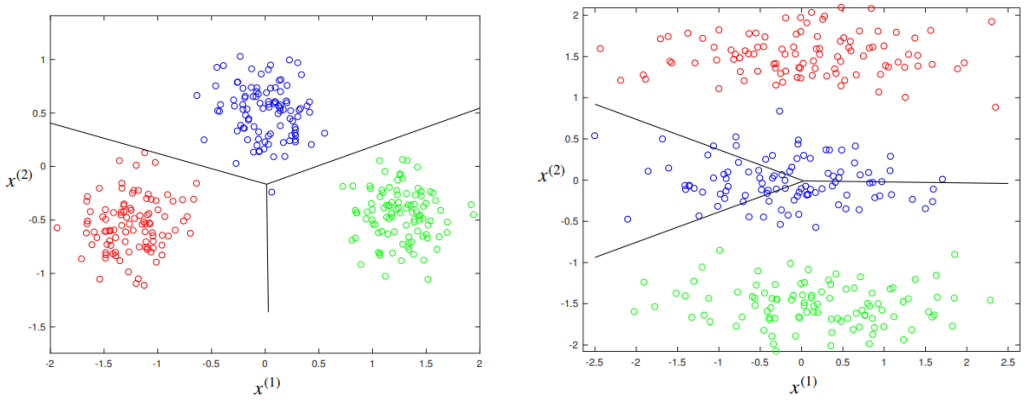
$$k = \underset{j}{\text{argmax}}\, \hat{y}^{(j)}$$

That is, the label is the index (position) of the maximum value in the vector $\hat{y}$
Example if we have

$$\hat{y} = \begin{pmatrix} \hat{y}^{(1)} \\ \hat{y}^{(2)} \\ \hat{y}^{(3)} \end{pmatrix} = \begin{pmatrix} 2.3 \\ 0.7 \\ -1.5 \end{pmatrix}$$

here the argmax_j(biggest score) is 2.3, so the label predicted is the label 1.

Example of success and failure case of 2D inputs and 3 classes least-square classification:



Drawbacks

As in the binary case, the multi-class least-square classification suffer from outliers, because it tries to minimize the distance (regression). So we need an other model like the logistic regression one but for multi-class: **Multi-class logistic regression** WOWOWOW !!

2.4.10 Multi-class logistic regression

We still use the one-hot enconding to represent multi-class labels.
We again need to use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$ to represent the model parameters.
But now, the probability for a class k is given by the **softmax** function (takes scores and transforms them into probabilities)

$$\hat{y}^{(k)}(x) = \frac{\exp(w_{(k)}^T x)}{\sum_{j=1}^{C} \exp(w_{(j)}^T x)}$$

The empirical risk can then be derived from the multi-class version of the cross entropy, which yields

$$R(W) = -\sum_{i=1}^{N}\sum_{k=1}^{C} y_i^{(k)} \ln \hat{y}_i^{(k)} = -\sum_{i=1}^{N} \ln \hat{y}_i^{(\text{true class of i})}$$

(i ←go through the samples, k ←go through the classes)
As in the binary case :

- a single $y_i^{(k)}$ is 1 for every sample i, so we really have one term per training sample. (as if sample k belongs to class i then only the term for this class/label stay in the sum (other are 0)).
- training is done by minimizing this empirical risk using gradient descent.

Gradient

The gradient descent is the same algorithm as for the muti-outputs binary case, the only difference is that the parameters are shaped as a matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$, not a vector anymore :
Algorithm - gradient descent

- 1. Initialize W_0 (e.g, randomly)
- 2. While not converged $\implies$ *update* $\mathrm{W}_k \leftarrow W_{k-1} - \eta \nabla_W R(W_{k-1})$

Thus the gradient will also be a matrix

$$\nabla_W R(W_{k-1}) = \sum_{i=1}^{N} x_i(\hat{y}_i - y_i)^T \quad \in \mathbb{R}^{(D+1)\times C}$$

Prediction

At test time, given a new input sample x, the probability for any class k is computed via the softmax function as

$$\hat{y}^{(k)}(x) = \frac{\exp((w_{(k)}^*)^T x)}{\sum_{j=1}^{C} exp((w_{(j)}^*)^T x)}$$

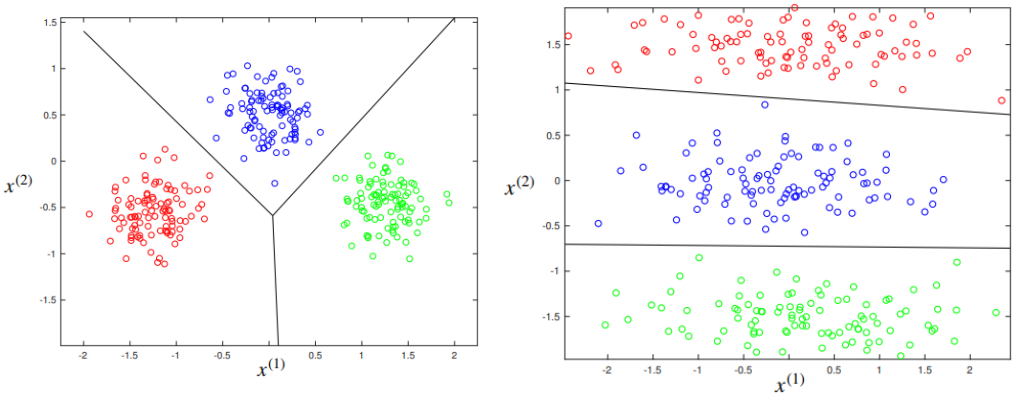
The final class label is then predicted as

$$k = \underset{j}{\operatorname{argmax}}\, \hat{y}^{(j)}(x)$$

That is, as in least-square classification, the label is the index (position) of the maximum value in the vector $\hat{y}$.

Example

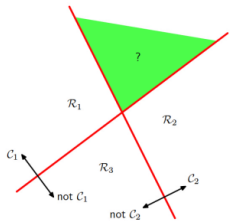
2D inputs, 3 classes: Logistic regresison results



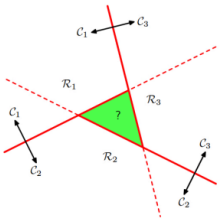
2.4.11 Dealing with multiple classes

SVM cannot be extended to the multi-class scenario.
Instead one can use several binary classifier (e.g, one-vs-rest or one-vs-one, however leaves ambiguities and inconsistencies as a sample can be classified being from more that one class at a time).

- E.g., one-vs-rest: Classify each individual category vs all the other ones jointly



- E.g., one-vs-one: Train one classifier for every pair of classes

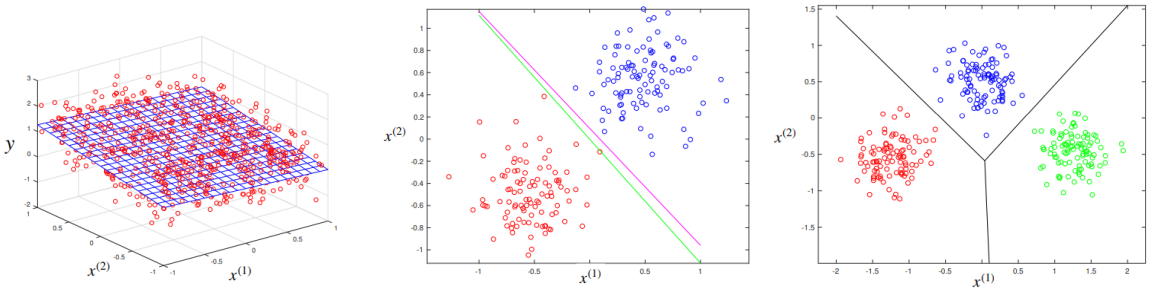


2.4.12 Comparing linear classifiers

We have now seen 3 different linear(multi-class) classifiers: Least-square classification, Logistic regression, SVM.
In practice, one almost never uses the least-square one (only for pedagogiqual standpoint).
Depending on the data to classify, we may use Logistic regression or SVM as they are both better than the other one depending of the data.

2.4.13 Recap - and new challenge

Linear regression : Fit a linear model to the observations.
Binary linear classification: Use a linear model to separate the positive and negative samples.
Multi-class linear classification: Use a linear model to separate the classes, via a one-hot encoding of the label.



Unfortunately, no linear model can directly fit non linear data, so we will go from linear to non linear models.

2.5 Non-linear Model

2.5.1 Histograms

A histogram in $\mathbb{R}^D$ is a $D-$dimensional vector such that

$$x^{(d)} \in [0,1], \forall 1 \leq d \leq D \quad \sum_{d=1}^{D} x^{(d)} = 1$$

Given a vector in $\mathbb{R}^D$ whose values are all non-negative(and at least one is strictly positive), one can obtain a histogram by dividing each value by the sum of values.

2.5.2 k-Nearest neighbor method

We assign the label/value of the most common nearest neighbor.
For $classification$ it consists to give to similar data samples the same label, and for $regression$, the same associated value.

k-NN: Algorithm

Classification:

1. Compute the distance between the test sample x and all training samples $\{x_i\}$
2. Find the k sample $\{x_{NN1}, \ldots, x_{NNK}\}$ with the minimum distances
3. Find the most common label among these k nearest neighbors (majority vote)
4. Assign the corresponding label/value $\mathbf{y}_{MV}$ to the test sample

If several labels appear the same number of times among the k-NN, one strategy consists of taking the one whose samples have the smallest average distance to the test sample.

Regression:

1. Compute the distance between the test sample x and all training samples $\{x_i\}$
2. Find the k sample $\{x_{NN1}, \ldots, x_{NNK}\}$ with the minimum distances
3. Compute the value $\hat{y}$ for the test sample based on that of these $k-$NN

k-NN: Properties

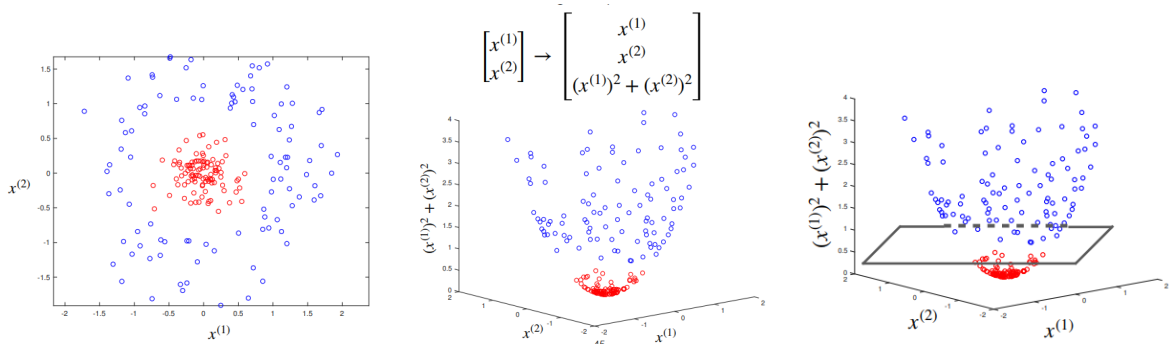
- The results may change for different distance functions. (assigning to a different class).
- The k-NN **doesn't train a model** (no learning), it only keep the data from the training, and use them to predict the label for the test data.
- This is refered to as a **non-parametric** learning method (no parameters to optimize during training) and k is said to be a $hyper-parameter$, set manually.
  So we need: good data representation, a distance function and a given value k $\rightarrow$the results **depends** on the value k.
- The features have to be **normalized** to compute the distance (all features to the same weight).
- It is **computationally expensive** as for each test sample, we need to compute the distances to all training samples. To avoid this problem we could approximate the nearest neighbor by using: Hashing or kd-trees.
- **Curse of dimensionality**, in the real world data tends to be very high dimensional (e.g., for an image of 536 x 356 in 3D $x_i \in \mathbb{R}^{536 \cdot 356 \cdot 3}$), in such dimension, all points tend to be far apart.

2.5.3 Feature expansion

To handle some non linear data, we could use a polynomial curve that transform the input and separate our classes.

And then we can apply a linear model to the resulting variable to find the decision boundary :

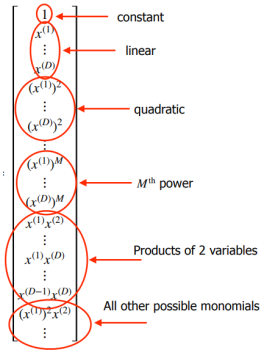
Example



So this consists, of performing a **change of variable** $x \rightarrow \phi(x)$.

In general for $D-$ dimensional inputs, we can obtain a large $\phi(x)$.

ϕ(x) is found by mapping the samples from different classes in two different direction.



Example

$$class1 : |x| < 1.2 \quad class2 : |x| > 2.0$$

Then : $x^2 - 1.44 \leq \phi(x) \leq x^2 - 4$

For class that need a polynomial of degree bigger than 2, what we can do, is to look where we want our gradient to vanish after the application of $\phi(\cdot)$. Then we start with $(\phi(x))'$ and integrate it to find the appropriate $\phi(x)$.

Example if our gradient vanishes at x = -2, 0, 2

$$(\phi(x))' = (x+2)(x-0)(x-2) = x(x^2-4) = x^3 - 4x \implies \phi(x) = \int \phi(x)' dx$$

The resulting expanded features can then be used in any linear alogrithm that we have seen previously.

As the polynomial feature expansion uses a mapping from $x \in R^D$ to another representation $\phi(x) \in \mathbb{R}^F$, where $F \gg D$, we can use a linear model in this new space and write

$$\hat{y}_i = w^T \phi(x_i)$$

where now $w \in \mathbb{R}^F$.

To use it with linear regresion, we rewrite the least-square loss like this

$$\min_w \sum_{i=1}^{N} (w^T \phi(x_i) - y_i)^2 \quad w \in \mathbb{R}^F (\text{not } \mathbb{R}^{D+1} \text{ anymore})$$

This new function is still convex, and its gradient is given by:

$$\nabla R = 2 \sum_{i=1}^{N} \phi(x_i)(\phi(x_i)^T w - y_i)$$

Setting it to zero we get a $F \times F \cdot F \times 1$ matrix

$$\left(\sum_{i=1}^{N} \phi(x_i)\phi(x_i)^T \right) w^* = \sum_{i=1}^{N} \phi(x_i) y_i$$

Then we group the transformed inputs $\{\phi(x_i)\}$ and the outputs $\{y_i\}$ in a matrix vector of the form

$$\phi = \begin{pmatrix} \phi(x_1)^T \\ \vdots \\ \phi(x_N)^T \end{pmatrix} \in \mathbb{R}^{N \times F} y = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

Then, we have

$$w^* = (\phi^T \phi)^{-1} \phi^T y$$

Mapping to a higher-dimensional space

We can observe, that the mapping $\phi(\cdot)$ always increases the dimension of the input. We do this because increasing the dimensionality allows linear models to represent nonlinear relationships, but only if the feature mapping is nonlinear.

Feature expansion: Properties

It is an effective way to use linear model on non linear data samples, but it requires choosing the degree of the polynomial.

In fact, we could use **any** function on the input (not only polynomials !!).

Lecture 6 — 24-03-2026: Overfitting, Cross-validation and Regularization

2.5.4 Model complexity

When using non linear model we should care about the model complexity, because by using those type of model we have added flexibility, that can make the model too complex fot the task at hand.

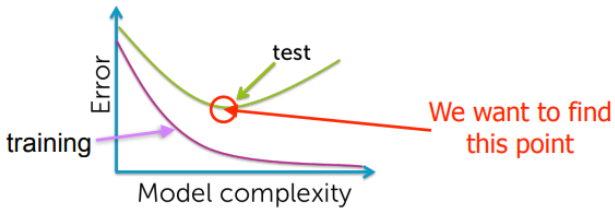
E.g,: a high degree polynomial is more complex than a low degree one.

2.5.5 Overfitting vs Underfitting

Overfitting: when the error on the test data is larger than the training data.

Underfitting: when the error is large on *both* training and test data.

The general phenomenon look like this:

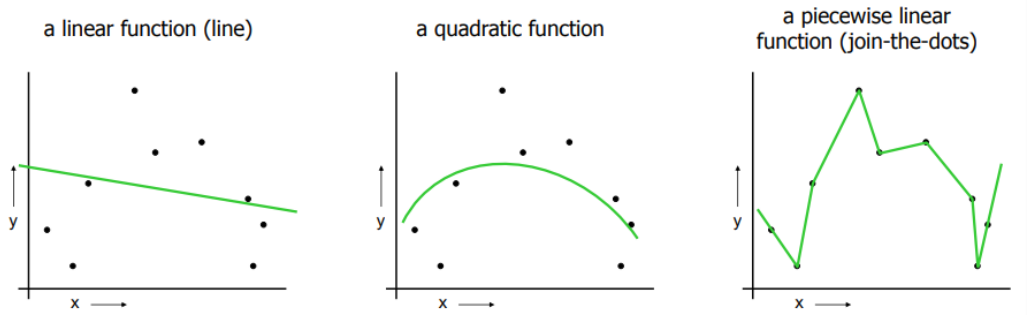


Thus we want to find the point that minimize the test data error (model selection), without having prior access to the test data.

To do that we could use **Cross-validation**.

2.5.6 Cross-validation

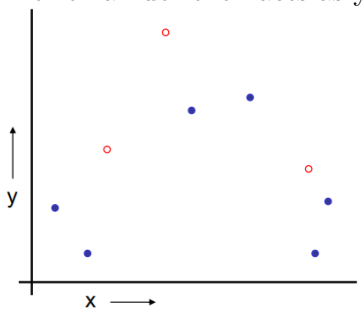
First let's start with an example of a regression problem with $1D$ input x, $1D$ output y, and let's consider 3 different models of increasing complexity:



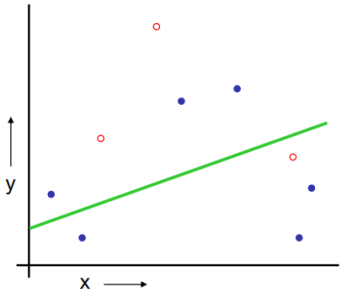
Here the best models is the quadratic function, because we have non prior knowledge of the test data (only the same distribution), hence the piecewise linear function will probably work poorly on it (it is an overfitted model as it goes through each point of the training set but it is completely wrong between those points).

The validation set method

1. Randomly choose, e.g., 30% of the data to form a validation set
2. The remainder then acts as your new training set

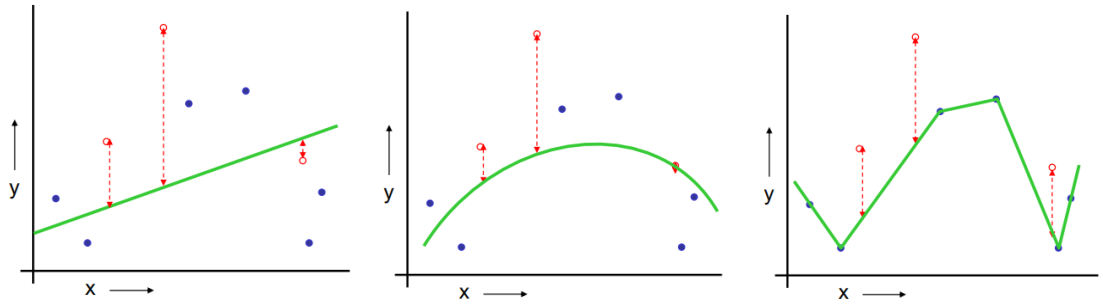


3. Perform regression on this new training set only



E.g., Fit a line

4. Estimate the performance on the test data using the validation set



Upsides:

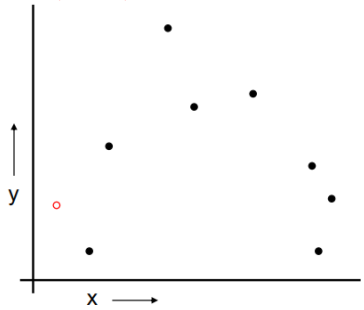
- Very simple
- We can choose the model based on the validation error

Downside: Unreliable estimate of future performance.

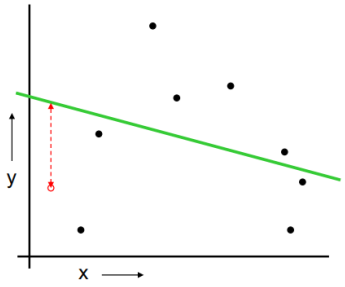
Leave-one-out cross-validation (LOOCV)

For $i = 1, \ldots, N$

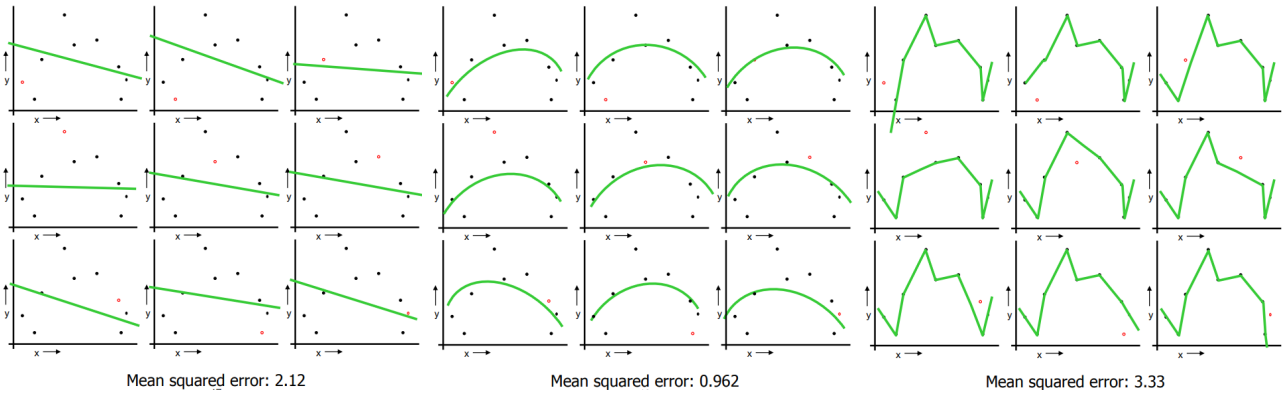
1. Let (x_i, y_i) be the ith sample



2. Temporarily remove (x_i, y_i) from the training set
3. Perform regression on the remaining $N - 1$ samples (e.g., fit a line)
4. Compute the error on (x_i, y_i)



When you have done all points, report the average error.

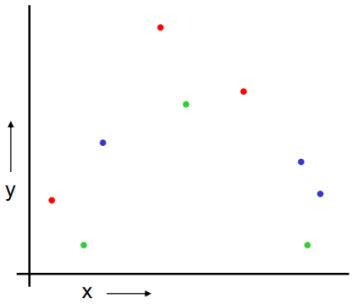


Upside: Doesn't waste data

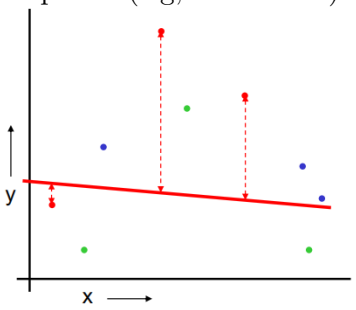
Downside: Expensive

k-fold cross-validation

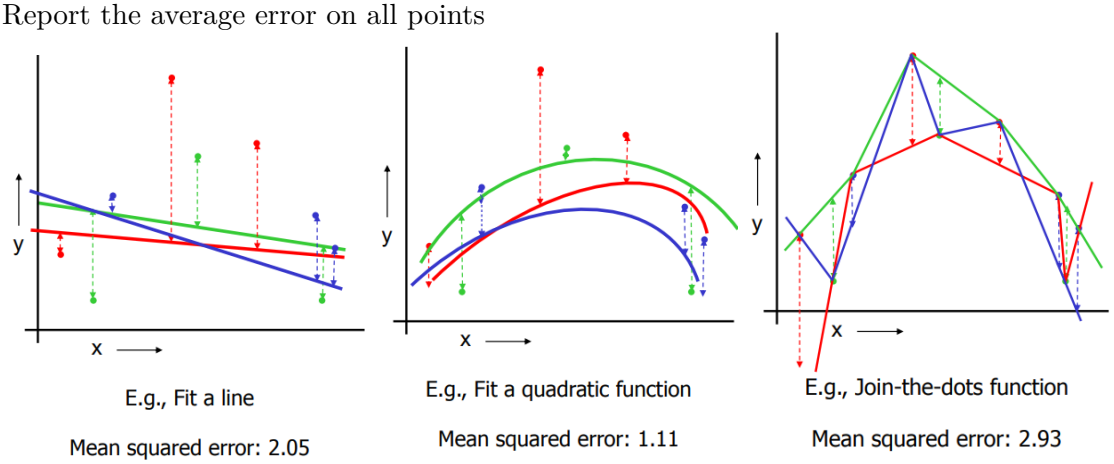
1. Randomly split the dataset into k partitions (e.g., $k = 3$, shown with different colors)



2. For the red partition, train on all the points **not** in the red partition. Compute the errors on the red points (e.g, with a line).



3. Same for the green and the blue
4. Report the average error on all points



Which kind of cross-validation?

	Downside	Upside
Validation-set	Unreliable estimate of future performance	Cheap
Leave-one-out	Expensive	Doesn't waste data
10-fold	Wastes 10% of the data. 10 times more expensive than validation-set	Only wastes 10%. Only 10 times more expensive instead of N times
3-fold	Wastier than 10-fold. Expensivier than validation-set	Slightly better than validation-set
N-fold	Identical to Leave-one-out	

To resume, cross-validation can help to:

- Select the best model among a small number of candidates, but impractical with too many.
- Find the best values for one hyper-parameter (e.g, k in kNN), for this, one treats each value of the hyper-parameter as a separate model.

2.5.7 Overfitting with a linear model

Because the linear model is simple, overfitting occurs fairly rarely.

Nevertheless, if there are fewer training samples than input dimensions ($N \leq D + 1$), then one can always fit a hyperplane that passes exactly through the training samples (overfitted model), it can occur for example when using feature expansion.

Penalizing overfitting

Thus to avoid this problem of overfitting, we will penalize overfitting by adding a regularizer to the empirical risk.

This leads to a new training objective function of the form

$$E(w) = R(w) + \lambda E_w(w)$$

where $R(w)$ is the empirical risk, $E_w(w)$ the regularizer, λ is a hyper-parameter that defines the influence of the regularizer ($\lambda > 0$).

So know we want not only to minimize the error on the data but to keep the weight as low as possible too. By adding this constraints, it reduces the complexity of overfitted model by making them simpler (the higher the hyperparameter λ, the stronger is the constraint, e.g., $\lambda = 1'000'000$ force the weights to be almost zero and results on a flat line ($\hat{y} \approx 0$)).

Example of regularizer

L1 regularizer : sum of the absolute values

$$E_w(w) = \sum_j \left| w^{(j)} \right|$$

L2 regularizer : sum of squares of the weights

$$E_w(w) = w^T w$$

Regularized linear regression

We use the L2 regularizer, this lets us write regularized linear regression as

$$\min_w \sum_{i=1}^{N} (\phi(x_i)^T w - y_i)^2 + \lambda w^T w$$

This remains a convex function (for $\lambda \geq 0$) and so we can still find the solution by zeroing out the gradient.

We have

$$\nabla E = \nabla R + \nabla E_w = 2 \sum_{i=1}^{N} \phi(x_i)(\phi(x_i)^T w - y_i) + 2\lambda w$$

Setting the gradient to zero yields

$$\left(\sum_{i=1}^{N} \phi(x_i)\phi(x_i)^T + \lambda I_F \right) w^* = \sum_{i=1}^{N} \phi(x_i) y_i$$

where I_F is the $F \times F$ identity matrix.

Grouping the inputs in a matrix Φ and the outputs in a vector y, as before, gives the final solution

$$w^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T y$$

Note that now we cannot use the pseudo-inverse anymore (as we add a regularizer).

Regularized linear regression is also referred to *as ridge regression.*

Note that the regularizer affects the optimal value of the parameters, but **not** the mathematical form of the prediction, which remains

$$\hat{y} = (w*)^T \phi(x)$$

Multi-output ridge regression

When dealing with multiple output dimensions, the parameters are expressed in matrix form.

We can then write multi-output ridge regression as

$$\min_w \sum_{i=1}^{N} \left\| W^T \phi(x_i) - y_i \right\|^2 + \lambda \left\| W \right\|_F^2$$

where $\|W\|_F^2$ is the square Frobenius norm, equivalent to the sum of the square of all values in the matrix $\mathbf{W}$.

By zeroing out the gradient we obtain the solution

$$W^* = \left(\Phi^T \Phi + \lambda I_F \right)^{-1} \Phi^T Y$$

Where $\mathbf{Y}$ is an $N \times C$ matrix (with C output dimensions)

Regularization on logistic regression

Regularization may also be added to the cross-entropy loss used to train the logistic regression model.
With a convex regularizer, such as the sum of squares, the overall regularized risk remains convex.
In fact, some implementation of the logistic regression has a regularizer by default (e.g., scikit-learn).

Regularization and SVM

As the SVM training problem is expressed as

$$\min_{w^{(0)},\tilde{w}} \frac{1}{2C}\|\tilde{w}\|^2 + \sum_{i=1}^{N} \max\left(0,\, 1 - y_i \cdot \left(w^{(0)} + \tilde{w}^T x_i\right)\right)$$

Minimizing $\|\tilde{w}\|^2$ (corresponds to the inverse of the margin (distance between the decision boundary and the nearest sample)) is equivalent to maximizing the margin, which already acts as a built-in regularizer !

Lecture 7 — 31-03-2026: Kernel Methods

2.5.8 Kernel function

When using feature expansion, we realised that we could use any function of the input — but two problems arise:

- **Choice**: how do we choose a good mapping $\phi(\cdot)$?
- **Scalability**: when $\phi(x) \in \mathbb{R}^F$ with $F \gg D$ (or even $F = \infty$), explicitly computing $\phi(x)$ and then $\phi(x_i)^T\phi(x_j)$ becomes intractable.

Kernel functions solve both problems at once.
Many machine learning algorithms rely on the **dot product** between inputs:

$$x_i^T x_j$$

This quantity is proportional to the cosine of the angle between the vectors, thus encoding a notion of **similarity**.
When performing **feature expansion**, inputs are mapped through a function $\phi(\cdot)$, leading to:

$$\phi(x_i)^T\phi(x_j)$$

which defines a new similarity measure in the transformed feature space.
Instead of explicitly computing $\phi(\cdot)$, one can define a function: $k(x_i, x_j)$ called a **kernel function**, such that:

$$k(x_i, x_j) = \phi(x_i)^T\phi(x_j) = \|\phi(x)\|^2$$

The mapping $\phi(\cdot)$ may be unknown or computationally expensive, but the kernel function allows us to compute similarities implicitly.

Examples of kernel functions

Polynomial kernel

$$k(x_i, x_j) = (x_i^T x_j + c)^d$$

has two hyper-parameters: The bias c, the degree of the polynomial d.

His corresponding mappings $\phi(\cdot)$ for $c = 1$ and $d = 2$ is

$$\phi(x) = [\ (x^{(1)})^2, (x^{(2)})^2, \sqrt{2}x^{(1)}x^{(2)}, \sqrt{2}x^{(1)}, \sqrt{2}x^{(2)}, 1]$$

And corresponds to the quadratic kernel

$$k(x_i, x_j) = (x_i^T x_j + 1)^2$$

Note that computing the kernel value only requires 3 multiplications, versus 6 when computing the dot product $\phi(x_i)^T \phi(x_j)$.

Gaussian kernel

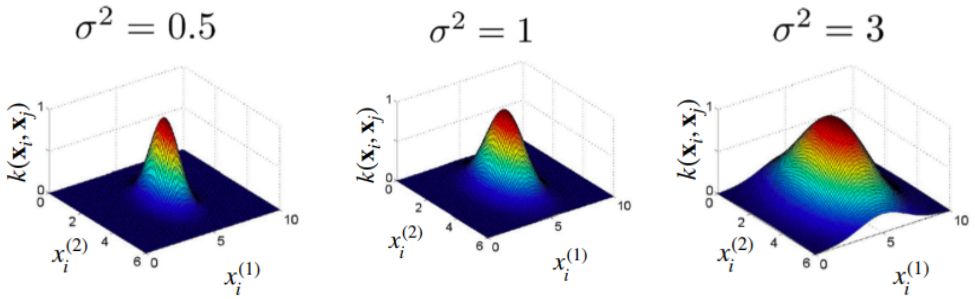
$$k(x_i, x_j) = \exp(-\frac{\|x_i - x_j\|^2}{2\sigma^2})$$

Hyper-parameter : the bandwith σ.

But for this kernel there is no explicit mapping $\phi(\cdot)$ (in theory, this kernel corresponds to a mapping to an infinite dimensional space).

Visualization:

2D inputs, one sample $x_j = 5, 3^T$, and vary x_i



Trick

Since a kernel function corresponds to a similarity, as a dot product, one can replace any dot product between two samples in the solution of an ML algorithm with a kernel function. This trick is called *kernelizing* an algorithm (not only for linear model !).

Kernelization: Notation

The kernel function computes a similarity between two samples, and can be written as $k(x_i, x_j)$.

We could to use the kernel function to encode the similarity between one sample and all traning samples (matrix X):

$$k(X, x_i) = \begin{pmatrix} k(x_1, x_i) \\ k(x_2, x_i) \\ \vdots \\ k(x_N, x_i) \end{pmatrix} \in \mathbb{R}^N$$

Furthermore, we can define the **kernel matrix**, a squared matrix, which each entry (i, j) contains the kernel function evaluate between training sample i and training sample j

$$K = \begin{pmatrix} k(x_1, x_1) & k(x_1, x_2) & \cdots & k(x_1, x_N) \\ k(x_2, x_1) & k(x_2, x_2) & \cdots & k(x_2, x_N) \\ \vdots & \vdots & \vdots & \vdots \\ k(x_N, x_1) & k(x_N, x_2) & \cdots & k(x_N, x_N) \end{pmatrix} \in \mathbb{R}^{N \times N}$$

Kernel regression

To kernelize linear regression we need to rewrite the solution so that $\phi(\cdot)$ only appears inside dot products — which we can then replace by $k(\cdot, \cdot)$, to do so we can make use of the Representer theorem (useful too to kernelize other algorithms).

Representer theorem

- Main Idea: Optimal solutions exist within a finite linear combination of training data
- Goal: Reduce infinite-dimensional problems to finite-dimensional ones

In essence, this theorem states that the minimizer of a regularized empirical risk function can be represented as a linear combination of the points in the high dimensional space. That is, we can write

$$w^* = \sum_{i=1}^{N} \alpha_i^* \phi(x_i) = \Phi^T \alpha^*$$

So instead of searching for a minimizer of the empirical risk in terms of $\mathbf{w}$, we search for one in terms of new variables $\{\alpha_i\}$

Therefore, the empirical risk :

$$R(w) = \sum_{i=1}^{N} (w^T \phi(x_i) - y_i)^2$$

is re-written as

$$R(\alpha) = \sum_{i=1}^{N} (\alpha^T \Phi \phi(x_i) - y_i)^2 = \sum_{i=1}^{N} (\alpha^T k(X, x_i) - y_i)^2$$

(as $\Phi \cdot \phi(x_i)$ is the dot product of all training samples and sample i !)

The gradient of the risk w.r.t α is given by

$$\nabla R(\alpha) = 2 \sum_{i=1}^{N} k(X, x_i)(k(X, x_i)^T \alpha - y_i) = 2KK\alpha - 2Ky$$

with $\mathbf{K}$ the $N \times N$ training kernel matrix, obtained by evaluating the kernel function between all pairs of training samples.

Setting the gradient to 0 yields

$$2KK\alpha^* = 2Ky \iff Ka^* = y$$

which gives the solution

$$\alpha^* = (K)^{-1} y$$

Putting this solution back in the definition of w^* gives us

$$w^* = \Phi^T (K)^{-1} y$$

This solution still depends on Φ, which we argued could be unknown.

Therefore the prediction for a new sample $\mathbf{x}$ is given by :

$$\hat{y} = (w^*)^T \phi(x) = y^T (K)^{-1} \Phi \phi(x) = y^T (K)^{-1} k(X, x)$$

Note that the quantity $y^T (K)^{-1}$ only depends on the training data, and thus can be pre-computed and reuse for every test sample.

Kernel ridge regression

Using the Representer theorem, the regularized empirical risk

$$E(w) = \sum_{i=1}^{N} (w^T \phi(x_i) - y_i)^2 + \lambda w^T w$$

Can be re-written as

$$E(\alpha) = \sum_{i=1}^{N} (\alpha^T \Phi \phi(x_i) - y_i)^2 + \lambda \alpha^T \Phi \Phi^T \alpha = \sum_{i=1}^{N} (\alpha^T k(X, x_i) - y_i)^2 + \lambda \alpha^T K \alpha$$

The gradient becomes

$$\nabla_a E(\alpha) = 2K(K + \lambda I_N)\alpha - 2Ky$$

Setting it to zero yields the solution

$$\alpha^* = (K + \lambda I_N)^{-1} y$$

Kernel ridge regression with multiple outputs

To handle nonlinear problems, we can then again work in a different feature space $\phi(\cdot)$ and write the regularized empirical risk

$$E(W) = \sum_{i=1}^{N} \left\| W^T \phi(x_i) - y_i \right\|^2 + \lambda \left\| W \right\|_F^2$$

And by using the Representer theorem, we can again derive a solution that depend on kernel values $k(x_i, x_j)$ instead of $\phi(\cdot)$.

The solution is given by

$$W^* = \Phi^T (K + \lambda I)^{-1} Y$$

where Φ is the same matrix concatenating the training inputs $\{\phi(x_i)\}$, and $Y \in \mathbb{R}^{N \times C}$ is the matrix stacking the training output vectors.

The prediction vector for a test sample is given as

$$\hat{y} = (W^*)^T \phi(x) = Y^T (K + \lambda I)^{-1} \Phi \phi(x) = Y^T (K + \lambda I)^{-1} k(X, x)$$

Remark

As the results of kernel ridge regression strongly depend on the choice of kernel and of the hyperparameter values for a given kernel, it may be natural to ask ourself how to find good parameters.

The answer is by **cross-validation**. Each kernel and each hyperparameter value corresponds to a different model. We can then separate our training data into training and validation sets, train each model with the new training data, and choose the one that performs the best on the validation data

Advantages and Drawbacks of kernel methods

Advantages:

- Can effectively handle nonlinearly-separable data
- Often have nice solutions (e.g., closed-form, convex)

Drawbacks:

- Depend on the choice of kernel and hyper-parameter(s)
- Computational complexity depends on the number of samples.
  - E.g., for kernel ridge regression, one needs to invert an $N \times N$ matrix, which requires $O(N^3)$ operations

Lecture 8 — 14-04-2026: Data Representations & Processing

2.6 Data representation

2.6.1 Why ?

1. Data heterogeneity: we cannot compare vectors of different dimensions (e.g, subtract $x_i \in \mathbb{R}^{200 \cdot 200 \cdot 3}$ and $x_j \in \mathbb{R}^{210 \cdot 223 \cdot 3}$, or compare speech samples of different length).
2. Data size: A single, fairly small image is already a very long vector, so what about the size of a complete data set ?
3. Data noisiness: Not every piece of data encodes valuable information (E.g, for image recognition/text categorization: not all pixels/words are indicative of the class).
4. Some representations are better suited to certain tasks than others.

The resulting representations can be directly used as input to an ML algorithm (applicable to all the methods we have seen so far, but in general to most machine learning algorithms).

2.6.2 Attributes

Attributes could be of different types:

- Categorical (e.g., sex: M, F, I)
- Real (continuous) (e.g., length in mm)
- Integer (discrete)

2.6.3 Bag of Words

To represent text document we will use bag of words, it consists of a mapping of each words to the number of it's occurence in the text.

Constructing a dictionary

1. Consider only the words that appear in the dataset (n samples).
2. Do not consider the too common words, e.g., "the", "a", "to", "it", "is", "for",...

Vector representation of BoW

With D words in the dictionary, each sample can be representd as a vector in $\mathbb{R}^D$.

Histograms

The final BoW representation is a histogram such that

$$x_1^{(d)} \in [\,0, 1]$$

$$\sum_{d=1}^{D} x_1^{(d)} = 1$$

Example 2 samples :
(1) John likes to watch movies. Mary likes movies too.
(2) John also likes to watch football games.
Corresponding counts :
BoW1 = { "John":1, "likes":2, "to":1, "watch":1, "movies":2, "Mary":1, "too":1};
BoW2 = {"John":1, "also":1, "likes":1, "to":1, "watch":1, "football":1, "games":1};
Our dictionary is all the words appearing in the 2 samples after removing "to" :
"John", "likes", "watch", "movies", "Mary", "too", "also", "football", "games"
The vector representation of the BoWs are ($D = 9$) :

$$\tilde{x}_1 = \begin{pmatrix} 1 & 2 & 1 & 2 & 1 & 1 & 0 & 0 & 0 \end{pmatrix}$$

$$\tilde{x}_2 = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}$$

And the histogram are

$$x_1 = \begin{pmatrix} 1/8 & 1/4 & 1/8 & 1/4 & 1/8 & 1/8 & 0 & 0 & 0 \end{pmatrix}$$

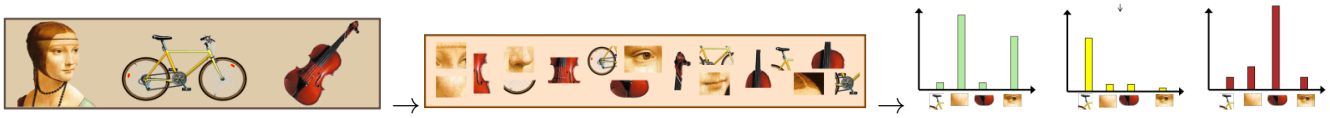
$$x_2 = \begin{pmatrix} 1/6 & 1/6 & 1/6 & 0 & 0 & 0 & 1/6 & 1/6 & 1/6 \end{pmatrix}$$

2.6.4 Bag of visual words

For images we could adapt the BoW, as objects are made of common parts/patches (can be thought of as visual words), we could use them instead of words.

2.6.5 Visual dictionary

1. Extract patches from all images
2. Group the patches based on similarity to optain prototypes that act as dictionary words (The center of each cluster (group) is taken as a visual word)
3. For each patch in an image
   - find the nearest visual word
   - add one to the corresponding element in a BoW vector
4. Normalize the BoW vectors to obtain histograms



2.6.6 Data pre-processing

Even with a good representation, the data might benefit from being pre-processed (e.g., some attribute values are missing/noisy or the feature dimensions are of incommensurate magnitudes).

Missing data

Attributes values that are not recorded (e.g., the Air Quality UCI dataset contains missing values, which are tagged with a -200 value)

Potential solutions:

- Ignore the corresponding sample
- Fill in the missing values with a global constant
- Fill in the missing values with the corresponding attribute average over the dataset
- Fill in the missing values with the corresponding attribute average over the samples belonging to the same class

Noisy data

Noise: Random measurement error

Incorrect values : Faulty data collection, Data entry problems, Data transmission problems

Other problems : Duplicate entries, Inconsistent data

Cleaning noisy data

1. Cleaning by **binning**:
   For each attribute:
   (a) Sort the data
   (b) Partition it into bins
   (c) Smooth the data by taking the bin means or medians
   E.g:
   - **Data :** 0, 4, 12, 16, 16, 16, 18, 24, 26, 28
   - **Equal width**

Bin 1: 0, 4	$[-\infty, 10)$
Bin 2: 12, 16, 16, 18	$[10, 20)$
Bin 3: 24, 26, 28	$[20, +\infty)$

   - **Equal frequency**

Bin 1: 0, 4, 12	$[-\infty, 10)$
Bin 2: 16, 16, 18	$[14, 21]$
Bin 3: 24, 26, 28	$[21, +\infty)$
2. Clustering : Detect outliers as the clusters with too few elements and remove these samples
3. Leverage human knowledge: remove values that have non-sense (e.g., Blood pressure cannot be 0, salary cannot be negative).

2.6.7 Data normalization

Goal: Scale each individual attribute/feature dimension to fall within a specified range.

Min-max normalization

For each feature dimension d, compute the normalized feature

$$\tilde{x}_i^{(d)} = \frac{x_i^{(d)} - x_{min}^{(d)}}{x_{max}^{(d)} - x_{min}^{(d)}}$$

where $x_{min}^{(d)} = \min_{i=1}^{N} x_i^{(d)}$ and $x_{max}^{(d)} = \max_{i=1}^{N} x_i^{(d)}$

z-score normalization

For each feature dimension d, compute the normalized feature

$$\tilde{x}_i^{(d)} = \frac{x_i^{(d)} - \mu^{(d)}}{\sigma^{(d)}}$$

where $\mu^{(d)} = \frac{1}{N} \sum_{i=1}^{N} x_i^{(d)}$

$\sigma^{(d)} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (x_i^{(d)} - \mu^{(d)})^2}$

max normalization

For each feature dimension d, compute the normalized feature

$$\tilde{x_i}^{(d)} = \frac{x_i^{(d)}}{x_{max}^{(d)}}$$

where $x_{max}^{(d)} = \max_{i=1}^{N} \left| x_i^{(d)} \right|$

decimal scaling normalization

For each feature dimension d, compute the normalized feature

$$\tilde{x_i}^{(d)} = \frac{x_i^{(d)}}{10^k}$$

where k is the smallest integer such that $\max_{i=1}^{N} \left| \tilde{x}_i^{(d)} \right| \leq 1$

Remark

For all data processing techniques, the normalized (or cleaned) features are then used as input to an ML algorithm

- Normalization is not aways necessary, and may or may not help depending on the data at hand and on the chosen ML algorithm

Data representations and data cleaning/normalization acts on the individual samples

- Although the normalization statistics come from the whole training set

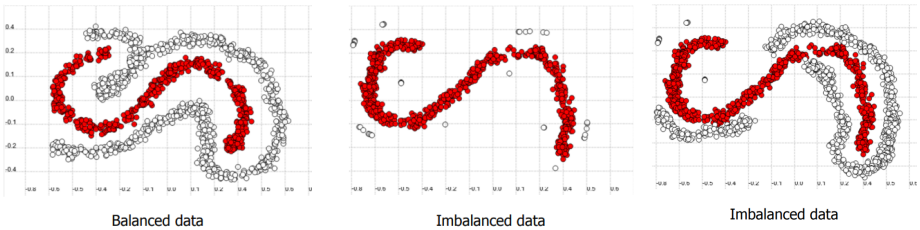
Nevertheless, even with good representations, the data set itself might make from learning more difficult (imbalanced data).

2.6.8 Imbalanced data

If a class is very majoritary, the classifier learn to predict always this class (high accuracy but predict poorly on the minoritary class) instead of learning patterns.

Two type of imbalance :

- Between-class imbalance : one class dominates
- Within-class imbalance : bad spatial repartition of the samples inside a class



Learning with imbalanced data

Two main approaches:

- One that act on the data: Sampling Methods
- One that act on the cost function: Cost-Sensitive Methods

Sampling Methods

Two ways to compensate the lack of data:

- Increase the Dataset, by generating new datapoints for the smallest class
- Decrease the Dataset, by removing redundant datapoints from the largest class

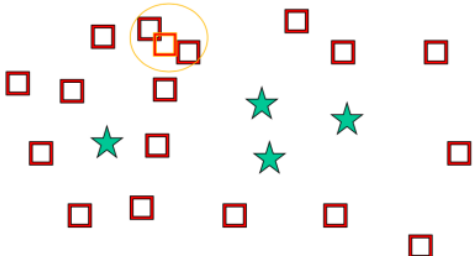
Decrease the data: Undersampling

Naive (random) undersampling:

- Randomly remove samples from the largest class until you obtain a balanced data set
  - Drawback: May remove important samples

More clever undersampling:

- Remove the redundant samples
  - Good if the class to undersample is densely populated (e.g., lowdimensional representation because of the curse of dimensionality)



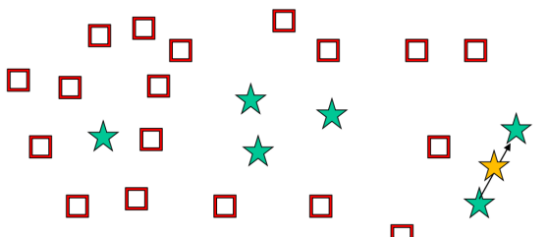
Increase the data: Oversampling

Naive (random) oversampling:

- Replicate the data samples from the smallest class until you obtain a balanced data set
  - Drawback: May lead to overfitting

More clever oversampling:

- Create new data samples based on neighboring existing ones
  - This may create noisy samples



Empirical risk

Given N training samples $\{(x_i, y_i)\}$, the empirical risk is defined as

$$R(\{x_i\}, \{y_i\}, w) = \frac{1}{N} \sum_{i=1}^{N} l(\hat{y}_i, y_i)$$

where $\{x_i\}$ and $\{y_i\}$ are the sets of training inputs and labels, respectively.

In general, each loss term $l(\hat{y_i}, y_i)$ has the same influence, with imbalanced data, this means that most of the empirical risk comes from the largest class.

Sample re-weighting

A simple solution to this problem consist of re-weighting the samples, to put more emphasis on the rare class.

This gives an empirical risk of the form

$$R(\{x_i\}, \{y_i\}, w) = \frac{1}{N} \sum_{i=1}^{N} \beta_i l(\hat{y}_i, y_i)$$

where β_i is the weight for sample i.

This strategy is quite general. E.g., it applies to least-square classification, logistic regression, SVM (in the three cases, since the weights are fixed during optimization, the respective optimization problems remain convex and can thus be solved to optimality).

From handcrafted representations to learned ones

As the representations and data processing strategies we have discussed so far (e.g., feature expansion, kernel methods, bag of (visual) words, …) were all manually designed, it doesn't guarantee that they are the best representations for the task at hand.

Ideally, one would therefore like to learn the representation jointly with the classifier, it leads us to **neural networks**.

2.7 Deep Learning

So far, all the representations we have used were manually designed. The key idea of neural networks is to **learn the representation jointly with the classifier or with the linear model that comes afterward (same for regression)**, removing the need for hand-crafted features.

The model is built as a stack of **layers**, each composed of **neurons** connected to every neuron of the previous layer. Each neuron computes a weighted sum of its inputs, then passes the result through a **non-linear activation function** — without this non-linearity, the whole network would collapse into a single linear model. Thus we learn jointly all the weights of the network in a single training pass: the weights of the hidden layers define the representation of the data, while the weights of the last layer correspond to the final linear model.

Depending on the output layer and the loss function used, the same architecture handles both **regression** (square loss) and **classification** (softmax + cross-entropy).

This framework is referred to as *neural network*, *multilayer perceptron (MLP)*, or *deep learning* when there are at least two hidden layers.

2.7.1 Parametric feature expansion

As we have seen, parametric feature expansion isn't optimal as we need to define manually the polynomial expansion $\phi(x)$ and we would like to learn the representation conjointly with the model. To achieve this goal we have to add **nonlinearity** in the process.

We will use a nonlinear function (**activation function**) $f : \mathbb{R} \to \mathbb{R}$.

2.7.2 Hidden representation

Let's first apply a linear transformation to the input (pre-activation)

$$a_{(1)} = W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix} = \begin{pmatrix} W_{(1)}^{(1,0)} + W_{(1)}^{(1,1)} x \\ W_{(1)}^{(2,0)} + W_{(1)}^{(2,1)} x \\ \vdots \\ W_{(1)}^{(M,0)} + W_{(1)}^{(M,1)} x \end{pmatrix} = \begin{pmatrix} a_{(1)}^{(1)} \\ a_{(1)}^{(2)} \\ \vdots \\ a_{(1)}^{(M)} \end{pmatrix}$$

Then we apply f elementwise to $a_{(1)}$, and it gives our complete $M-$dimensional *hidden representation* (analogous to a learn $\phi(x)$ of the feature expansion)

$$x \to \begin{pmatrix} f(a_{(1)}^{(1)}) \\ f(a_{(1)}^{(2)}) \\ \vdots \\ f(a_{(1)}^{(M)}) \end{pmatrix} = f\left(W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix} \right) = z_{(1)} \in \mathbb{R}^M$$

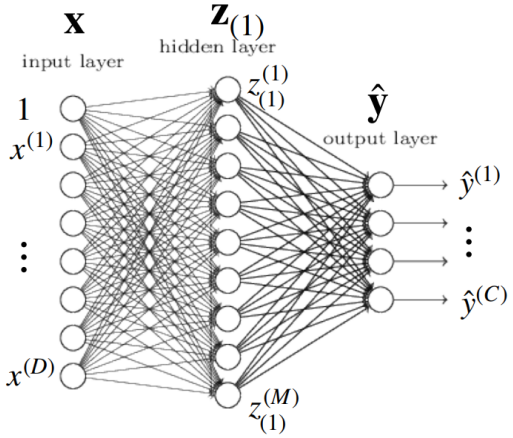
each of its M elements is referred to as hidden unit.

Complete model

We can then perform prediction by applying a second linear model to the hidden representation. This gives

$$\hat{y} = w_{(2)}^T z_{(1)} = w_{(2)}^T f\left(W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix}\right) = w_{(2)}^T \begin{pmatrix} f(a_{(1)}^{(1)}) \\ f(a_{(1)}^{(2)}) \\ \vdots \\ f(a_{(1)}^{(M)}) \end{pmatrix} = \sum_{j=1}^{M} w_{(2)}^{(j)} f\left(W_{(1)}^{(j,0)} + (W_{(1)}^{(j,1)}\right)$$

And the resulting simple artificial neural network with one hidden layer can be depicted as shown below.



Because we use a matrix-vector product for each layer, the layers are called *fully-connected*

- Each hidden dimension (hidden unit) $z_{(1)}^{(j)}$ depends on all input dimensions $x^{(1)}, ...x^{(D)}$
- Each output dimension $\hat{y}^{(k)}$depends on all hidden units $z_{(1)}^{(1)}, ..., z_{(1)}^{(M)}$

In short, it means that each output neurons is connected to each input neurons.

Extension to multi-dimensional input and output

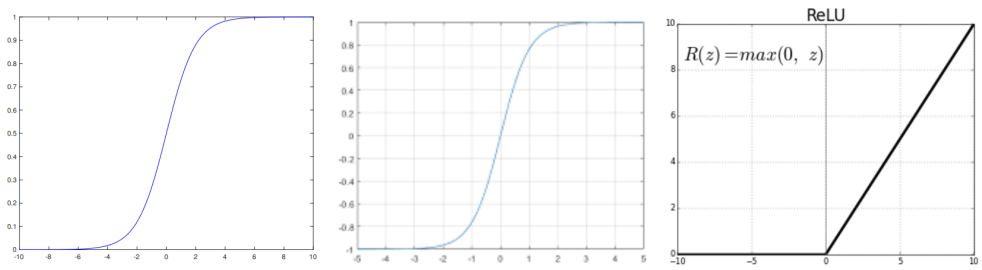
The formulation above assumed $x \in \mathbb{R}$ and $\hat{y} \in \mathbb{R}$. In the general case:

- **Input** $x \in \mathbb{R}^D$: the bias is absorbed into x by appending a 1, so $W_{(1)} \in \mathbb{R}^{(D+1)\times M}$, and the hidden representation becomes:
$$z_{(1)} = f\left(W_{(1)}^T x\right)$$
- **Output** $\hat{y} \in \mathbb{R}^C$: we replace the vector $w_{(2)}$ by a matrix $W_{(2)} \in \mathbb{R}^{M\times C}$, giving:
$$\hat{y} = W_{(2)}^T z_{(1)} = W_{(2)}^T f\left(W_{(1)}^T x\right)$$

Activation function: Examples

- Sigmoid: $f(a) = \frac{1}{1+exp(-a)}$
- Hyperbolic tangent: $f(a) = tanh(a)$
- Rectified Linear Unit (ReLU)

$$f(a) = \begin{cases} a & \text{if } a > 0 \\ 0 & \text{otherwise} \end{cases} = \max(a, 0)$$



2.7.3 Multilayer Perceptron (MLP)

Architecture

With a single hidden layer we can already model complicated functions, but we can stack multiple layers.

In essence, each hidden layer takes as input the output of the previous layer.

For the first hidden layer:

$$z_{(1)} = f\left(W_{(1)}^T x\right)$$

For any subsequent layer l:

$$z_{(l)} = f\left(W_{(l)}^T z_{(l-1)}\right)$$

For a network of L layers, the output is:

$$\hat{y} = W_{(L)}^T z_{(L-1)}$$

This process of going from input to output is called the **forward pass**. We can write the prediction as a function of all parameters $W = \{W_{(l)}\}$:

$$\hat{y} = W_{(L)}^T f\left(W_{(L-1)}^T f\left(W_{(L-2)}^T f\left(\cdots f\left(W_{(1)}^T x\right)\cdots\right)\right)\right) = \hat{y}(W)$$

As we have seen with the $1D$ example, because we use a matrix-vector product at each layer, the layers are called **fully-connected**: each hidden unit $z_{(l)}^{(j)}$ depends on all units of the previous layer.

Universal approximation

Any continuous function can be approximated by a linear combination of translated and scaled activation functions. This holds for ReLU and other activation functions under mild assumptions.

2.7.4 Training an MLP

Empirical risk minimization

Training is done by minimizing an empirical risk over all parameters W:

$$R(W) = \frac{1}{N}\sum_{i=1}^{N} \ell(\hat{y}_i(W), y_i)$$

For regression, the typical loss is the **square loss**:

$$R(W) = \frac{1}{N}\sum_{i=1}^{N} \|\hat{y}_i(W) - y_i\|^2$$

Since the problem is non-convex and has no closed-form solution, learning is done by **gradient descent**.

Backpropagation

Because the empirical risk is an average over the training samples, its gradient will also be an average of gradients.
Thus we can focus on a single loss term $\ell_i = \ell(\hat{y}_i, y_i)$ where

$$R(W_\ell) = \frac{1}{N}\sum_{i=1}^{N} \ell(\hat{y}_i(W), y_i) = \frac{1}{N}\sum_{i=1}^{N} \ell_i$$

For the k-th dimension of the output of any layer l, let us define the pre-activation:

$$a_{(l)}^{(k)} = \sum_j W_{(l)}^{(k,j)} z_{(l-1)}^{(j)}$$

which denotes the dot product between the kth column of $W_{(l)}$ and $z_{(l-1)}$.

What we aim to compute is the derivative of the loss function ℓ_i w.r.t. any parameter $W_{(l)}^{(k,j)}$
By the chain rule:

$$\frac{\partial \ell_i}{\partial W_{(l)}^{(k,j)}} = \underbrace{\frac{\partial \ell_i}{\partial a_{(l)}^{(k)}}}_{\delta_{(l)}^{(k)}} \cdot \underbrace{\frac{\partial a_{(l)}^{(k)}}{\partial W_{(l)}^{(k,j)}}}_{z_{(l-1)}^{(j)}}$$

Defining $\delta_{(l)}^{(k)} = \frac{\partial \ell_i}{\partial a_{(l)}^{(k)}}$, we get:

$$\frac{\partial \ell_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} \cdot z_{(l-1)}^{(j)}$$

For the last layer L (square loss, no final activation):

$$\delta_{(L)}^{(k)} = 2(\hat{y}_i^{(k)} - y_i^{(k)})$$

For earlier layers $l < L$, $\delta_{(l)}$ is computed from $\{\delta_{(l+1)}^{(j)}\}$, $W_{(l+1)}$, and f'.

In other words, given $\delta_{(l+1)}$, we can compute $\delta_{(l)}$. So because we can explicitly compute $\delta_{(L)}$ for the last layer, we can in turn compute any $\delta_{(l)}$.

This gives us a way to compute the gradient at any layer, starting from the last one and it's called the **backpropagation** algorithm:

1. Propagate x_i forward through the network and at each layer compute and store $z_{(1)}, \ldots, z_{(L-1)}$
2. Compute $\delta_{(L)}$ from the loss
3. Propagate $\delta_{(L)}$backward to obtain each $\delta_{(l)}$
4. At each layer, compute $\frac{\partial \ell_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} z_{(l-1)}^{(j)}$
   where z_{l-1} has been computed during the forward pass.

So the training consists of repeating this until convergence:

1. **Forward pass :** propagate x_i through each layer to compute and store $z_{(l)}$
2. **Backpropagation :** compute each $\delta_{(l)}$, starting from the last layer, and gives us the gradient $\frac{\partial \ell_i}{\partial W_{(l)}^{(k,j)}}$
3. **Gradient descent :** use those gradients to update the weights W_i

2.7.5 Optimization

Gradient descent and SGD

The standard gradient descent update at iteration k:

$$W_k \leftarrow W_{k-1} - \eta \frac{1}{N} \sum_{i=1}^{N} \nabla_W \ell_i(W_{k-1})$$

This is expensive for large N as at each iteration we have to compute the gradient over all the samples of the dataset.

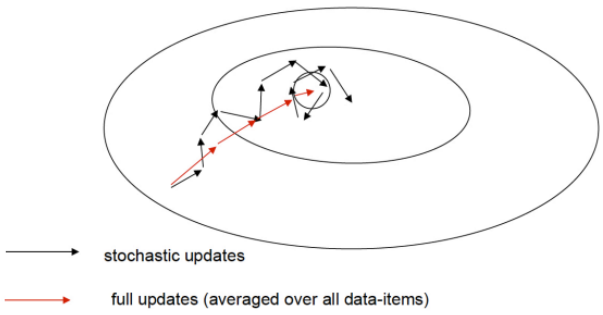
Stochastic gradient descent (SGD) uses a single sample per iteration:

$$W_k \leftarrow W_{k-1} - \eta \nabla \ell_{i(k)}(W_{k-1})$$

But as the gradient obtained by a single sample might be a poor approximation of the true, complete gradient, we use mini-batches B samples for a more reliable gradient estimate:

$$W_k \leftarrow W_{k-1} - \eta \sum_{b=1}^{B} \nabla \ell_{i(b,k)}(W_{k-1})$$

SGD introduces noise in the updates, which can help escape bad local minima, but causes oscillations that may require decreasing η.



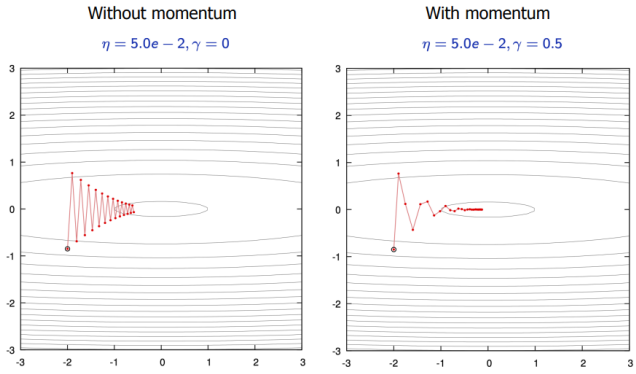
SGD extensions

Momentum Adds inertia to the parameter updates:

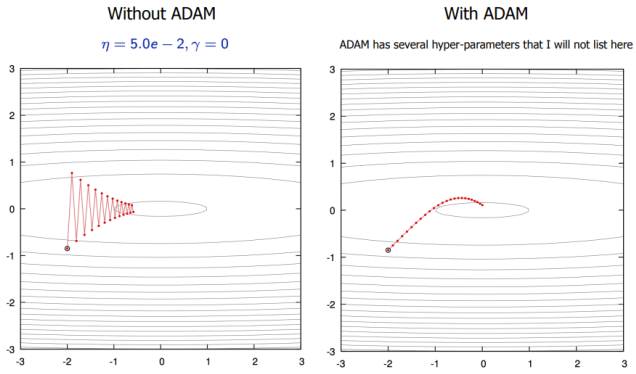
$$\nu \leftarrow \gamma\nu + \eta\nabla R(W), \qquad W \leftarrow W - \nu$$

with $\gamma > 0$, momentum

- Accelerates if the gradient does not change too much
- Dampens (slow down) the oscillations in narrow valleys



Adaptive learning rate Methods such as **ADAM** maintain a moving average per coordinate to rescale each dimension of the gradient separately.

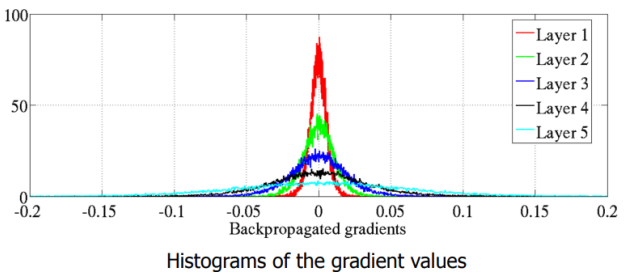


2.7.6 Practical training issues

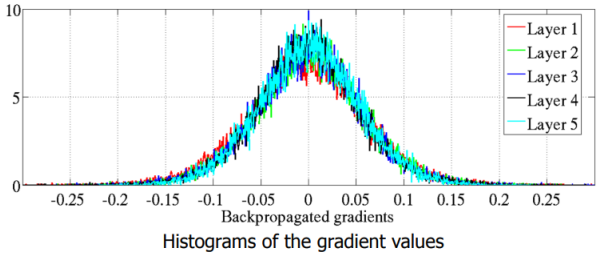
Gradient vanishing and initialization

The **vanishing gradient** problem arises when backpropagating through deep networks: at each layer, the gradient is multiplied by $f'(a)$, the derivative of the activation function. Since sigmoid and tanh saturate for large $|a|$, their derivative becomes close to 0 in these regions. With poor weight initialization, activations tend to fall in these saturating regions, causing the gradient to shrink exponentially as it propagates back through the layers — meaning the first layers receive almost no learning signal.

To overcome this we could use *Xavier (Glorot) initialization*: a weight initialization strategy where the range of the initial, random values of the weights is layer dependent.



Histograms of the gradient values



Histograms of the gradient values

Regularization

Deep networks have many parameters and can overfit. Solutions include:

- L_1 / L_2 regularization (seen previously)
- **Dropout**: randomly remove units and their connections during training

2.7.7 From regression to classification

For classification, the output layer uses the **softmax** function to produce class probabilities:

$$\hat{y}^{(k)} = \frac{\exp\left(w_{(L)(k)}^T z_{(L-1)}\right)}{\sum_{j=1}^{C} \exp\left(w_{(L)(j)}^T z_{(L-1)}\right)}$$

where $w_{(L)(j)}$ indicates the jth column of $W_{(L)}$.
The empirical risk is then the **cross-entropy**:

$$R(W) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{C} y_i^{(k)} \ln \hat{y}_i^{(k)}$$

And the backpropagation then requires computing the derivative of the cross-entropy w.r.t each $\hat{y}^{(y)}$ and the derivative of the softmax function w.r.t each $W_{(L)}^{(j,k)}$.

2.7.8 Working with images

Treating an image as a big vector seems counter-intuitive. An image of a reasonable size yields a huge vector (e.g., $x_i \in \mathbb{R}^{536 \cdot 356 \cdot 3} = \mathbb{R}^{572448}$), thus a huge amount of parameters in an MLP, but also:

- Small image regions have similar properties and should not be processed independently.
- A translation should not affect the results.

The goal is therefore to **reduce the number of parameters and improve robustness**. This motivates the use of **Convolutional Neural Networks (CNNs)**.

2.7.9 Convolutional Neural Network (CNN)

A CNN is a neural network designed to exploit the local structure of its input: nearby elements are more related than distant ones (e.g., in an image, pixel (i, j) is more related to pixel $(i, j+1)$ than to a more distant one), and the same patterns can appear at different locations. This makes CNNs naturally suited for images, but also for any spatially structured signal such as audio (1D) or volumetric data (3D).

2D convolution

A 2D convolution applies a small filter (kernel) of size $w \times w \times C$ to an input of size $W \times W \times C$ by sliding it across all spatial positions (moving horizontally and vertically – hence the term **2D**, although the operation is internally 3D). At each spatial position, the dot product between the kernel and the overlapping $w \times w \times C$ region of the input is computed, summing over all channels to produce a single scalar value – measuring how much the local region resembles the filter pattern, regardless of color. The resulting 2D array of scalars is called the **feature map**, of size $(W - w + 1) \times (W - w + 1)$. (intuitively the $+1$ is for the first position as if $w = W$ it would result to a 1×1 feature map)

Formally, the feature map F is defined as:

$$conv(X, K) = F_{pq} = \sum_{i=1}^{w} \sum_{j=1}^{w} K_{ij} \cdot X_{p+i-1,\, q+j-1}$$

where K is the kernel and X the input.

Example with X of size $3 \times 3 \times 1$ and kernel K of size $2 \times 2 \times 1$, the output is of size $(3 - 2 + 1) \times (3 - 2 + 1) = 2 \times 2$:

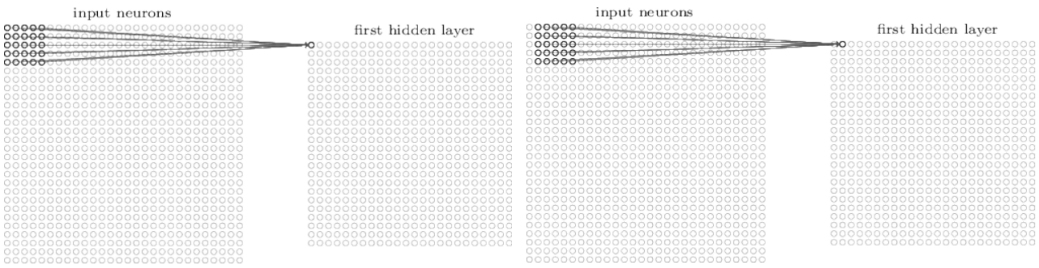
$$X = \begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{pmatrix}, \quad K = \begin{pmatrix} k_{11} & k_{12} \\ k_{21} & k_{22} \end{pmatrix}$$

$$F = \begin{pmatrix} k_{11}x_{11} + k_{12}x_{12} + k_{21}x_{21} + k_{22}x_{22} & k_{11}x_{12} + k_{12}x_{13} + k_{21}x_{22} + k_{22}x_{23} \\ k_{11}x_{21} + k_{12}x_{22} + k_{21}x_{31} + k_{22}x_{32} & k_{11}x_{22} + k_{12}x_{23} + k_{21}x_{32} + k_{22}x_{33} \end{pmatrix}$$

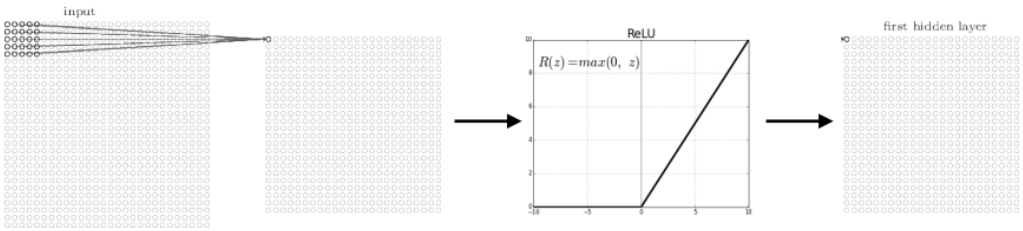
Convolutional layer

A convolutional layer has **fewer parameters than fully-connected layers** thanks to two key properties:

- The parameters of the filter (e.g., 5×5) are **learned** (as for the weights of an MLP).
- The parameters are **shared across different spatial locations** (the same filter is applied everywhere).

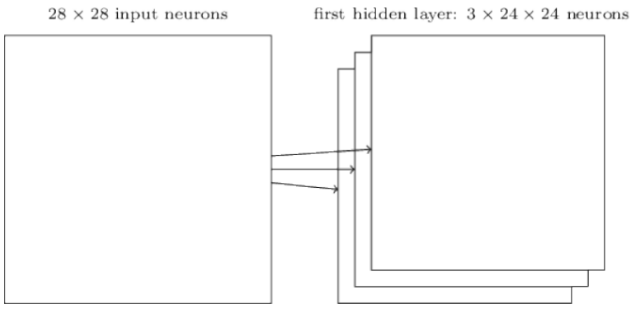


Nonlinearity As for MLPs, each convolutional output is passed through an **activation function** (e.g., ReLU). The activation is applied in an **element-wise** manner, i.e., to each spatial location independently.

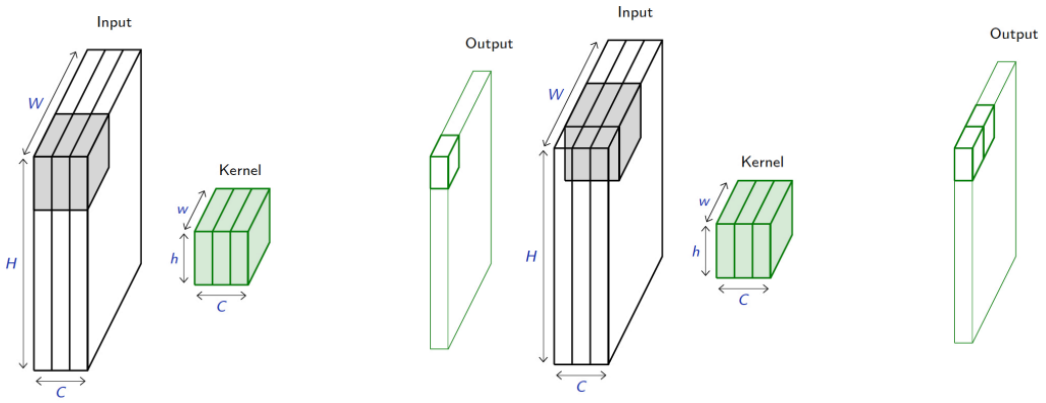


Multiple filters/channels We can use multiple filters in a single layer to create multiple **output channels**. Each filter has its own set of learnable parameters.

Example with 3 filters of size 5×5 on a 28×28 input, we obtain a $3 \times 24 \times 24$ output (3 channels, each of size 24×24), with $3 \cdot 5 \cdot 5 = 75$ learnable parameters.



Convolution with multiple input channels With multiple input channels (e.g., RGB image with 3 channels), the convolutions are in fact done in **3D**: the kernel has the same number of channels as the input ($w \times h \times C$), and the dot product is computed across all channels simultaneously to produce one output value per spatial location.

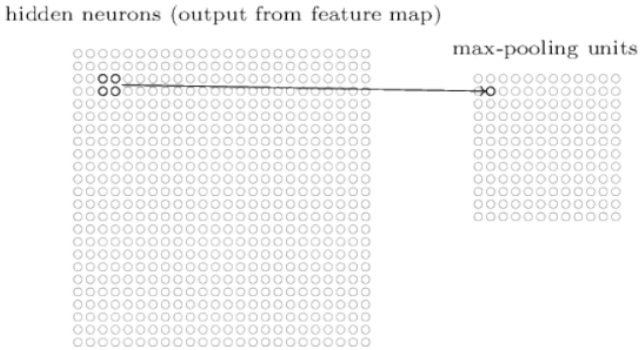


Pooling layer

The pooling layer goes from local information to a more **global representation** by aggregating values over a small spatial region. Different pooling strategies exist:

- **Max pooling**: keep the maximum value in each region.
- **Average pooling**: take the average of the values in each region.

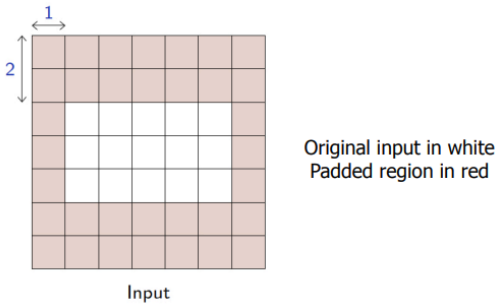
Pooling reduces the spatial size of the feature maps, which helps reduce the number of parameters in subsequent layers and provides some translation invariance.



Padding

To be able to center the convolution kernel on the boundary locations, **padding** extends the input. This can be done by mirroring or copying the boundary values, or by simply adding zeros (zero-padding). With a padding $p = (p_h, p_w)$ it increases the input $(W \times W)$ to $(W + 2p_h \times W + 2p_w)$.

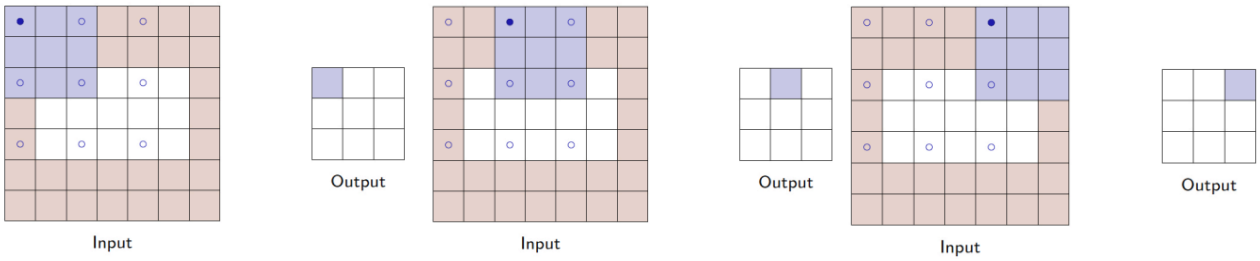
Example $C \times 3 \times 5$ input with a padding of $(2, 1)$ becomes $C \times 7 \times 7$ (2 lines of zeroes (zero-padding) up and down, and 1 column of zeroes to the left and to the right).



Convolution stride

In a standard convolution, the filter is shifted pixel by pixel. The shifts can be made larger by increasing the **stride** $s = (s_h, s_w)$: the filter jumps s_h pixels in height and s_w pixels in width at each step, reducing the output size to $\left\lfloor \frac{W}{s_h} \right\rfloor \times \left\lfloor \frac{W}{s_w} \right\rfloor$.

Example stride of $(2, 2)$ on a padded input with a 3×3 kernel: the kernel only hops every 2 pixels, producing a smaller output.



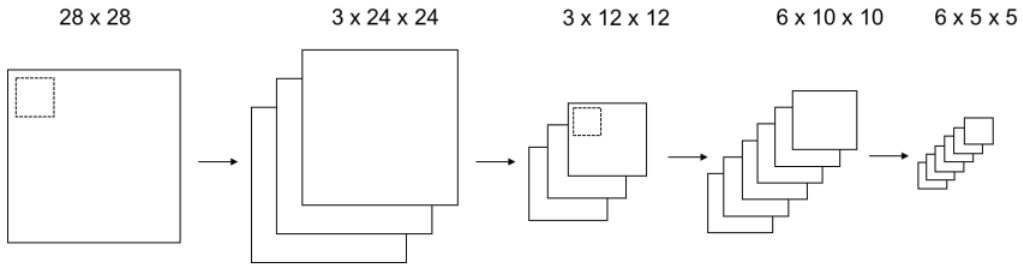
Goal of strided convolution: Reduce the spatial size of the feature map (downsampling), which decreases the computational cost and memory usage.

Drawback: Loss of spatial information (skipping pixels) which may hurt the model if fine-grained details are important.

CNN architecture

We can stack convolution + activation + pooling operations to build a CNN.

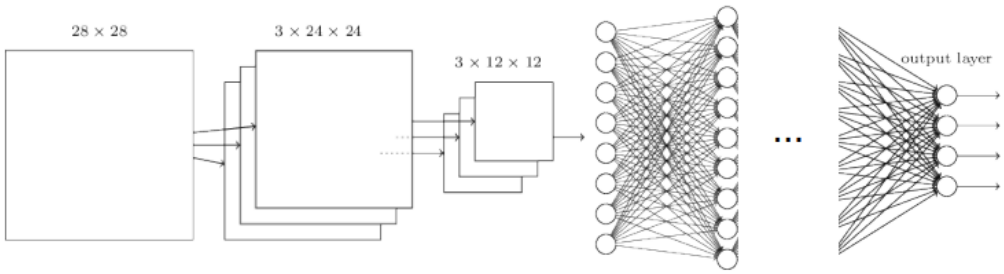
- The number of channels can vary between layers (e.g., first 3, then 6, then 16, ...).
- The size of the filters can vary (e.g., first 5×5, then 3×3).



Output layer for vector predictions Eventually, one typically wants to output:

- A vector of class probabilities for classification.
- A vector of 3D joint coordinates for human pose estimation.

This can be achieved by appending **fully-connected (FC) layers** at the end of the CNN. This requires **vectorizing** the last convolutional output first (e.g., $3 \times 12 \times 12 \to 432$-dimensional vector).

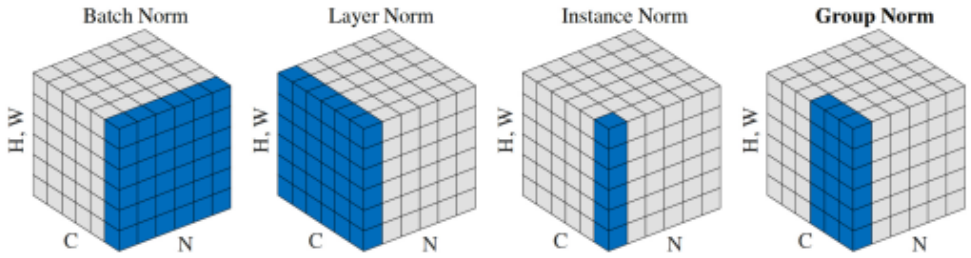


Normalization layers

Inspired by the fact that data normalization can help, several layers have been developed to normalize the feature maps inside the network. They differ by the dimensions over which the mean and variance are computed:

- **Batch normalization** : normalizes across the batch (per channel).
- **Layer normalization** : normalizes across all channels (per sample).
- **Instance normalization** : normalizes per channel and per sample.
- **Group normalization** : normalizes within groups of channels.

Notation: N = number of samples in a mini-batch, C = number of channels, (H, W) = size of the feature map.



CNN: Learning

Learning the weights of CNN layers is done in the same manner as for MLPs, using **stochastic gradient descent (SGD)** with mini-batches. The same idea of **backpropagation** is applicable to convolutional layers.

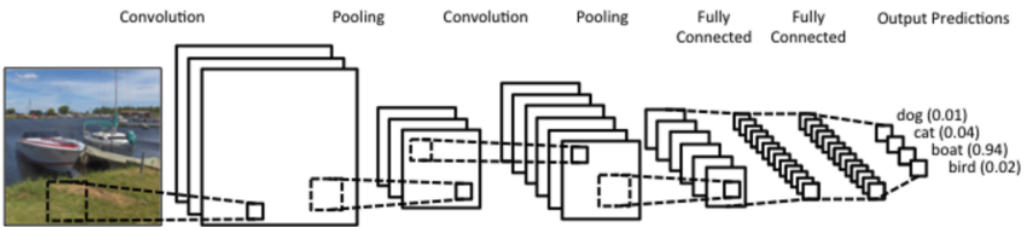
Gradient of pooling layers depends on the strategy:

- **Average pooling**: the gradient is backpropagated to all neurons used during pooling.
- **Max pooling**: the gradient is backpropagated only to the neuron that corresponded to the maximum value (the others get zero).

The same SGD extensions apply (Momentum, Adaptive learning rates like ADAM), as well as the same regularization techniques (Early stopping, Weight decay, Dropout).

CNN: Interpretation

A CNN can be thought of as **jointly learning the data representation and the classifier** (or regressor): the early layers learn low-level features (edges, textures), the middle layers learn parts (eyes, wheels), and the deeper layers learn high-level concepts (faces, objects).

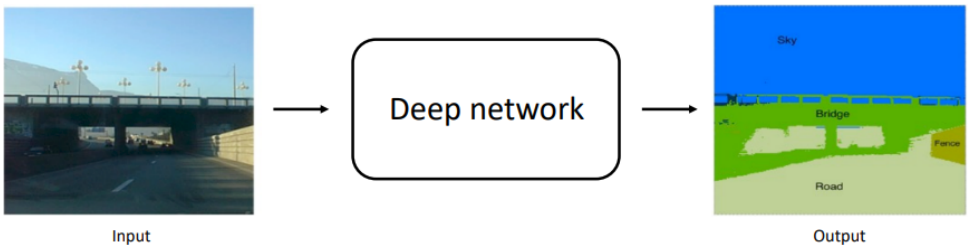


One can visualize the features extracted at different layers :

- **Layer 1**: oriented edges, color blobs.
- **Layer 2**: textures, simple patterns (circles, stripes).
- **Layer 3**: more complex patterns, parts of objects (faces, wheels, text).

2.7.10 Semantic segmentation

When working with images, one does not necessarily want to simply classify them. **Semantic segmentation** is the task of assigning a label to **every pixel** in the input image (e.g, sky, road, bridge, fence, ...).



Problem with standard CNNs

With convolutions (without padding and/or with stride > 1) and pooling layers, the size of the feature maps **decreases** at every layer. With the operations seen so far, we have no way of **increasing** this size back to the original resolution, which is needed for pixel-wise prediction.

Solutions

Two main solutions to upsample the feature maps:

- **Upsampling**, followed by convolutions (e.g., nearest-neighbor or bilinear upsampling, then convolution to refine).
- **Transposed convolutions** (also known as deconvolutions or fractionally-strided convolutions).

1D convolution as a matrix product

Before looking at transposed convolutions, let us first observe that a 1D convolution can be expressed as a **matrix product**. The input is a vector of dimension W, the kernel of size w, and the output of dimension $W - w + 1$.

Example with input $[1, 4, -1, 0, 2, -2, 1, 3, 3, 1]$ and kernel $[1, 2, 0, -1]$:

$$z = \begin{pmatrix} 9 \\ 0 \\ 1 \\ 3 \\ -5 \\ -3 \\ 6 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ & & & \ddots & & & & & & \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 2 & 0 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 4 \\ -1 \\ 0 \\ 2 \\ -2 \\ 1 \\ 3 \\ 3 \\ 1 \end{pmatrix}$$

So a convolution can be represented by a **rectangular matrix** going from dimension W to dimension $W - w + 1$.

Transposed convolution

Since a convolution can be represented by a rectangular matrix going from dimension W to dimension $W - w + 1$, **increasing the dimensionality** can be achieved by using the **transposed matrix**:

$$\begin{pmatrix} w_1 & w_2 & w_3 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & w_3 \end{pmatrix}^T = \begin{pmatrix} w_1 & 0 & 0 & 0 \\ w_2 & w_1 & 0 & 0 \\ w_3 & w_2 & w_1 & 0 \\ 0 & w_3 & w_2 & w_1 \\ 0 & 0 & w_3 & w_2 \\ 0 & 0 & 0 & w_3 \end{pmatrix}$$

This goes from dimension $W - w + 1$ to dimension W, so it **upsamples** the input (transposed conv is also called *up-convolution*).

Example of 1D transposed convolution With input $[2, 3, 0, -1]$ ($W = 4$) and kernel $[1, 2, -1]$ ($w = 3$): each input value multiplies the entire kernel, and the resulting scaled kernels are summed at shifted positions:

- Input value 2 : $[2, 4, -2]$ at position 0.
- Input value 3 : $\rightarrow [3, 6, -3]$ at position 1.
- Input value 0 : $\rightarrow\rightarrow [0, 0, 0]$ at position 2.
- Input value -1: $\rightarrow\rightarrow\rightarrow [-1, -2, 1]$ at position 3.

Summing them gives the output $[2, 7, 4, -4, -2, 1]$ of dimension $W + w - 1 = 6$.

So it goes from dimension W to dimension $W + w - 1$

In matrix form

$$\begin{pmatrix} 1 & 2 & -1 & 0 & 0 & 0 \\ 0 & 1 & 2 & -1 & 0 & 0 \\ 0 & 0 & 1 & 2 & -1 & 0 \\ 0 & 0 & 0 & 1 & 2 & -1 \end{pmatrix}^T \begin{pmatrix} 2 \\ 3 \\ 0 \\ -1 \end{pmatrix} = \begin{pmatrix} 2 \\ 7 \\ 4 \\ -4 \\ -2 \\ 1 \end{pmatrix}$$

2D convolution as matrix product

A similar matrix representation can be obtained for 2D convolutions:

- The input is **vectorized**, e.g., by concatenating its columns (e.g., matrix 4×3 to a matrix 12×1).
- The matrix is obtained by spreading the different columns of the kernel at the matching locations.

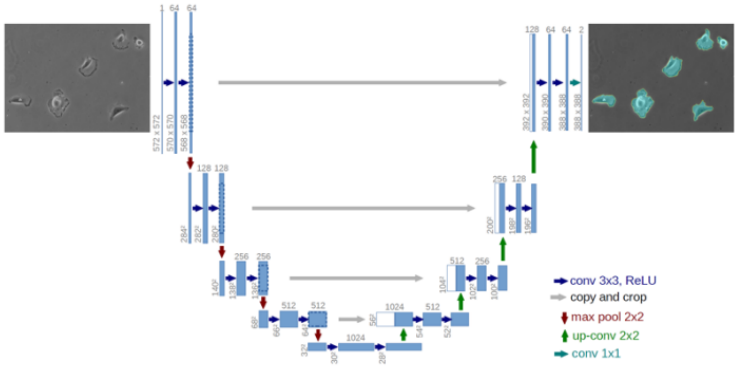
The same trick (using the transpose) gives 2D transposed convolutions for upsampling.

Fully-convolutional networks

Stacking upsampling-convolutions, or transposed convolutions, after regular ones lets us go back to the original spatial resolution. This is the principle of **fully-convolutional networks (FCNs)** like **U-Net**, widely used for semantic segmentation.

The U-Net architecture has a characteristic "U" shape:

- A **contracting path** (encoder): regular convolutions + pooling reduce the spatial size while increasing the number of channels.
- An **expanding path** (decoder): transposed convolutions (up-conv) increase the spatial size back to the original resolution while decreasing the number of channels.
- **Skip connections** between matching encoder and decoder levels (copy and crop) allow the decoder to recover spatial details lost during downsampling.

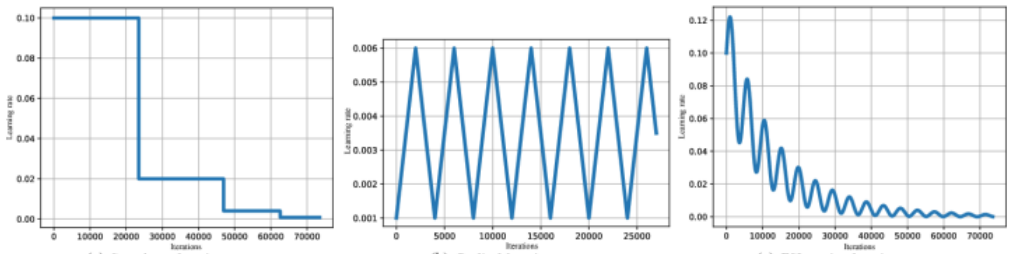


2.7.11 The keys to the success of Deep Learning

- **Big data**: large annotated datasets, e.g., ImageNet with millions of labeled images.
- **GPUs**: massively parallel computations make training of deep networks feasible.

Tricks of the trade

- **Pre-training**: people often start with a standard architecture pre-trained on ImageNet, then fine-tune (adapt) it on the target task. This transfers learned features and accelerates training.
- **Data augmentation**: artificially increase the dataset size by applying transformations that preserve the label, e.g., for images: flip, rotate, affine transform, color jittering, ...
- **Learning rate scheduling**:
  - Decrease the learning rate η after a pre-defined number of epochs (step-decay (divide η by a fix factor after a number of predefined epochs)).
  - Use a cyclic learning rate (make η oscillates between a min and max value).

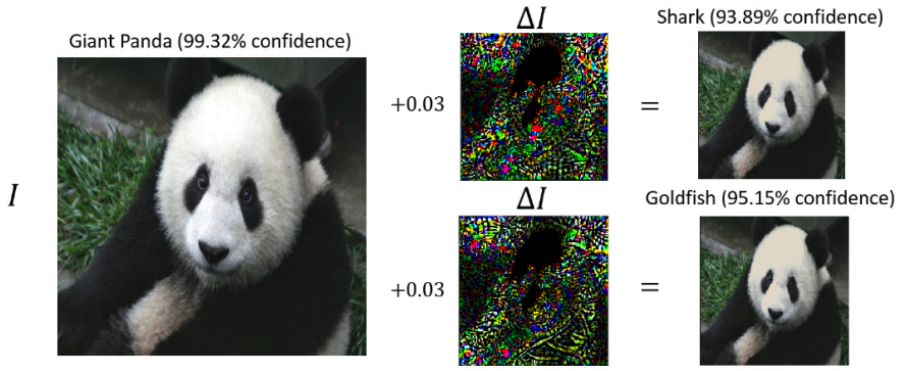


(a) Step-decay learning rate (b) Cyclical learning rate (c) ESLearning learning rate

2.7.12 Despite the success: Adversarial examples

Even very accurate deep networks can be fooled by **adversarial examples**: small, imperceptible perturbations ΔI added to the input image I can completely change the prediction.

Example An image classified as "Giant Panda" with 99.32% confidence, after adding a tiny perturbation (+0.03), is classified as "Shark" with 93.89% confidence (or as "Goldfish" with 95.15% confidence with a different perturbation), even though the modified image still looks like a panda to humans.



Intuition: Deep networks learn highly non-linear decision boundaries in a high-dimensional input space. Even if the perturbation is small in pixel space, it can move the input across a decision boundary in the network's internal feature space. Networks rely on specific patterns/textures rather than the high-level semantic concepts humans use, so carefully crafted perturbations targeting these patterns can fool the network without changing the human-perceived content of the image.

This shows that despite their success, deep networks are not as robust as humans, and robustness to adversarial examples remains an active area of research.

2.8 From images to text

The deep learning tools we have seen so far (MLPs, CNNs) were tailored to inputs of a **fixed structure**: a vector of fixed size, an image of fixed resolution. **Natural Language Processing** (NLP) breaks this assumption: a document is a sequence of words whose length varies from sample to sample. The goals of this lecture are:

- to discuss how to **represent text** (words and sequences) so it can be fed to a neural network;
- to introduce **recurrent neural networks** (RNNs), the historical answer to variable-length inputs;
- to introduce **transformers**, the architecture that has come to dominate modern NLP (and beyond).

2.9 Representing text

2.9.1 The heterogeneity problem

Text data is naturally **heterogeneous in length**: documents in the 20 Newsgroups dataset, for instance, range from a one-line apology to a long technical post. For supervised methods like an MLP or a softmax classifier, we have always assumed inputs of identical size, so we need a way to turn an arbitrary piece of text into a fixed-size representation, or to build models that natively accept variable-length sequences.

2.9.2 Bag of Words: a document-level representation

A first idea is the **Bag of Words (BoW)** representation seen earlier in the course: fix a dictionary of D words and represent each document by a vector $\tilde{\mathbf{x}} \in \mathbb{R}^D$ where entry i counts how many times the i-th word appears in the document. Two documents are then compared by comparing their word *counts*.

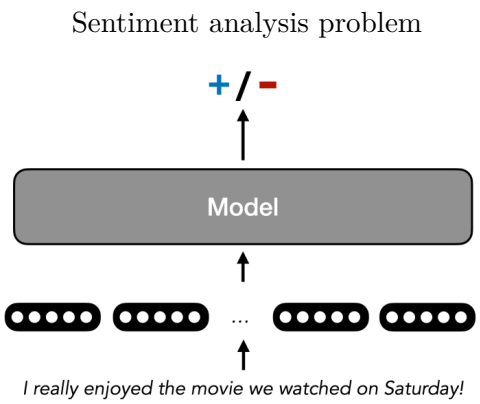
BoW is fine when the task is at the **document level** (sentiment analysis, author age prediction, topic classification), but it has two strong limitations:

- it **throws away word order** – "the dog bit the man" and "the man bit the dog" look identical;
- it cannot solve **word-level tasks** such as named entity recognition, translation, or text generation, where we need a representation for each individual word.

This motivates moving from document-level to **word-level** representations.

2.9.3 Word representations

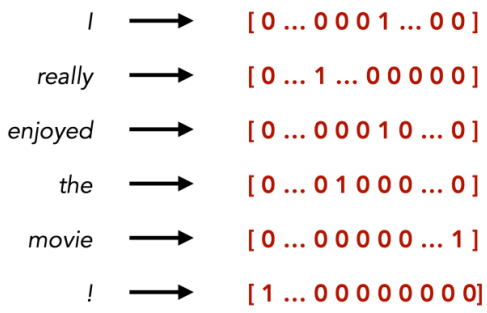
How do we represent natural language sequences for NLP problems ?



The way neural language processing (NLP) do it is by representing **words** as **vectors**, thus a sentence becomes a sequence of vectors.

2.9.4 Words as vectors: the naive one-hot approach

The simplest way to represent each word as a vector is to assign each word a unique index in a vocabulary V and use a **one-hot vector** $x_i \in \{0,1\}^{|V|}$: every entry is 0 except the one corresponding to the word, which is 1.



But this representation has two serious drawbacks:

- **Huge dimensionality**: vocabularies easily reach 10^4–10^6 words.
- **No notion of similarity**: any two distinct words are *orthogonal*. For example, *enjoyed* and *loved* are semantically very close yet their one-hot vectors have cosine similarity 0. The model would have to relearn the meaning of each word independently from data.

2.9.5 Word embeddings

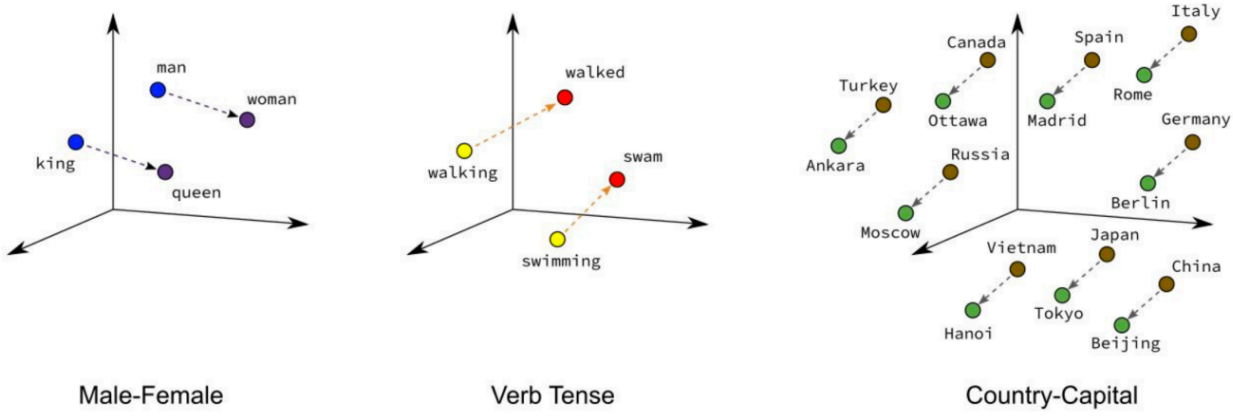
Word embeddings replace the sparse one-hot vector with a **high-dimensional, real-valued** vector in $\mathbb{R}^d$, with d typically in $O(10^2$–$10^3)$ – still much smaller than $|V|$. The crucial property is that:

the geometry of the embedding space reflects the semantics of words.

Words with similar meaning end up close together, and even directions in the embedding space become meaningful.



Example In those illustrations we see that words with similar meaning are close. And crazier that for example king − man + woman ≈ queen, and that the direction going from a country to its capital is roughly constant across country/capital pairs.



The dot product measures alignment Because words now live in $\mathbb{R}^d$, comparing two words means comparing two vectors, and the natural tool is the **dot product**. Geometrically it tells us how well two vectors *align*: it is positive when they point in similar directions, zero when they are perpendicular, and negative when they point in opposite directions. Computationally it is just a weighted sum of coordinates – exactly the kind of operation deep networks are built from.

This also lets us *quantify* a semantic direction. Suppose we conjecture that cats − cat encodes a "plurality" direction. Taking the dot product of this vector with singular nouns and with their plurals, the plurals consistently score higher; dotting it against the embeddings of *one, two, three*, … even gives increasing values. A direction in the space thus carries meaning, and the dot product measures how strongly a word expresses it – the very operation the attention scores will rely on.

How are embeddings learned?

Two paradigms exist:

- **Supervised**: optimize the embeddings for a specified task (e.g, sentiment analysis). This works but requires labels and the learned embeddings are *task-specific* – they may not generalize to other tasks.
- **Unsupervised (or self-supervised)**: learn embeddings from raw, unlabelled text by exploiting the structure of language. The guiding principle is Firth's quote (1957):

  "You shall know a word by the company it keeps."

  Concretely, the meaning of a word is approximated by looking at the words that typically appear **nearby**. For example, the blank in "*I really enjoyed the ___ we watched on Saturday*" is very probably *movie* or *game*, and the corresponding embeddings should be similar.

Word2Vec and **GloVe** are the standard self-supervised approaches that exploit this idea.

Remark. Word embeddings are pre-trained once on a huge corpus and then *reused* as a fixed lookup table for downstream tasks – a first form of *transfer learning* (a machine learning technique where a model pre-trained on a large dataset is reused as the starting point for a new, related task) in NLP.

2.9.6 Why individual words are not enough

Even with great word embeddings, processing words **one at a time** is unsatisfactory:

- we still face the **heterogeneity** problem – documents have variable length;
- individual words can be **misleading**: in the review *"The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. …Their ambience was just as* **good** *as the food and service."*, the word *good* is positive in isolation but the surrounding context determines whether the review is actually a praise or a complaint.

Therefore, instead of modeling words, we want to model a complete sentence or document as a sequence. Note that each individual word is still modeled as a vector as discussed previously.

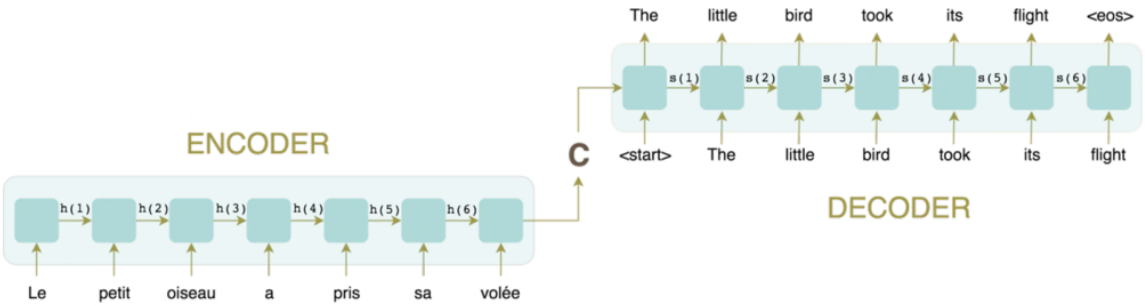
Embeddings that soak up context In that very first step, each vector is simply plucked out of the embedding table, so it can only encode the meaning of a *single word in isolation*, blind to its neighbours. The whole purpose of the network that follows is to let each vector progressively **absorb context**, so that by the end it points in a far more specific direction. A vector that starts its life as the embedding of *king*, for instance, can be gradually tugged by the successive blocks until it encodes "a king who reigned in Scotland, who seized the throne by murdering his predecessor, and is described in Shakespearean English" – i.e. *Macbeth*. The token is unchanged; its meaning has been enriched by everything around it. This contextual enrichment is precisely the job of the **self-attention** mechanism introduced next.

2.10 Modeling sequences

2.10.1 Sequence-to-sequence: encoder–decoder

The dominant template for such models is the **encoder–decoder** (also called **sequence-to-sequence**, or seq2seq) architecture. The **encoder** reads the input sequence and compresses it into a representation; the **decoder** produces an output sequence from this representation.

Example Translation: an encoder reads a French sentence and a decoder writes the English translation.

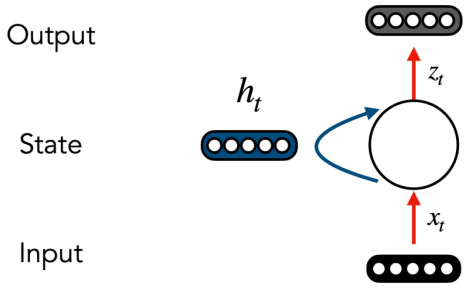


The remaining question is: *how do we model each of these blocks?* Historically, the answer was **recurrent neural networks**; today it is **transformers**. But let's see how they both work.

2.11 Recurrent Neural Networks (RNNs)

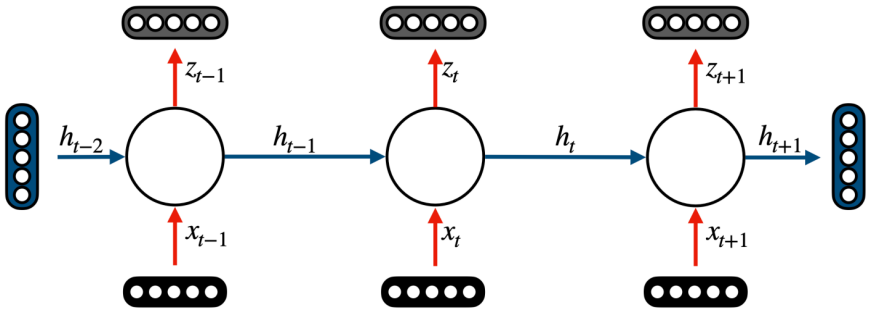
2.11.1 Idea

A **Recurrent Neural Network** is a neural network with a **feedback loop**: it maintains an internal **state** h_t that is updated at each time step from the new input x_t and the previous state h_{t-1}. The state then summarizes everything seen so far.



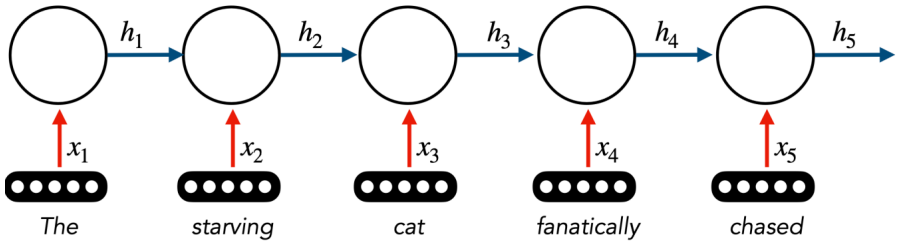
2.11.2 Unrolling

The recurrence is conceptually a loop, but for computation and visualization we **unroll** the network across time, obtaining a feed-forward graph where each time step has its own cell sharing the *same* weights:



Crucially, the same parameters are reused at every time step. This makes the model applicable to sequences of *any length*, regardless of what was seen at training time.

Example Making a sentence of 5 words : *"The starving cat fanatically chased"*, results to updating the internal state 5 times



2.11.3 Equations for the standard (Elman) RNN

For an input sequence $\{x_t\}$, the basic RNN computes

$$h_t = \sigma\big(W_{hx}\, x_t + W_{hh}\, h_{t-1} + b_h\big), \qquad z_t = \sigma\big(W_{zh}\, h_t + b_z\big),$$

where σ is a non-linear activation (often tanh), h_t is the hidden state and z_t is the output at time t. The weight matrix W_{hx} transforms the input x_t so he could contribute to the new state (same for W_{hh}), W_{zh} is the matrix that project the hidden state to the output space. We could see the matrix as traductors, from a space to an other. All three weight matrices and the two biases are *shared* across time steps.

2.11.4 Three uses of an RNN

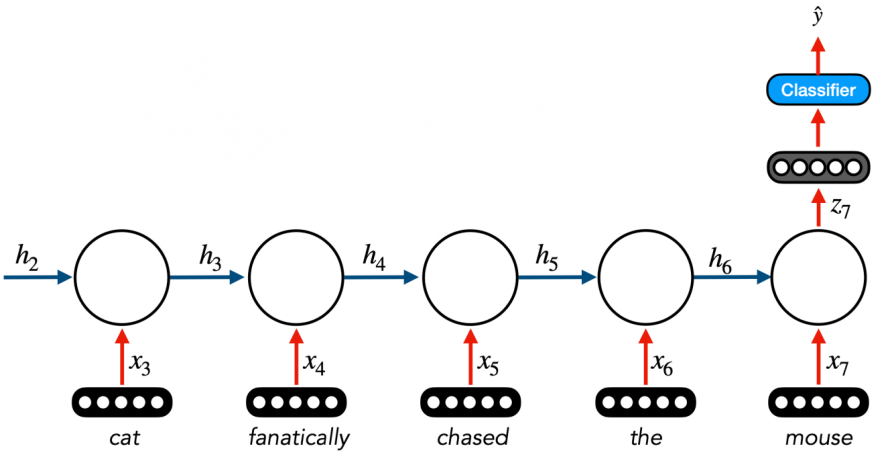
Depending on what we read off the unrolled network, an RNN can be used for very different tasks.

Sequence classification

We feed the whole sequence to the RNN and use only the *last* output z_T as a summary, which is fed to a classifier (a linear projection followed by a sigmoid for binary tasks or a softmax for multi-class):

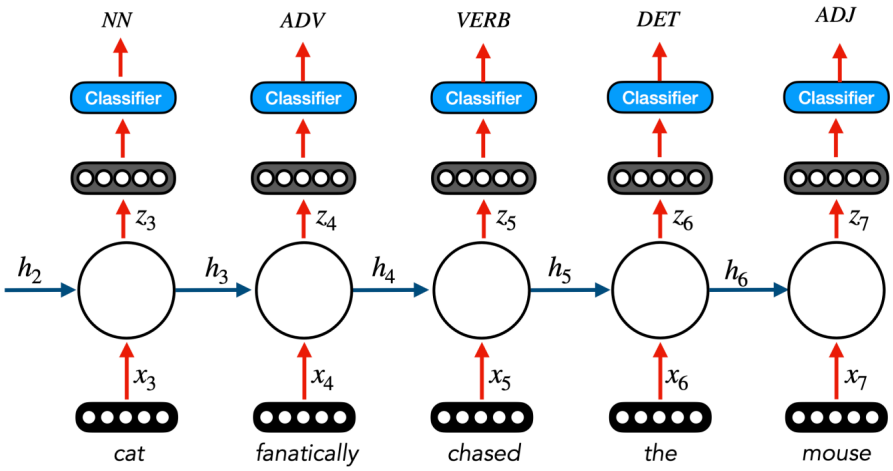
$$\textbf{Binary: } P(y) = \sigma(W_o z_T) \qquad \textbf{Multi-class: } P(y) = \text{softmax}(W_o\, z_T).$$

This is what one would do for, e.g, sentiment analysis.



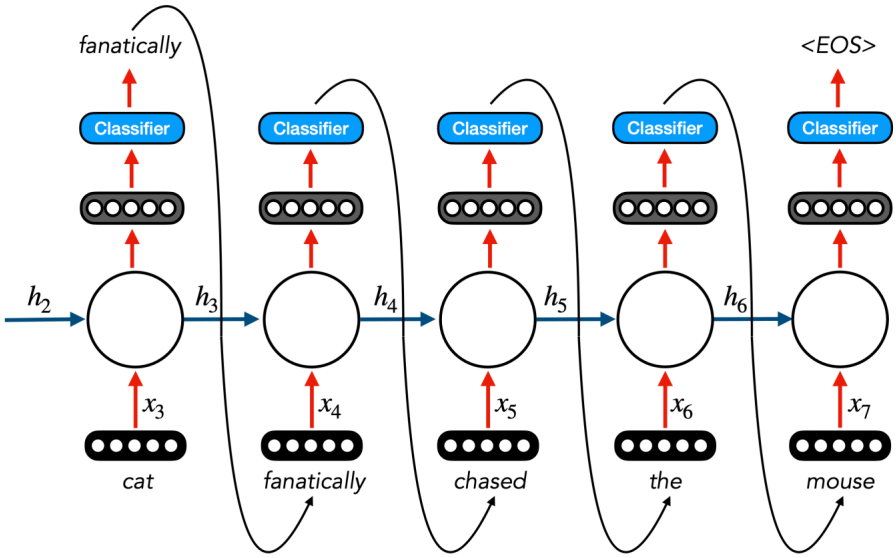
Sequence labeling

For per-word tasks (e.g, part-of-speech tagging, named entity recognition), we attach a classifier to *every* output z_t to produce one label per token.



Generation

To **generate** text, we feed the predicted output of step t as the input of step $t+1$. Starting from a special start token (and possibly some context), the RNN produces a sequence one token at a time until it emits an end-of-sequence (`<EOS>`) token.



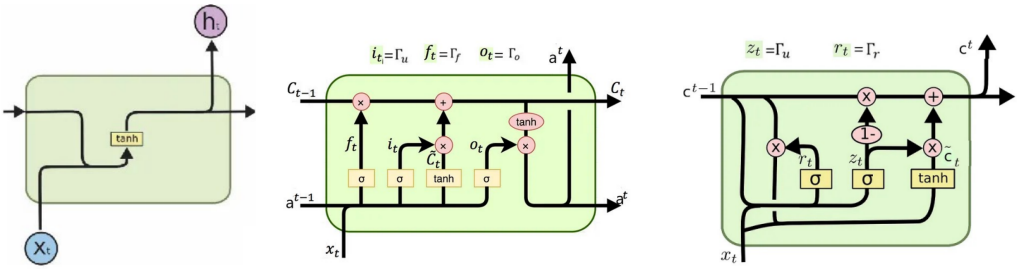
2.11.5 RNN variants: LSTM and GRU

The standard RNN above struggles to remember information across many time steps – when we backpropagate through many copies of the same weights, gradients tend to vanish (or explode). This issue was already encountered in deep MLPs, and is more severe here because the depth is the length of the sequence.

Two improved cells are commonly used in practice:

- **LSTM** (Long Short-Term Memory): introduces a separate **cell state** c_t and three *gates* (input, forget, output) that control what is written to, read from, and erased from the cell. The gates use sigmoids and elementwise products, allowing the network to learn to keep information for many steps.

- **GRU** (Gated Recurrent Unit): a simplified gated cell with only two gates (update, reset). Cheaper than an LSTM, comparable performance in many applications.



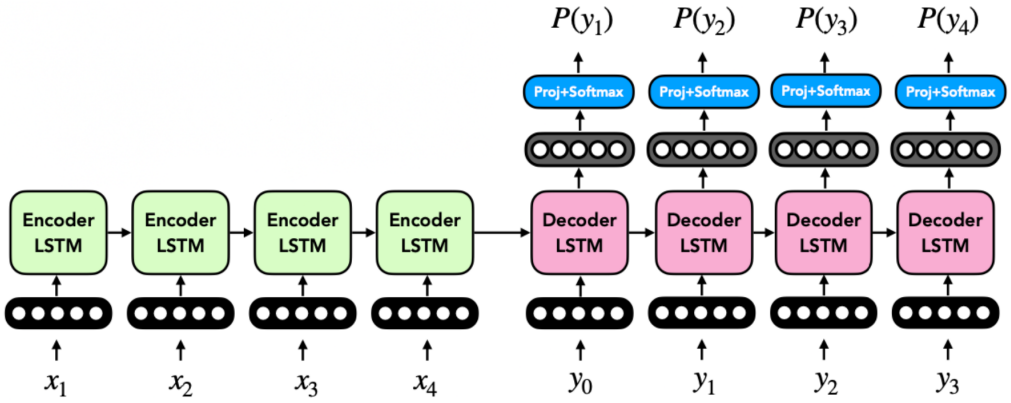
Remark. For the purposes of this course, the important point is that an LSTM (or GRU) is just an RNN with a more careful internal cell that mitigates vanishing gradients and learns longer-range dependencies. The high-level behaviour – a recurrent cell processing a sequence one token at a time – is exactly the same.

2.11.6 Training an encoder–decoder RNN

For an encoder–decoder model, both the encoder and the decoder are RNNs (typically LSTMs). The encoder processes the input sequence; its final hidden state initializes the decoder, which generates the output sequence token by token. The model is trained by minimizing the **average cross-entropy** of the predicted tokens with respect to the ground-truth output:

$$\mathcal{L} = \sum_{t=1}^{T} -\log P\big(y_t \mid y_1, \ldots, y_{t-1},\ x_1, \ldots, x_n\big).$$

Gradients are backpropagated through both the decoder and the encoder.



2.11.7 Limitations of RNNs

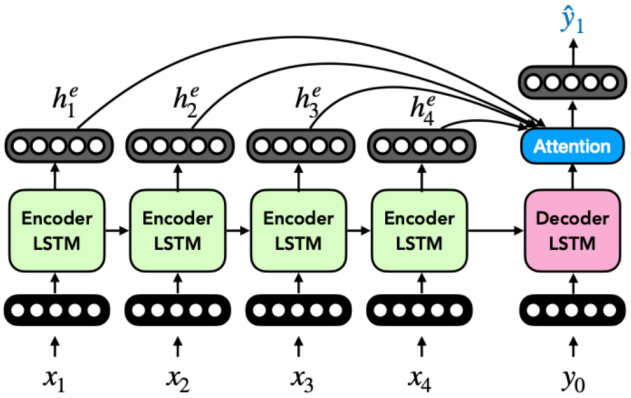
RNNs suffer from two structural problems:

- the **state is a single fixed-size vector**, thus it is a massive compression of information.
- it is **hard to learn long-range dependencies**, despite LSTMs/GRUs, especially when the relevant information is many steps away from where it is needed.

2.12 Attention

2.12.1 Motivation

The RNN encoder produces a hidden state h_t^e at *every* time step, not just at the end. Instead of throwing all of these away and keeping only the last one, the **attention mechanism** lets the decoder, at each generation step, look back at *all* encoder states and decide which ones are relevant. The decoder *focuses on different parts of the input at different generation steps.*



2.12.2 The attention function

At decoding step 1, the current decoder hidden state h_1^d is compared with each encoder hidden state h_t^e via a *similarity function* f, producing a raw **attention score** ("idea of what to decode"):

$$a_t = f\big(h_t^e,\, h_1^d\big), \qquad t = 1, \ldots, T.$$

Three classical choices for f are summarised below:

Attention Function	Formula
Multiplicative	$a = h^e \,\mathbf{W}\, h^d$
Linear	$a = v^T \,\phi\big(\mathbf{W}\,[h^e;\, h^d]\big)$
Scaled dot prod.	$a = \dfrac{(\mathbf{W}h^e)^T\,(\mathbf{U}h^d)}{\sqrt{d}}$

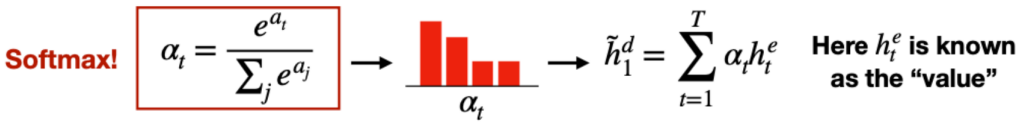
The scaled dot product is what transformers use; the scaling by $\sqrt{d}$ keeps the magnitude of the dot product bounded as the dimension d grows, which stabilises training.

The raw scores are then turned into a **probability distribution** over the encoder states via softmax:

$$\alpha_t \;=\; \frac{e^{a_t}}{\sum_j e^{a_j}},$$

and the resulting **context vector** is a weighted average of the encoder states:

$$\tilde{h}_1^d \;=\; \sum_{t=1}^{T} \alpha_t\, h_t^e.$$



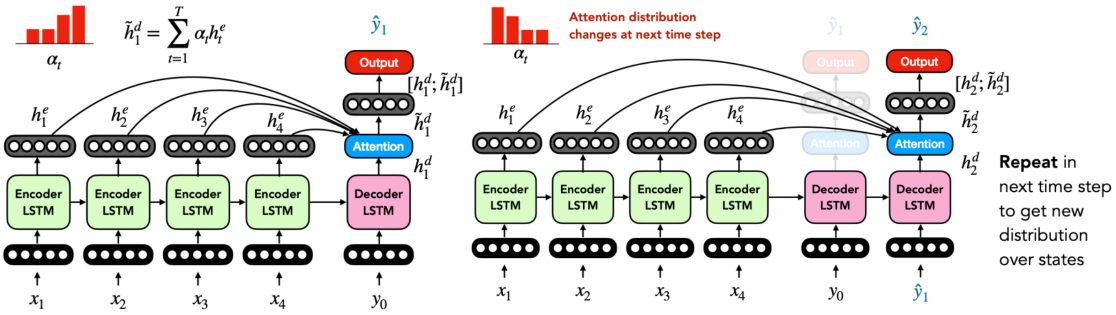
Terminology to remember. In this picture:

- the decoder state h^d that we use to "ask the question" is the **query**;
- each encoder state h_t^e plays two roles:
  - as a **key**, against which the query is compared (computes a_t) "answering the question";
  - as a **value**, which is the thing being summed in the weighted average.

This query/key/value vocabulary will become essential when we describe self-attention.

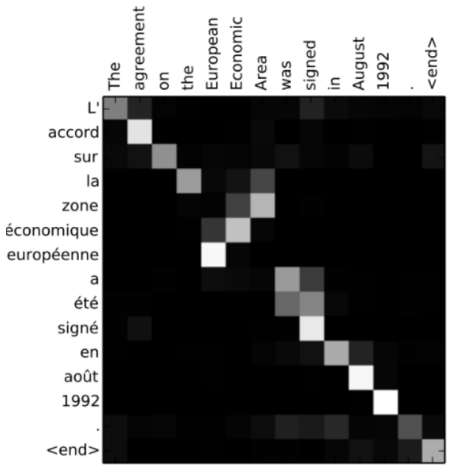
2.12.3 The attentive encoder–decoder

Once $\tilde{h}_1^d$ has been computed, it is **concatenated** (or otherwise combined) with the original decoder state h_1^d, and fed to the output layer to predict the most likely token $\hat{y}_1$. The decoder then advances to the next step, recomputes a new attention distribution from h_2^d, and so on. *The attention distribution changes at every time step*, allowing the model to focus on different parts of the input as it generates.



2.12.4 Bonus: interpretability

A nice side benefit of attention is that, by plotting α_t at each generation step, we can **visualise** which input words the model looks at when producing each output word. On a French–English translation task this typically yields a near-diagonal alignment matrix, with off-diagonal entries appearing exactly where the languages reorder words. We get *word alignment for free* as a by-product of training.

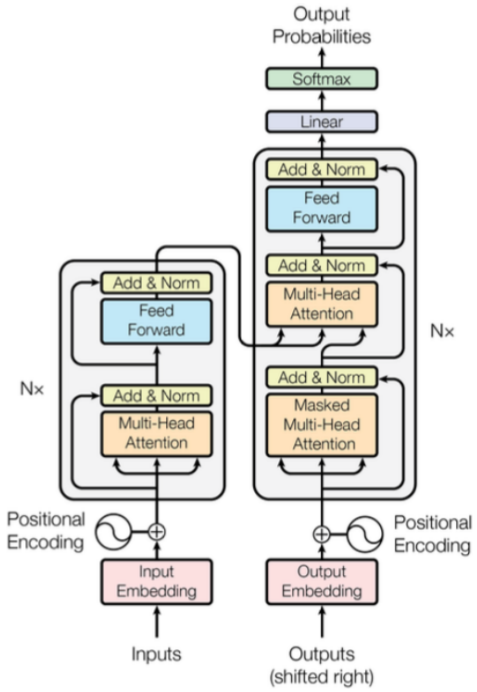


2.12.5 Recurrence as problem

Attention solves the bottleneck problem of RNNs (the decoder can now look at every encoder state), but the **encoder itself is still recurrent**: the state h_t^e depends on h_{t-1}^e, so we cannot compute the encoder hidden states in parallel. On modern hardware (GPUs/TPUs), this is a severe limitation – we want the model to be **parallelisable** across the sequence length. This leads us to transformers.

2.13 Transformers

The **transformer** removes recurrence altogether and encodes a sequence using only **self-attention** and **feed-forward** layers.



2.13.1 Self-attention

The key idea is to apply attention to the input *within itself*: each token gets to attend to every other token in the same sequence – including itself. In the encoder, at layer ℓ, every element plays *all three roles*: each token is at once a query, a key, and a value.

For a single "query token" h_s^0 at position s, self-attention computes

pairwise attention score attention distribution weighted sum

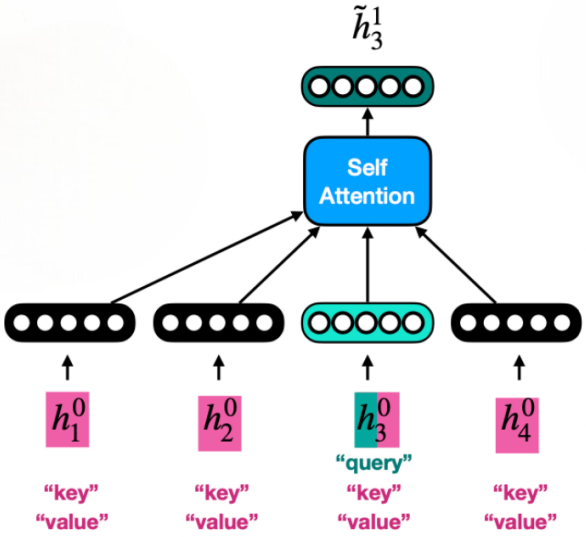
$$\mathbf{a} \;=\; \frac{(\mathbf{W}^Q q)\,(\mathbf{W}^K K)}{\sqrt{d}}, \qquad \alpha \;=\; \mathrm{softmax}(\mathbf{a}), \qquad \tilde{h}_s^\ell \;=\; W^O\,\alpha\,(V\mathbf{W}^V),$$

where:

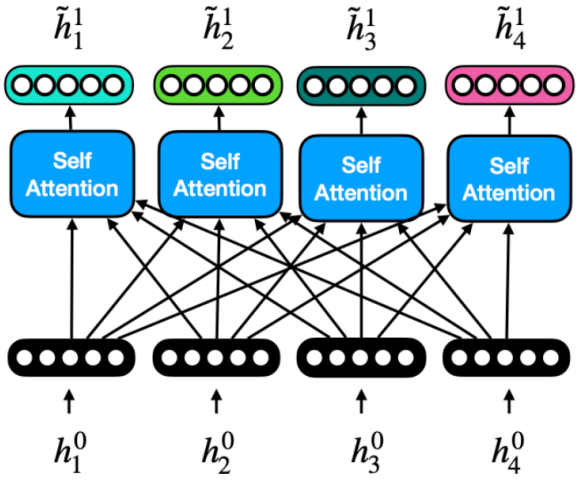
- $q = h_s^\ell$ is the query (one element of the sequence);
- $K = V = \{h_t^\ell\}_{t=0}^T$ are the keys and values (*all* elements);
- $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$ are **learned** projections that turn the same input vector into a query, a key and a value – they are what makes the three roles different;
- W^O is a final output projection.

The intuition behind queries, keys and values A helpful way to picture the three roles is as a question–answer game. Each token uses its **query** to *ask a question* about the others: a noun like *creature*, in the phrase "a fluffy blue creature", is effectively asking "are there any adjectives just before me?". Each token uses its **key** to *answer*: the words *fluffy* and *blue* produce keys that say "yes, I am an adjective sitting right before you", so their keys *align* with the query of *creature* – which is exactly why their dot products (the attention scores) are large. Finally, the **value** answers a different question: "if I am relevant to you, what should I add to your meaning?". In short, the query decides *what to look for*, the key decides *who matches*, and the value decides *what gets passed along*.

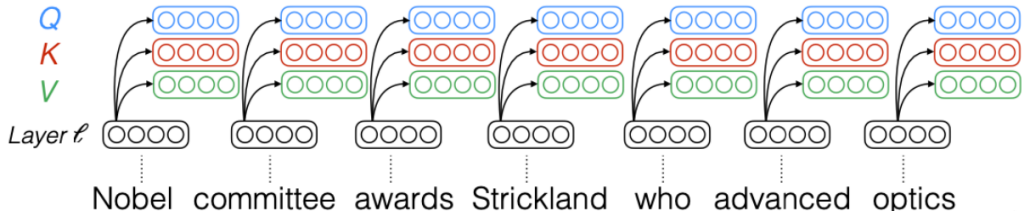
Queries and keys live in a smaller space The query and key projections $\mathbf{W}^Q, \mathbf{W}^K$ do not preserve the embedding dimension: they map each embedding (dimension $d \sim 12\,000$ in GPT-3) down to a much smaller **query–key space** of dimension d_k (typically $d_k = 128$). The scores are dot products computed in this smaller space, and the d appearing in the $\sqrt{d}$ scaling factor is precisely this query–key dimension d_k. Working in a low-dimensional query–key space also keeps the number of attention parameters modest – which matters a great deal once many heads are run in parallel.



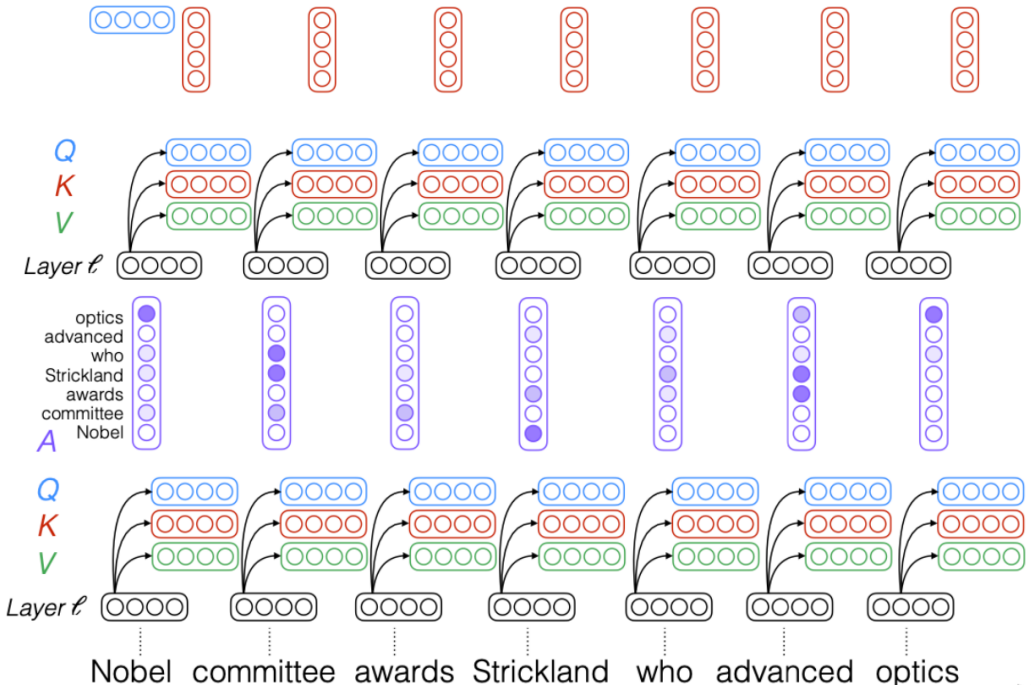
This is repeated with every token playing the role of the query, in parallel, producing one output vector $\tilde{h}_s^\ell$ per input position. The whole layer is just a few matrix multiplications – there is no recurrence and the computation across positions is fully parallel.



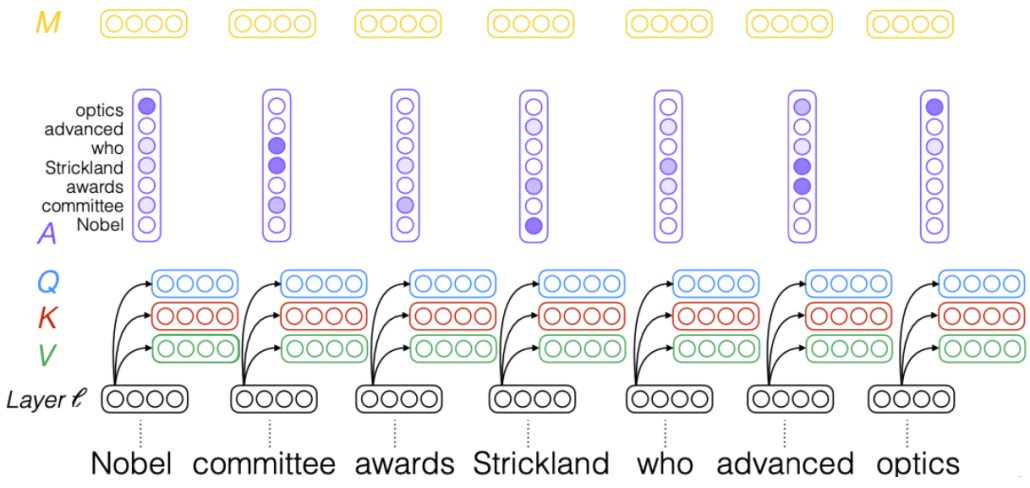
Example walk-through. Take the sentence "Nobel committee awards Strickland who advanced optics". For each word, W^Q, W^K, W^V produce a query, a key and a value.



Comparing the query of every word with the keys of all words gives an attention matrix A whose row s encodes *which words token s pays attention to*.



Multiplying A by the values produces a new representation M of each token, enriched by the rest of the sentence – e.g, the representation of "Strickland" may now also encode information from "awards" and "optics".



Attention computes an additive update It is worth being precise about *how* a token's representation is refined, because it connects directly to the residual connections. The attention output $\tilde{h}_s^\ell$ is best read not as a replacement but as a **proposed change** Δe: the value vector of a relevant word answers "here is what I should add to your embedding", and the weighted sum of these value vectors (weighted by the attention scores α) is the change to apply. Through the residual connection, this change is then *added* to the original, context-free embedding,

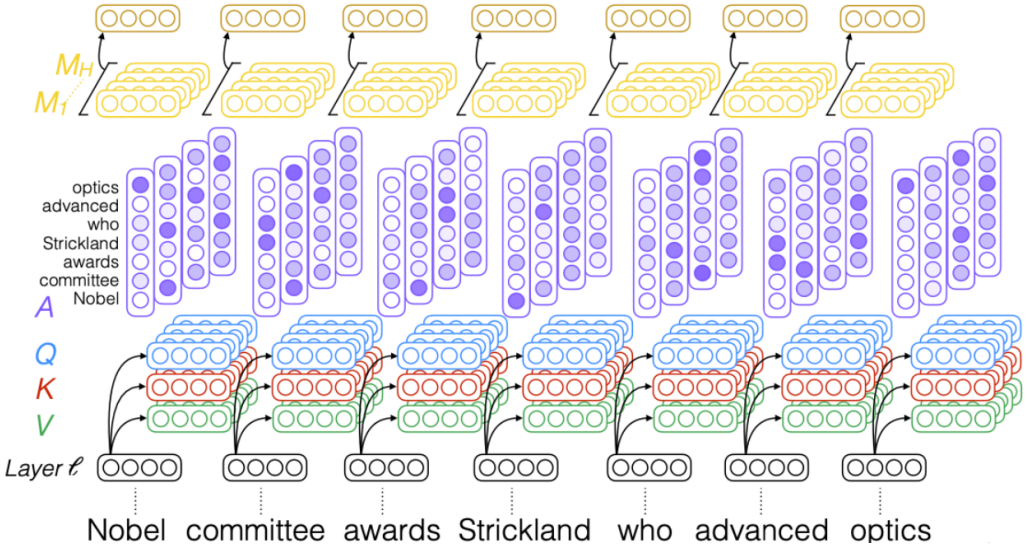
$$e_{\text{new}} \;=\; e + \Delta e, \qquad \Delta e \;=\; \tilde{h}_s^\ell,$$

so each token keeps what it already encoded and attention only *nudges* it towards a more context-specific direction – exactly the $x + \text{sublayer}(x)$ logic.

For the example *"a fluffy blue creature"*, the embedding of *creature* starts generic, and attention adds a "fluffiness" and "blueness" component until it points towards *fluffy blue creature*.

2.13.2 Multi-head self-attention

A single attention pattern is restrictive: a word may need to attend to its grammatical subject *and* to a related noun *and* to a pronoun, all at once. **Multi-head self-attention** runs H independent self-attention computations ("heads") in parallel, each with its own $\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V$. The outputs $M_1, \ldots, M_H$ are concatenated and projected back to the original dimension. Each head can specialise in a different type of relation.

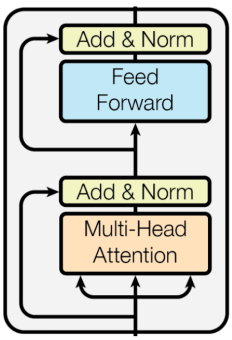


This is *the main innovation of the transformer*.

2.13.3 The transformer block

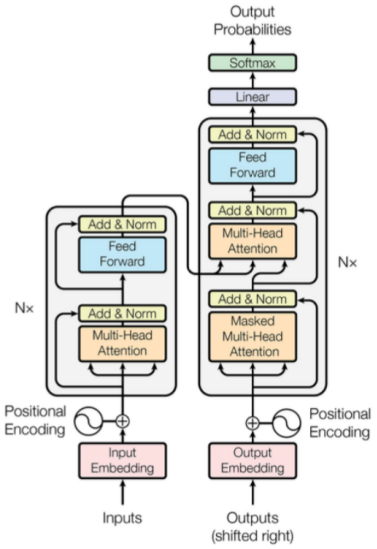
Multi-head attention is the heart of a transformer block, but a block contains more:

- a **multi-head self-attention** layer;
- a **feed-forward network** (a small MLP applied independently at each position);
- **residual connections** around each sub-layer ($x + \text{sublayer}(x)$);
- **layer normalisations** after each residual.



What the feed-forward sub-layer does Whereas the attention sub-layer is where the vectors *exchange* information, in the feed-forward sub-layer they do **not** talk to each other at all: the same small MLP is applied independently at every position. A useful intuition is to picture it as asking a long list of questions about each vector – "does this encode a noun?", "something royal?", "a past-tense verb?", … – and updating the vector according to the answers. This is widely believed to be where much of the model's stored factual and relational knowledge lives. The two sub-layers thus play complementary roles: *attention moves information between tokens, the feed-forward layer processes each token in place*, and alternating them is what builds up rich representations.

2.13.4 Full transformer



The full transformer encoder is multiple cascaded transformer blocks; each block refines the representations and builds up increasingly compositional features of the input.

The transformer decoder (right side) is similar to encoder, but each blocks contains *two* attention sub-layers:

- First layer of block is **masked** multi-headed attention
- Second layer is multi-headed attention over *final-layer* encoder outputs **(cross-attention)**
- The third layer is feed-forward network (as encoder).

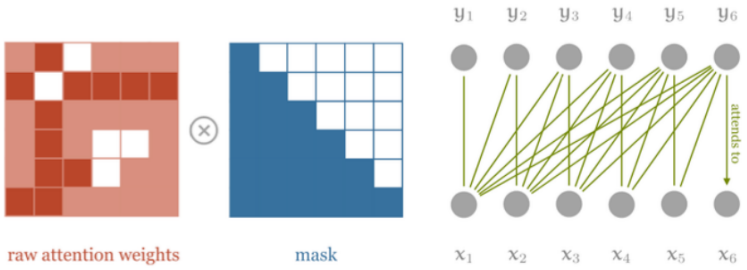
2.13.5 The decoder side: masked self-attention and cross-attention

Masked self-attention

The decoder generates tokens one at a time, so when producing token t it must only see tokens $< t$ – otherwise, at training time, it could trivially cheat by looking at the answer.

Masked self-attention enforces this by setting the attention scores of future positions to $-\infty$ before the softmax, which makes the corresponding weights $\alpha = 0$:

$$a_{st} = \frac{(\mathbf{W}^Q h_s^\ell)^T (\mathbf{W}^K h_t^\ell)}{\sqrt{d}}, \qquad a_{st} := a_{st} - \infty \quad \text{for } s < t, \qquad \alpha_{st} = \frac{e^{a_{st}}}{\sum_j e^{a_{sj}}} = 0 \text{ for } s < t.$$



This mask is applied as soon as the model as to respect a causal structure, i.e., when position t must see only the positions $\leq t$. So when generating token by token (e.g., decoder used to generate text)

Why this would be cheating: how the decoder is trained The reason the model *could* cheat is that training does not proceed the same way as generation. At generation time the decoder produces one token at a time, so to predict token t it only *has* tokens $1, \ldots, t-1$ – the future does not exist yet, and there is nothing to peek at. Training, however, uses an efficiency trick: instead of running T separate forward passes for a target of length T, we feed the **entire target sequence at once** (using the ground-truth tokens as inputs – so-called *teacher forcing*) and ask *every* position to predict the token that follows it, in a single forward pass. One training example then yields T signals at once: position 1 predicts y_1, position 2 predicts y_2, and so on.

The catch is that this puts the *whole* target sequence into the input. Since self-attention lets every position attend to every other, position t could simply look ahead at position $t+1$ – which already contains the answer y_t – and copy it, reducing "predict the next token" to the trivial "read the next slot". Masking the future before the softmax removes exactly these forward-looking entries, so each position sees only its past: training conditions then match generation conditions, while the parallel-training speed-up is preserved (still a single forward pass, just with the forbidden future entries of the attention matrix set to zero).

Example For a translation task "Le chat dort" → "The cat sleeps", we build two sequences shifted by one position:

position t	1	2	3	4
decoder input	`<start>`	The	cat	sleeps
target y_t	The	cat	sleeps	`<end>`

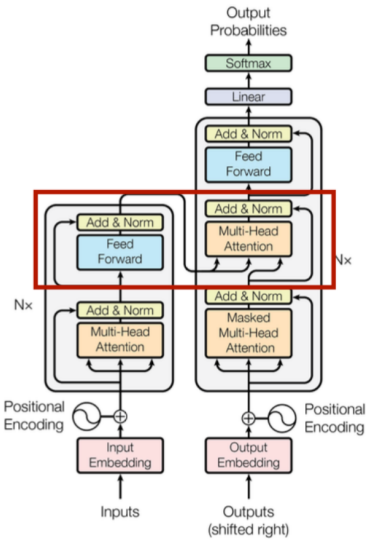
with decoder input = outputs (shifted right) in the image

At each position t, the decoder must predict y_t using only the inputs at positions $1, \ldots, t$. Without masking, when processing position $t = 1$ the decoder could attend to position $t = 2$ of its input, which contains exactly the target y_1 = "The" – the model would learn the trivial shortcut of copying the next input token and collapse at test time. Masking the future before the softmax prevents this.

Cross-attention

This is the same classical attention mechanism that we saw in the RNN encoder–decoder model: the decoder uses its current state as a **query** to attend over the **final** encoder layer's outputs (which act as keys and values). This is what lets the decoder *look at the source sentence* when translating.

So in a decoder block we have, in order: masked self-attention → cross-attention → feed-forward, each followed by Add & Norm.

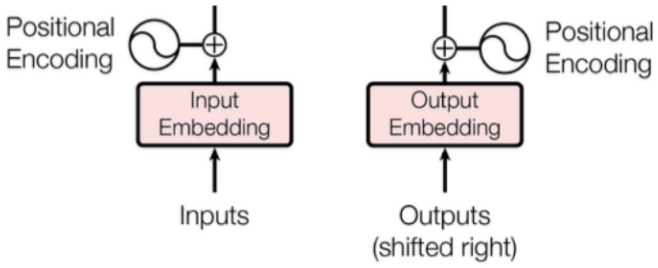


Example Consider translating "Le chat dort" → "The cat sleeps".

1. The encoder reads "Le chat dort" and produces three output vectors h_1^e, h_2^e, h_3^e, one per French token.
2. The decoder has so far produced `<start>`, "The", and is about to predict the next token. Inside the corresponding decoder block:
   - **Masked self-attention** mixes information across the English tokens already generated (`<start>`, "The") – English speaking to English.
   - **Cross-attention** uses the current decoder state as a query to look at the three French vectors. Its query will most likely match the key of "chat" ⇒ high attention on "chat" ⇒ the value of "chat" is pulled into the decoder's representation. The decoder now knows that the word to produce is related to "chat".
3. The feed-forward sub-layer refines the representation, and the output softmax predicts "cat".

2.13.6 Position embeddings

A self-attention layer is, by construction, **permutation-equivariant**: it does not see the order of its inputs (it only sees a *set* of vectors). For language this is obviously wrong – "dog bites man" $\neq$ "man bites dog". We therefore add a **position embedding** p_i to each input embedding to inject positional information.

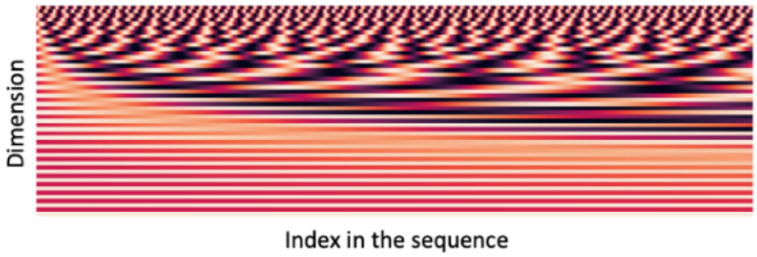


The original transformer used a sinusoidal encoding:

$$p_i = \begin{pmatrix} \sin(i/10000^{2\cdot 1/d}) \\ \cos(i/10000^{2\cdot 1/d}) \\ \vdots \\ \sin(i/10000^{2\cdot \frac{d}{2}/d}) \\ \cos(i/10000^{2\cdot \frac{d}{2}/d}) \end{pmatrix},$$

which gives every position a distinct, smoothly-varying signature.

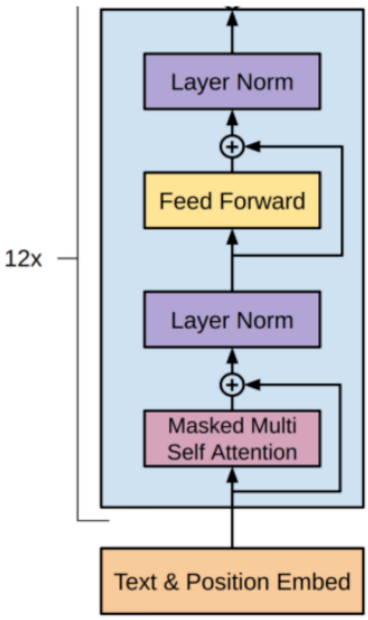
In practice, learned position embeddings (a simple lookup table p_i for each position i, trained end-to-end with the rest of the model) work just as well and are now standard.



2.14 GPT: a decoder-only transformer

GPT (Generative Pretrained Transformer) is called a *decoder* transformer because its job is to generate text autoregressively. Its block, however, *mixes features* from the encoder and decoder of the original transformer:

- from the **decoder**, it inherits the **masked multi-head self-attention** (so the model never sees future tokens during training);
- from the **encoder**, it inherits the simple two-sublayer structure (multi-head attention $\rightarrow$ feed-forward), with the usual residual connections and layer norms. There is **no cross-attention**: there is no encoder for the decoder to attend to, so each block just does self-attention over its own past.



2.14.1 (Pre)Training

GPT is pretrained on a large corpus of raw text (originally Toronto-Books, ∼13 GB, 7000 unpublished books) split into windows of 512 tokens. Its training objective is **next-word prediction** (language modelling):

$$\mathcal{L} \;=\; \sum_{t} -\log P\big(y_t \mid y_1, \ldots, y_{t-1}\big).$$

The model is purely self-supervised – the supervision is the next token of the text itself.

From the final vector to a distribution: unembedding, logits and temperature How is the distribution $P(\,\cdot \mid y_1, \ldots, y_{t-1})$ actually produced? Once the input has flowed through all the blocks, we take the final vector of the sequence and apply a learned **unembedding matrix** W_U. It has one row per vocabulary token (so it is essentially the embedding matrix W_E with its dimensions swapped) and maps the final vector to a list of $|V|$ real numbers – one raw score per possible next token. These unnormalised scores are called the **logits**. They can be negative or larger than 1 and need not sum to 1, so we pass them through a **softmax** to obtain a valid distribution:

$$P(\text{next token} = i) \;=\; \frac{e^{z_i/T}}{\sum_j e^{z_j/T}},$$

where z_i are the logits and T is a **temperature** constant. A large T flattens the distribution (more uniform, hence more "creative" samples); a small T sharpens it so the largest logit dominates; and $T \to 0$ recovers the deterministic *argmax*, i.e. always picking the single most likely token. At training time this read-off is done at *every* position simultaneously – each final vector predicts the token that should follow it, which is exactly the sum over t in the loss above; at generation time we only need the score from the last position.

Generating text Once we can produce a distribution over the next token, generation is just a loop – exactly the autoregressive idea seen for RNNs: start from a seed prompt, **sample** a token from the predicted distribution, **append** it to the sequence, and feed the whole thing back in to predict the next one, repeating until an end-of-sequence token. Sampling (rather than always taking the argmax) is where the temperature above comes into play.

A **chatbot** is built directly on top of this: one prepends a **system prompt** that sets up a helpful assistant, feeds the user's message as the start of the dialogue, and lets the model predict what such an assistant "would say next" (the extra fine-tuning step discussed below is what makes this behave well).

Remark. The model is limited by his context size, because if it is too big then it becomes very costly to compute the self attention as the complexity of the self attention depend on the input sequence (and the size of the embedding). This is why a very long conversation can make a model appear to "lose the thread": content that falls outside the context window is simply no longer visible to it (not the same as RNN, RNN has no limits but the information of the first tokens are diluted by more recent ones).

From input to output: assembling the blocks

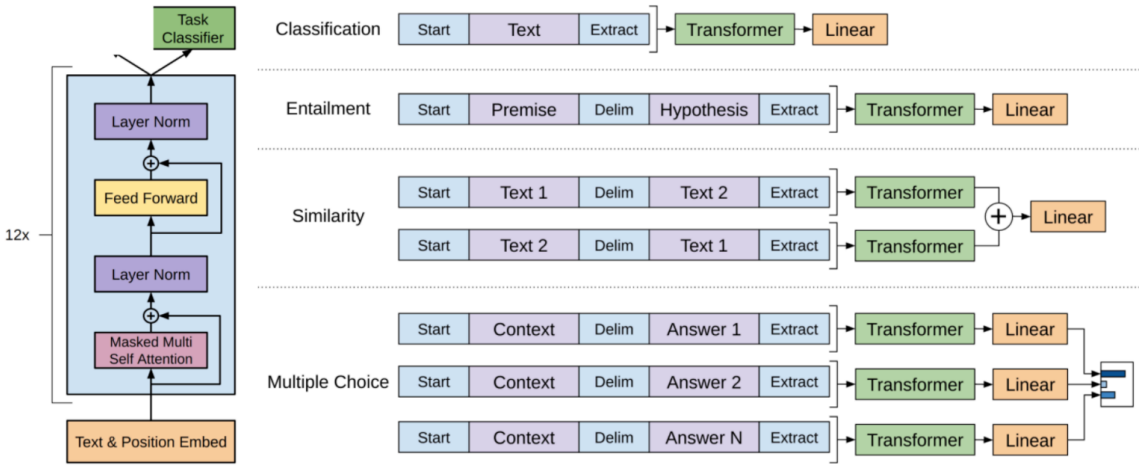
We can now assemble all the pieces into the complete journey of a piece of text through a GPT-style transformer, from raw input to a next-token distribution:

1. **Tokenise and embed**: split the text into tokens, look each up in the embedding matrix W_E to get a sequence of vectors, and add a **position embedding** to each.
2. **Stack of N identical blocks**: each block refines the vectors with, in order, a **masked multi-head self-attention** sub-layer (tokens exchange information, each looking only at the past) followed by a **feed-forward** sub-layer (each vector processed in place), every sub-layer wrapped in a residual connection and a layer norm. Across the stack, every vector gradually soaks up context.
3. **Unembed**: multiply the final vector by W_U to obtain the **logits**, one score per vocabulary token.
4. **Softmax (with temperature)** turns the logits into a probability distribution over the next token.
5. **Generate**: sample a token, append it, and loop back to step 1.

The two workhorses are therefore **multi-head attention**, which moves information *between* tokens, and the **feed-forward** layer, which computes *within* each token; everything else – embeddings, position encodings, residuals, layer norms, unembedding and softmax – is the wiring that turns these two operations into a model mapping text to a distribution over what comes next.

2.14.2 Fine-tuning and the paradigm shift

After pretraining, GPT can be **fine-tuned** on individual downstream tasks (classification, entailment, similarity, multiple-choice question answering) by appending a small task-specific head and continuing to train on the new dataset. The pretrained weights serve as initialization.



This represents a fundamental **paradigm shift** in NLP:

- *Before*: pretrain only the **word embeddings** (Word2Vec, GloVe, fastText), then train a task-specific model from scratch on top of them.
- *After*: pretrain a **full model** (millions to billions of parameters), and reuse it almost as-is for many downstream tasks.

2.14.3 Scale

The history of GPT is largely a history of scaling up:

- GPT (2018): 117M parameters, trained on 13GB of data ∼1B tokens
- GPT-2 (2019): 1.5B parameters, trained on ∼40 GB of data;
- GPT-3 (2020): 175B parameters, ∼500GB data ∼300B tokens;
- GPT-4 (2023): rumoured ∼1.76T parameters, ∼13T tokens.

Each step came with qualitatively new capabilities, often summarised under the umbrella of "emergent abilities".

2.14.4 Other famous text models

GPT is the most well-known but not the only transformer-based language model: **BERT** (Devlin et al., 2018), **RoBERTa** (Liu et al., 2019), **DistilBERT** (Sanh et al., 2019), **Electra** (Clark et al., 2020), and at EPFL, **Meditron** (medical LLM) and **Apertus** (fully open multilingual LLM).

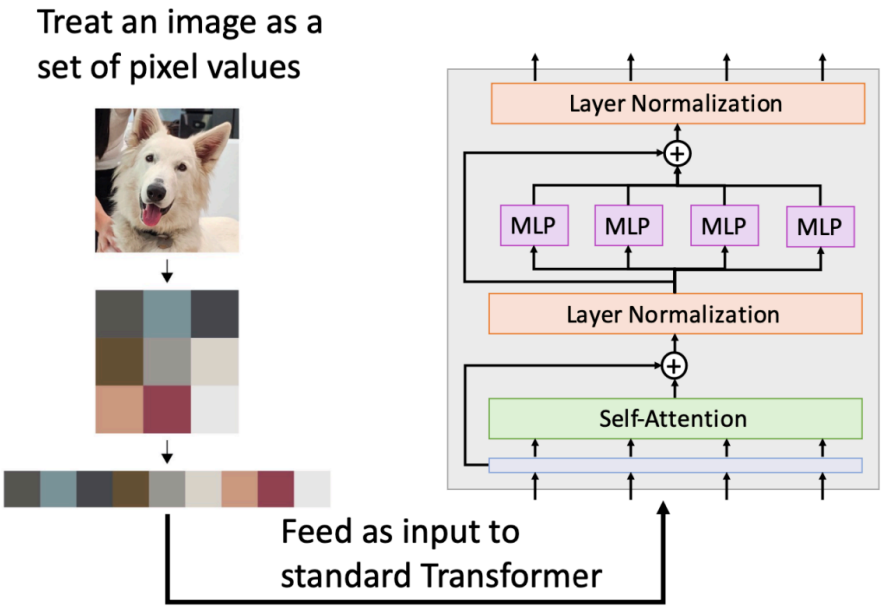
Remark. BERT-style models are *encoder-only* transformers, pretrained with a **masked language modelling** objective (predict a random subset of masked-out tokens using the entire surrounding context, not just the past). They are especially good at *understanding* tasks, whereas GPT-style decoder-only models are designed for *generation*.

2.15 Transformers beyond text

Although transformers were developed for NLP applications, transformers (and the attention mechanism) is generic: it just needs a sequence of vectors as input. The main idea generalizes to other modalities.

2.15.1 Transformers for images

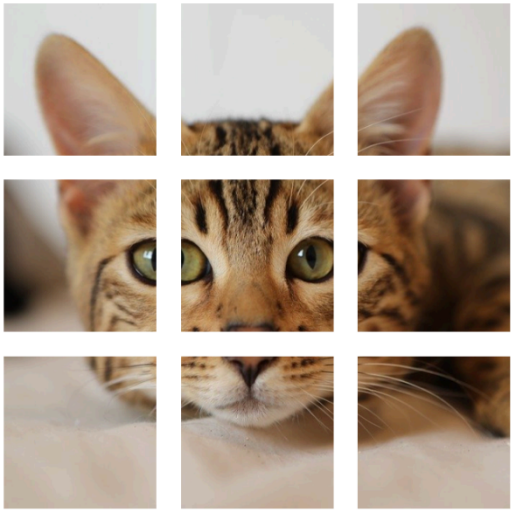
The most direct approach is to flatten an image into a sequence of pixel vectors and feed them to a standard transformer (Chen et al., 2020, *Generative Pretraining from Pixels*).



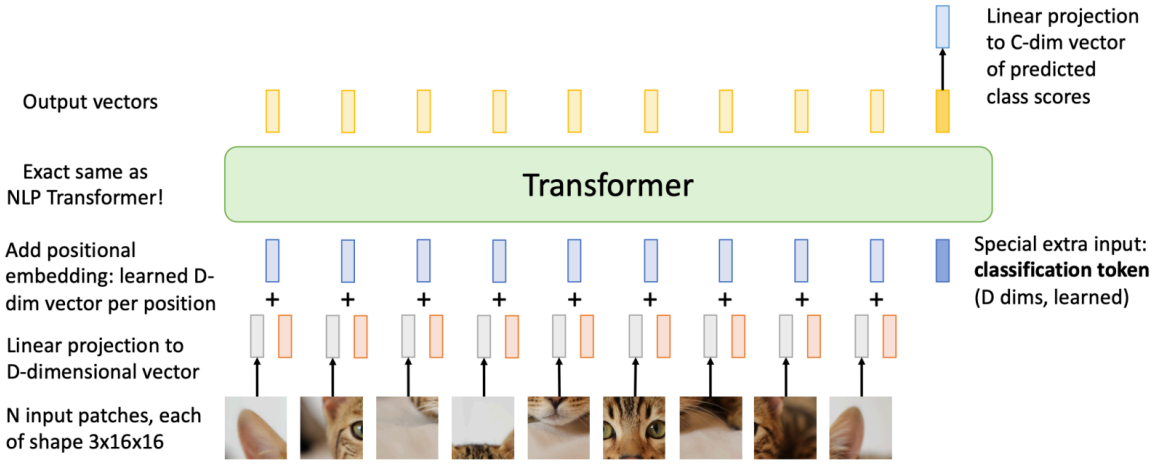
The problem is **memory**: self-attention on N tokens needs $O(N^2)$ memory for the attention matrix, and an $R \times R$ image gives $N = R^2$ tokens, hence R^4 entries per attention matrix per head per layer. The slide quotes 768 GB for a single example at $R = 128$ with 48 layers and 16 heads – clearly impractical.

2.15.2 Vision Transformer (ViT)

Vision Transformer (Dosovitskiy et al., 2021, "An Image is Worth 16×16 Words") solves this by treating an image not as a set of pixels but as a set of **patches**, typically 16×16 each.



An image is split into N non-overlapping patches, each patch is flattened and projected linearly to a D-dimensional vector, and a **learned positional embedding** is added. The result is a sequence of N vectors – exactly the input format expected by a standard NLP transformer.



For classification, a special **classification token** (a learned D-dimensional vector) is prepended to the patch sequence. After the transformer, the output corresponding to that token is fed to a linear classifier to produce class scores. This is analogous to BERT's `[CLS]` token.

Two amusing caveats. ViT is often described as a "computer vision model without convolutions", but:

- the very first operation (splitting the image into $p \times p$ patches and projecting each to a D-vector) is exactly a `Conv2D` with kernel size p and stride p – a convolution;
- the position-wise MLPs inside transformer blocks are equivalent to stacks of 1×1 convolutions.

So strictly speaking ViT is a network of $p \times p$ and 1×1 convolutions, plus self-attention – but the *spirit* of the architecture is genuinely non-convolutional: no large spatial kernels, no spatial pooling, no hand-designed hierarchy.

Takeaway. The success of transformers across very different modalities (text, images, audio, video, proteins, …) is one of the strongest empirical arguments that attention is a remarkably general architectural primitive.

Chapter 3
Unsupervised Learning

3.1 From supervised to unsupervised learning

Until now, we have focused on **supervised learning**, which proceeds in two stages:

- **Training**: use data with ground-truth labels to optimize model parameters by minimizing a loss between predicted and true labels.
- **Testing**: predict the output for new (unlabeled) data samples using the model with fixed parameters.

Supervised learning works well when one has a concrete prediction task to solve and labeled data. It is, however, not suited to scenarios where labels are unavailable, or where the goal is to *understand* the data rather than predict something about it.

This motivates **unsupervised learning**, where the goal is no longer prediction but rather to **transform the data** for further analysis. There is no "training/testing" split: a single algorithm produces a transformed version of the input data.

3.2 Recall from Linear Algebra

First let's recall some important notions from linear algebra.

Rank: number of lines (or columns) lineary independent. Intuitively: how many "truly different" directions are encoded in the matrix. So for a matrix $\mathbb{R}^{N\times D}$, the rank is at most $\min(N, D)$.

Eigenvector and eigenvalues:

$$Av = \lambda v$$

with v an eigenvector and λ the associate eigenvalue.

Geometrical intuition: an eigenvector is a direction that remains the same after the application of the transformation A, it is only stretched or compressed by a factor λ.

3.3 Dimensionality reduction

Dimensionality reduction (DR) is our first unsupervised learning task. It is useful for

- **Data visualization**: projecting high-dimensional data into 2D or 3D so it can be plotted and inspected.
- **Data size reduction**: e.g, vectorizing an image yields a huge vector ($x_i \in \mathbb{R}^{536\cdot 356\cdot 3} = \mathbb{R}^{572448}$). Dealing with such large data is cumbersome – can we compress it while keeping the relevant information?

3.3.1 Intuition: the data manifold

A 536×536 RGB image lies in a $\mathbb{R}^{572448}$ space, but so does pure random noise. The point is that natural images form a tiny subset of this huge space: we are very unlikely to encounter random noise in the real world. The data of interest typically lies in a much **lower-dimensional space** embedded inside the high-dimensional one, called a **manifold**.

Example: consider many 32×32 images obtained by translating and rotating a single digit 3. The data lies in a $32 \times 32 = 1024$-dimensional space, but inherently has only **3 degrees of freedom** (2 translations and 1 rotation), so it truly lies on a 3D manifold inside this 1024D space.

Dimensionality reduction is therefore the task of:

- Discovering the data manifold.
- Getting a low-dimensional representation of the data.
  - Sometimes at some loss of information (hopefully noise).

3.3.2 Formal setting

Let $x_i \in \mathbb{R}^D$ be a high-dimensional data sample and $y_i \in \mathbb{R}^d$ its low-dimensional representation, with $d < D$. Our goal is to find a mapping

$$y_i = f(x_i).$$

The simplest choice is a **linear** mapping, $y_i = W^T x_i$. This leads to PCA.



x **y**

3.3.3 Principal Component Analysis (PCA)

PCA is a linear DR method. Given N samples $\{x_i\}_{i=1}^N$, PCA produces a projection of the form

$$y_i = W^T(x_i - \bar{x}) \quad \text{s.t.} \quad W^T W = I_d,$$

where $\bar{x} = \frac{1}{N}\sum_{i=1}^N x_i$ is the empirical mean of the data. The data is unsupervised, so we need to clarify what we want this projection to achieve.

Objective: variance maximization

Intuitively, we want to keep most of the "important" signal while discarding noise. This can be achieved by finding directions along which the projected data has **large variance**: for the j-th output dimension, we want to maximize

$$\mathrm{var}\big(\{y_i^{(j)}\}\big) = \frac{1}{N}\sum_{i=1}^{N}\big(y_i^{(j)} - \bar{y}^{(j)}\big)^2,$$

where $\bar{y}^{(j)}$ is the mean of the j-th dimension of the data after projection.

Maximizing variance means choosing the direction that best preserves the distances between points, i.e. the one that makes the projected points as distinguishable as possible — which is exactly what we want when compressing information.

1D projection

Let us first consider projecting onto a 1D space. We replace the matrix $W \in \mathbb{R}^{D\times d}$ by a single vector $w_{(1)} \in \mathbb{R}^D$ with $\|w_{(1)}\| = 1$. The projected data has mean

$$\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i = \frac{1}{N}\sum_{i=1}^{N} w_{(1)}^T(x_i - \bar{x}) = w_{(1)}^T\left(\frac{1}{N}\sum_{i=1}^{N} x_i\right) - w_{(1)}^T\bar{x} = 0,$$

since the data is centered by construction. The variance after projection then becomes

$$\mathrm{var}(\{y_i\}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \bar{y})^2 = \frac{1}{N}\sum_{i=1}^{N}\big(w_{(1)}^T(x_i - \bar{x}) - 0\big)^2$$

$$= \frac{1}{N}\sum_{i=1}^{N} w_{(1)}^T(x_i - \bar{x})(x_i - \bar{x})^T w_{(1)} = w_{(1)}^T \underbrace{\left(\frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})(x_i - \bar{x})^T\right)}_{C} w_{(1)} = w_{(1)}^T C w_{(1)}$$

, where $C \in \mathbb{R}^{D\times D}$ is the **input data covariance matrix**.

Constrained optimization via Lagrangian

We seek to solve

$$\max_{w_{(1)}} w_{(1)}^T C w_{(1)} \quad \text{s.t.} \quad w_{(1)}^T w_{(1)} = 1.$$

To do so, we can form the Lagrangian (use to solve optimisation problem with such constraint)

$$\mathcal{L}(w_{(1)}, \lambda_1) = w_{(1)}^T C w_{(1)} + \lambda_1\big(1 - w_{(1)}^T w_{(1)}\big)$$

and setting its gradient w.r.t. $w_{(1)}$ to zero yields

$$\boxed{C w_{(1)} = \lambda_1 w_{(1)}}$$

which is the definition of an **eigenvector**: $w_{(1)}$ must be an eigenvector of C with eigenvalue λ_1. The natural question is: *which one?*

Answer: multiplying both sides on the left by $w_{(1)}^T$ and using the unit-norm constraint gives

$$w_{(1)}^T C w_{(1)} = \lambda_1 w_{(1)}^T w_{(1)} = \lambda_1.$$

The left-hand side is exactly the variance of the projected data (λ_i = variance in the direction of $w_{(i)}$). Since we want to *maximize* it, we should choose $w_{(1)}$ to be the eigenvector corresponding to the **largest eigenvalue** of C.

From 1D to d-dimensional projections

For $d > 1$ output dimensions, we proceed recursively:

- $w_{(2)}$ is the eigenvector of C with the second largest eigenvalue.
- $w_{(3)}$ is the third one, and so on.

The projection matrix is obtained by concatenating these eigenvectors as columns:

$$W = \begin{bmatrix} w_{(1)} & w_{(2)} & \cdots & w_{(d)} \end{bmatrix} \in \mathbb{R}^{D\times d}.$$

This automatically satisfies the constraint $W^TW = I_d$, since the eigenvectors of a (symmetric) matrix are orthogonal and of unit norm. The projected data $y_i = W^T(x_i - \bar{x}) \in \mathbb{R}^d$ has covariance matrix

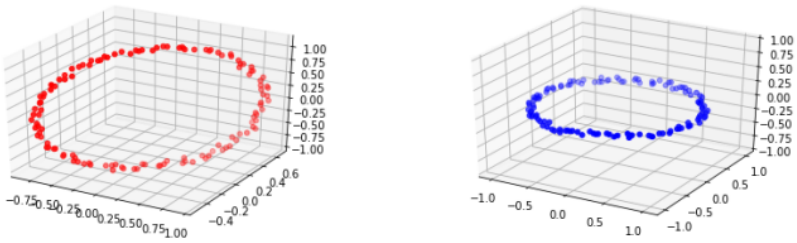
$$\frac{1}{N}\sum_{i=1}^{N} y_i y_i^T = W^T C W = W^T W \Lambda W^T W = \Lambda,$$

where $\Lambda \in \mathbb{R}^{d\times d}$ is diagonal with $\Lambda_{ii} = \lambda_i$ for $i \le d$. Since Λ is diagonal, the projected dimensions are uncorrelated.

Choosing the dimension d

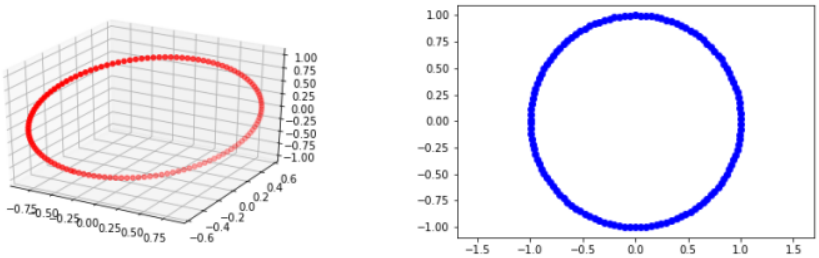
Several options exist:

- **Use all dimensions** ($d = D$): no dimensionality reduction. In 3D, this can be seen as a rotation of the data. There is no loss of information, and the dimensions in the new space are uncorrelated.



x **y**

- **Keep all eigenvectors with non-zero eigenvalues**: (remove direction that gives no information (variance)), the resulting $\{y_i\}$ have $d = rank(C) \leq D$ with no loss of information. If we have $N < D$ then $d = rank(C) \leq \min(N - 1, D)$. ($N - 1$ because we lose one degree of liberty to center the data).

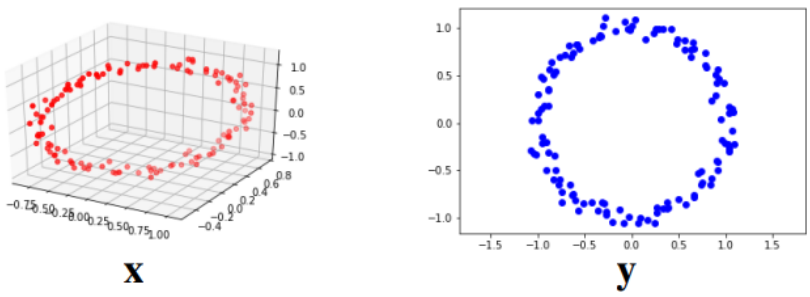


x **y**

- **Truncate**: in practice, one typically discards some non-zero eigenvalues. A common rule is to retain a pre-defined percentage of the data variance, measured as the sum of eigenvalues. For instance, to retain at least 90% of the variance, find the smallest d such that

$$\sum_{j=1}^{d} \lambda_j \;\geq\; 0.9 \cdot \sum_{k=1}^{D} \lambda_k,$$

with eigenvalues sorted in decreasing order. The resulting $\{y_i\}$ have an even lower dimension, at the cost of some loss of information.



x **y**

Note that even when truncating, the corresponding rectangular matrix W is the orthogonal matrix that minimizes the reconstruction error

$$e_i = \|\hat{x}_i - x_i\|^2.$$

where

$$\hat{x}_i \;=\; \bar{x} + W y_i \;=\; \bar{x} + W W^T (x_i - \bar{x}),$$

In other words, for any fixed target dimension d, no other orthogonal projection onto a d-dimensional subspace yields a smaller reconstruction error than PCA – the loss incurred by truncation is the smallest possible at that dimension.

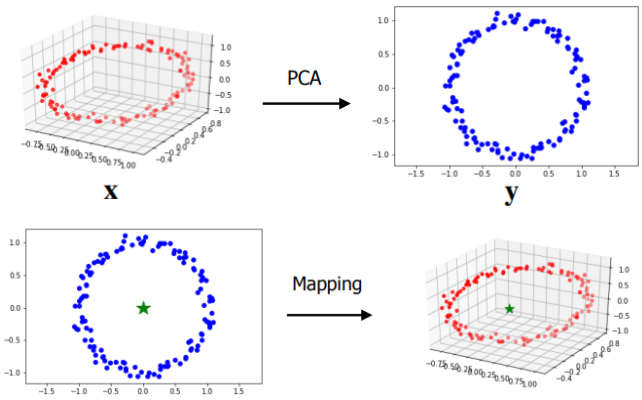
PCA as a continuous mapping

PCA does not only compress the data: it also defines a **continuous mapping back** from the low-dimensional space to the high-dimensional one. For any $y \in \mathbb{R}^d$,

$$\hat{x} = \bar{x} + W y \in \mathbb{R}^D.$$

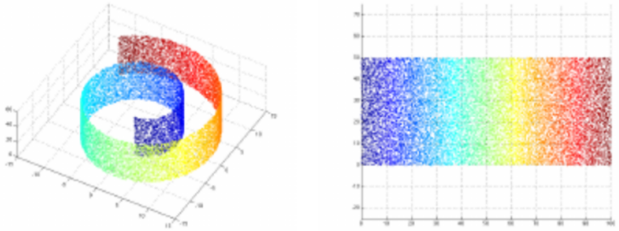
This mapping constrains $\hat{x}$ to lie in a d-dimensional affine subspace, which acts as a form of **regularization**: arbitrary points in the latent (hidden) space ($\mathbb{R}^d$) always decode to plausible points on the learned subspace (the d-dimensional subspace embedded in $\mathbb{R}^D$).

Example Add a new point on the compressed data to visualize it in the original data representation



3.3.4 Kernel PCA

PCA is a **linear** method: it just rotates and projects the data. It cannot "unroll" nonlinear structures such as the *Swiss roll*, where a projection would simply squash the roll instead of unfolding it.



Kernel PCA extends PCA to nonlinear settings using the same intuition as supervised kernel methods:

- Map the data to a high-dimensional feature space, $x_i \mapsto \phi(x_i)$.
- Then perform PCA in this feature space.
- Replace dot products with kernel evaluations $k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ to avoid ever computing ϕ explicitly.

Derivation (centered case)

Assume first that the data is centered in feature space: $\frac{1}{N}\sum_{i=1}^{N}\phi(x_i) = 0$. The covariance matrix in feature space is

$$C = \frac{1}{N}\sum_{i=1}^{N}\phi(x_i)\phi(x_i)^T \in \mathbb{R}^{F\times F}.$$

This is an $F \times F$ matrix (not $N \times N$), so it is not directly expressible from kernel values. PCA in feature space requires solving

$$Cw_{(1)} = \lambda_1 w_{(1)}, \qquad w_{(1)} \in \mathbb{R}^F.$$

Substituting C:

$$\left(\frac{1}{N}\sum_{i=1}^{N}\phi(x_i)\phi(x_i)^T\right)w_{(1)} = \lambda_1 w_{(1)}.$$

By the **Representer theorem**, $w_{(1)}$ can be expressed as a linear combination of the $\{\phi(x_i)\}$:

$$w_{(1)} = \sum_{j=1}^{N} a_j\,\phi(x_j).$$

Substituting this form into the eigenvector equation and (after some computation, skipped here) multiplying by $\phi(x_k)^T$ for each k yields a new eigenvalue problem

$$\boxed{Ka = \lambda_1 Na}$$

where $a \in \mathbb{R}^N$ contains the new parameters and $K \in \mathbb{R}^{N\times N}$ is the **kernel matrix** with entries $K_{ij} = k(x_i, x_j)$.

From a to projections

In theory we could recover $w_{(1)} = \sum_{i=1}^{N} a_i\phi(x_i)$, but this depends on the unknown $\{\phi(x_i)\}$. As in kernel ridge regression, we are not really interested in $w_{(1)}$ – we want the low-dimensional projections $\{y_i\}$. For a 1D representation,

$$y_i = \phi(x_i)^T w_{(1)} = \sum_{j=1}^{N} a_j\,\phi(x_i)^T\phi(x_j) = \sum_{j=1}^{N} a_j\,k(x_i, x_j),$$

which depends only on the kernel function. As in linear PCA, multiple output dimensions are obtained by taking the d eigenvectors of K with the largest eigenvalues.

Non-centered data

In general, even if the original data is centered, the data in feature space is typically not. Define the centered feature mapping

$$\tilde{\phi}(x_i) = \phi(x_i) - \frac{1}{N}\sum_{j=1}^{N}\phi(x_j).$$

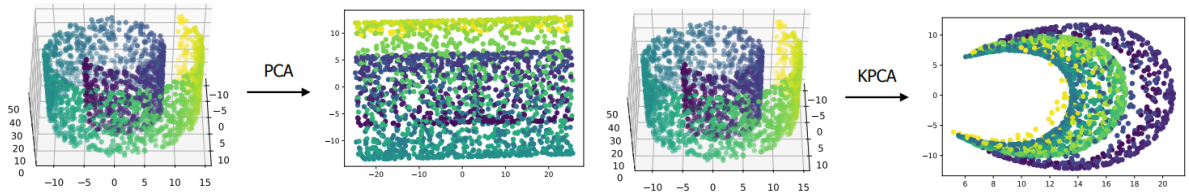
The corresponding kernel matrix $\tilde{K}_{ij} = \tilde{\phi}(x_i)^T\tilde{\phi}(x_j)$ can be computed directly from the original kernel matrix as

$$\tilde{K} = K - \mathbf{1}_N K - K\mathbf{1}_N + \mathbf{1}_N K\mathbf{1}_N,$$

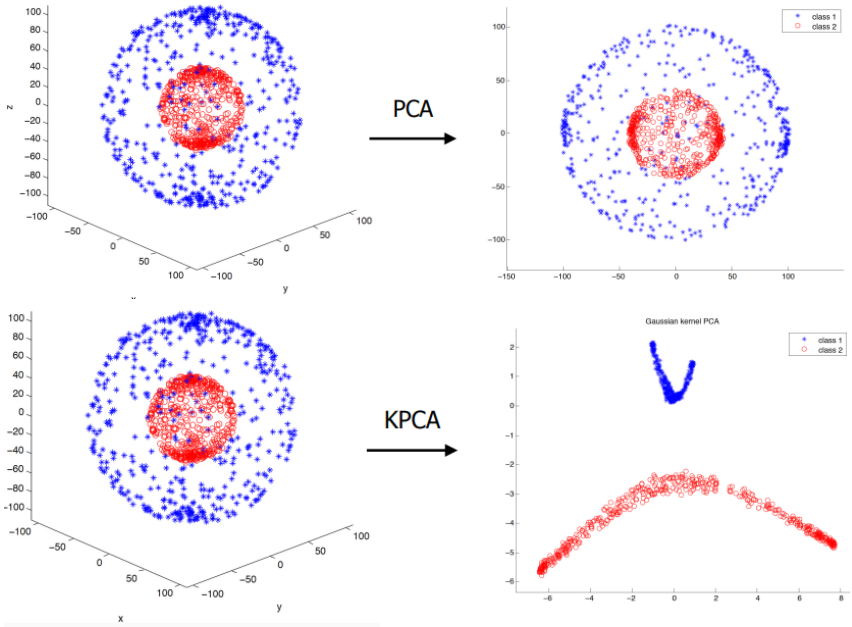
where $\mathbf{1}_N$ is the $N \times N$ matrix with every entry equal to $1/N$. One then runs the eigenvalue problem on $\tilde{K}$ instead of K.

Examples and intuition

- **Swiss roll**: linear PCA squashes it; kernel PCA with an RBF kernel can unroll it into a 2D plane.



- **Concentric spheres in 3D** (one inner "ball" of class 2 surrounded by an outer "shell" of class 1):



  - Linear PCA preserves the nested structure – classes stay tangled.
  - Kernel PCA with a polynomial kernel does not separate the classes well either.
  - Kernel PCA with an **RBF kernel** cleanly separates the two classes.

- **Why does RBF work here?** The RBF kernel $k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ measures *similarity by distance*. Points within the inner ball are mutually close, points on the outer shell are also mutually close, but inner and outer points are far apart. The kernel essentially encodes "which group am I close to", which is exactly the relevant feature here.

Kernel PCA vs PCA: the missing inverse mapping

- Both PCA and kernel PCA give a mapping from high to low dimension.
- PCA **also** provides a clean inverse mapping $\hat{x} = \bar{x} + Wy$ from low back to high dimension.
- **Kernel PCA does not**: there is no direct way to compute the high-dimensional version of an arbitrary point in the low-dimensional space (the inverse of ϕ is not available in general).

This raises the question: can we have a *nonlinear* method that *also* provides an inverse mapping? The answer leads us to autoencoders.

3.3.5 Autoencoders

From PCA to autoencoders

PCA provides two linear mappings:

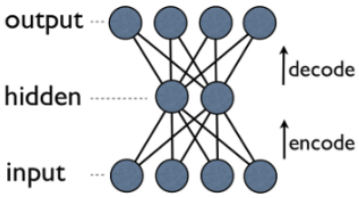
$$y = W^T(x - \bar{x}), \qquad \hat{x} = \bar{x} + Wy.$$

Because both are linear, PCA cannot handle nonlinearly-structured data such as the Swiss roll. There is no fundamental reason to restrict ourselves to linear mappings.

The simplest nonlinear extension is to insert an **element-wise nonlinear activation function** $f(\cdot)$ in the middle, giving

$$\hat{x} = W f(W^T x).$$

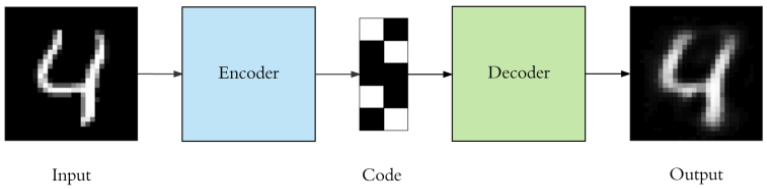
This is the basic form of an **autoencoder**.



General form

More generally, an autoencoder consists of two mappings:

- The **encoder** maps the input x to a latent representation (or *code*) y.
- The **decoder** maps the latent representation back to an output $\hat{x}$ in the same space as x.



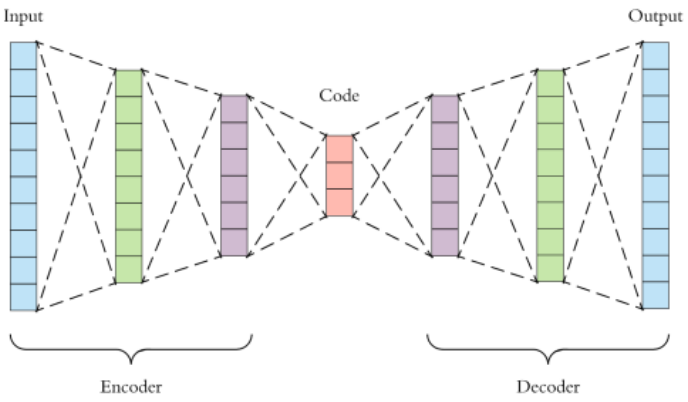
As in PCA, the output of the decoder is meant to be similar to the original input.

Deep autoencoders

There is no reason to restrict the encoder and decoder to a single layer. Stacking multiple layers (each with its own activation) yields a **deep autoencoder**. Each layer implements

$$z_{(j)} = f_{(j)}\big(W_{(j)}^T z_{(j-1)}\big),$$

with $z_{(0)} = x$ on the encoder side, and the latent code is the innermost activation. The decoder typically mirrors the encoder structure.



Training

Following the PCA reconstruction-error perspective, an autoencoder is trained by minimizing the **empirical reconstruction risk**

$$R(\{W_{(j)}\}) = \frac{1}{N}\sum_{i=1}^{N}\left\|\hat{x}_i(\{W_{(j)}\}) - x_i\right\|^2.$$

Like for MLPs, this objective is non-convex, and minimization is performed via **stochastic gradient descent** with backpropagation.

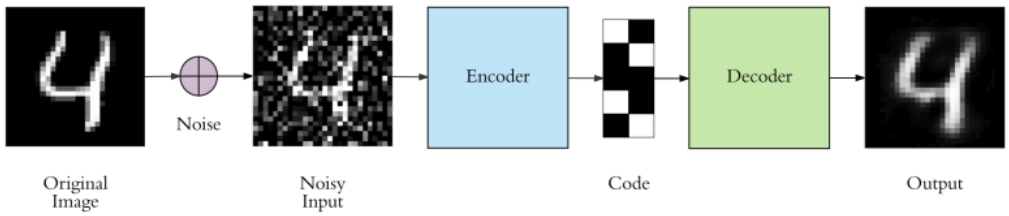
Dimensionality expansion and the trivial solution

With a linear mapping (PCA), reducing dimensionality is mandatory to do something interesting: keeping $d = D$ would simply rotate the data. With a **nonlinear** mapping, this restriction disappears, and one can even **expand** the latent dimensionality to be larger than the input dimension.

This raises a problem: with enough capacity, an autoencoder can simply learn the **identity** mapping (just copying the input), which is useless. Two main strategies prevent this trivial solution.

Denoising autoencoders

Add noise to the input but ask the network to reconstruct the **clean** (noise-free) version of the input. The network then has to learn the structure of the data manifold to remove noise rather than just copying. This forces the latent representation to capture meaningful structure.



Sparse autoencoders

The idea is to force the latent code to be **sparse**, i.e., to have most of its components equal to zero, with only a few non-zero ones. With limited "active" components, the network cannot trivially copy the input across all latent units – it must instead learn a representation where each active component captures something meaningful.

Letting $y_i \in \mathbb{R}^M$ be the code for sample i, sparsity is encouraged by adding an L_1 penalty to the reconstruction loss:

$$E_r(\{W_{(j)}\}) = \sum_{i=1}^{N}\sum_{k=1}^{M}|y_i^{(k)}|.$$

The total training objective then becomes

$$\mathcal{L} = \underbrace{\frac{1}{N}\sum_{i=1}^{N}\|\hat{x}_i - x_i\|^2}_{\text{reconstruction}} + \beta \cdot \underbrace{\sum_{i=1}^{N}\sum_{k=1}^{M}|y_i^{(k)}|}_{\text{sparsity}},$$

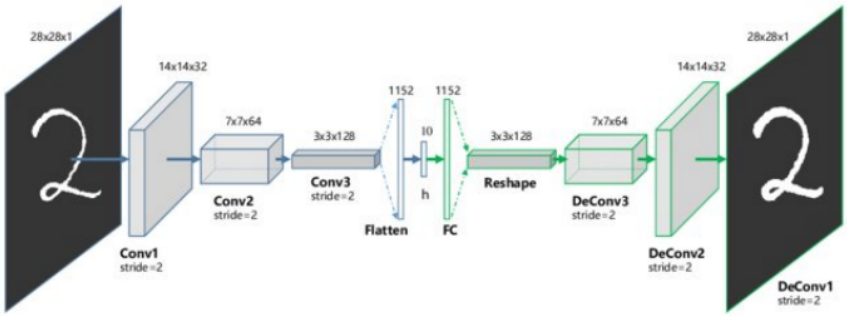
where $\beta > 0$ controls the strength of the sparsity constraint.

Why L_1 and not L_2? The L_1 norm has the special property of producing *exactly zero* components at the optimum, while L_2 would only shrink them towards small values without setting them to zero. This makes L_1 the natural choice for inducing true sparsity.

The two terms compete: reconstruction wants to use as many components as needed for fidelity, while sparsity wants to silence as many as possible. The network is therefore forced to use each active component efficiently – it must encode the most informative features rather than trivially copying the input.

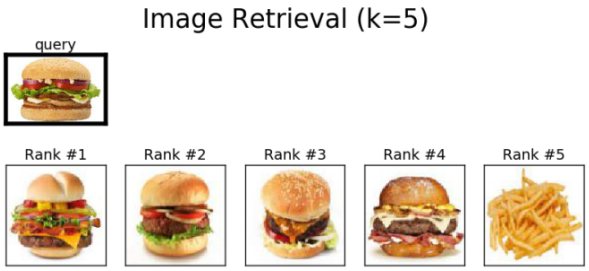
Convolutional autoencoders

For image data, the encoder and decoder can be built from convolutional layers (encoder) and upsampling-convolutions or transposed convolutions (decoder) – mirroring the FCN/U-Net idea seen in Lecture 10. Fully-connected layers are often inserted at the bottleneck. (E.g, Guo et al., ICONIP 2017.)

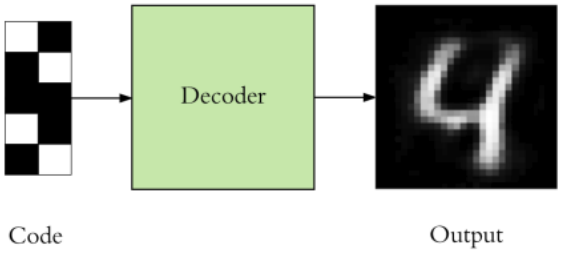


Applications

- **Compact representation / retrieval**: the latent code is a compact summary of the input. Retrieval in a large data collection can be done by k-nearest neighbors in code space (e.g, image retrieval).

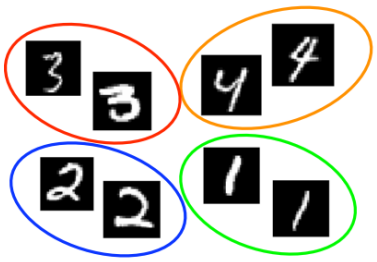


- **Generative use of the decoder**: as in PCA, the decoder defines a mapping from the latent space to the output (same as input) space. By picking arbitrary points in the latent space (we input directly in the latent space), one can generate new samples (e.g, new digit-like images). This is a stepping stone toward proper generative models.

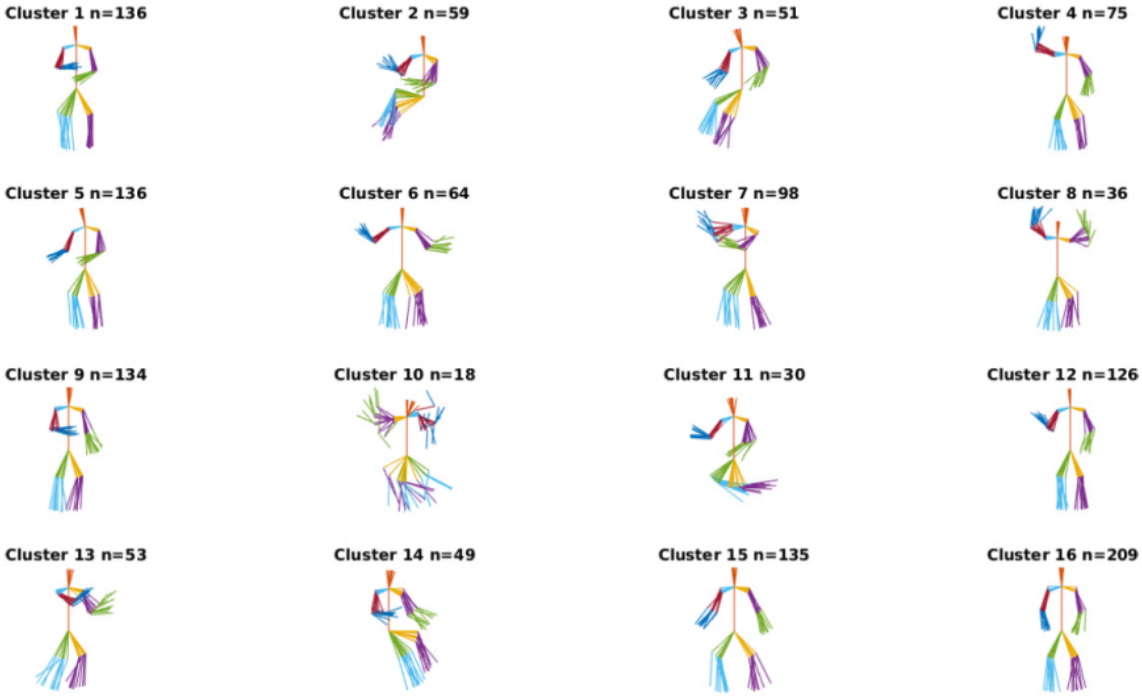


3.4 Clustering: another unsupervised task

Dimensionality reduction transformed the data into a lower-dimensional representation. **Clustering** is a different unsupervised task: given a set of input samples, we want to identify natural **groups** of similar samples *without* performing any data transformation. The output is therefore not a new representation but a **partition** of the data into clusters.



A typical motivating example is grouping human poses observed in a collection of paintings (Impett & Süsstrunk, 2017), so as to discover recurring poses across different artists. Each sample is then a high-dimensional pose vector, and clustering yields a small number of prototypical poses.



Another classical use is the construction of a **visual dictionary**: image patches are grouped by similarity, and the center of each group becomes a "visual word".

3.5 K-means clustering

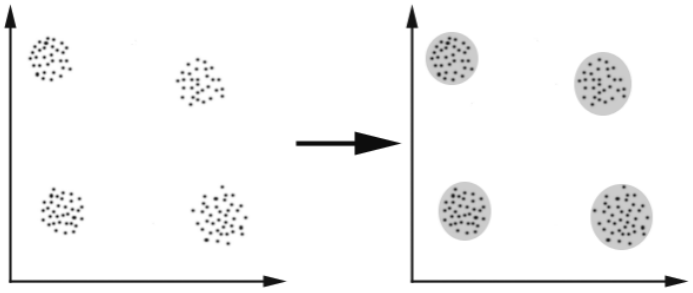
3.5.1 Intuition

K-means is the simplest clustering algorithm. The user specifies a number of clusters K (a **hyper-parameter**, assumed given) and the algorithm groups the N samples into K clusters such that:

- the distances between points *within* the same cluster are small,
- the distances *across* clusters are large.

This is encoded via K **cluster centers** $\{\mu_1, \ldots, \mu_K\}$: each sample is assigned to the nearest center, and the centers themselves are placed at the mean of their assigned samples.

Example For samples that act as a 2D points



3.5.2 Algorithm

The algorithm alternates between assigning points and updating centers:

1. **Initialize** the centers $\{\mu_1, \ldots, \mu_K\}$ (e.g, randomly).
2. **While not converged**:
    - **Assignment step**: for each point x_i, compute the Euclidean distance to every center μ_k, and assign x_i to the cluster with the smallest distance. Each point belongs to a single cluster.
    - **Update step**: recompute each center μ_k as the *mean* of the points currently assigned to it.

Convergence

The algorithm iteratively updates assignments and centers. We need a stopping criterion. Using a fixed number of iterations is arbitrary and dangerous if too small. A better criterion is to stop when the assignments (or the centers) no longer change between two iterations: **K-means is guaranteed to converge** to such a stable solution.

Important caveat: convergence does not mean optimality. The solution reached depends heavily on the **initialization** of the centers, and different initializations can lead to different (sometimes much worse) clusterings.

3.5.3 Properties and practical considerations

Pros:

- Simple algorithm, guaranteed to converge.
- Returns exactly K clusters.
- Fully unsupervised: only needs the data, no labels.
- The Euclidean distance works well when all dimensions are *homogeneous*.

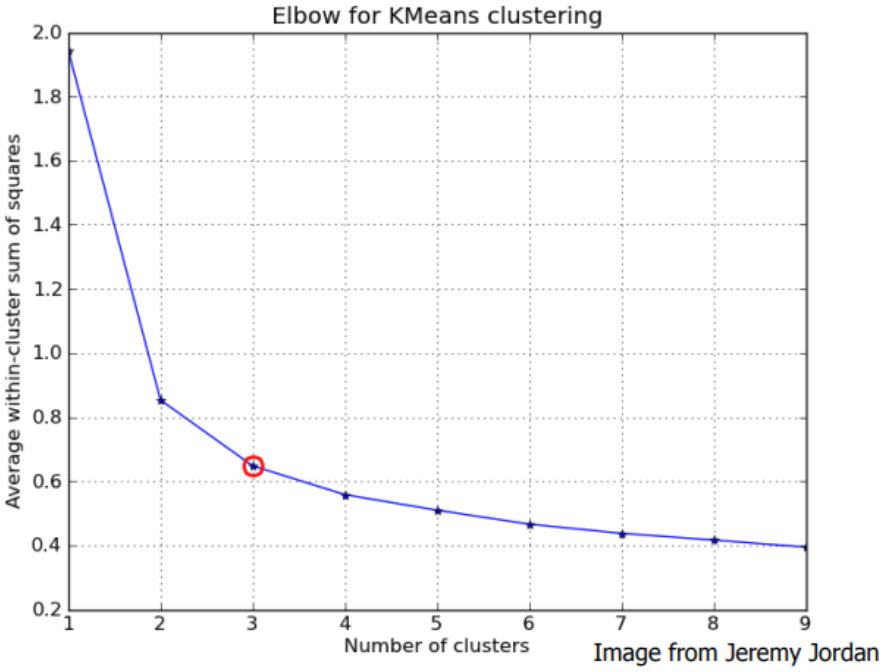
Cons and remedies:

- **May converge to a bad local optimum.** Run the algorithm several times with different initializations; for each run, compute the sum of distances from points to their assigned center; keep the run with the smallest sum.
- **Some clusters may end up empty** (no point assigned).
- **The user must choose K.** Since clustering is unsupervised, cross-validation is not an option. A common heuristic is the **elbow method** described below.
- **The dimensions may be heterogenous** thus the euclidian distance will works poorly. Normalize the data or use a different distance function.

If class labels happen to be available afterwards, they can be used to *label* each cluster: for each cluster, count the number of samples of each category and assign the cluster the majority category. Without labels, providing a semantic meaning to a cluster requires human inspection of its members.

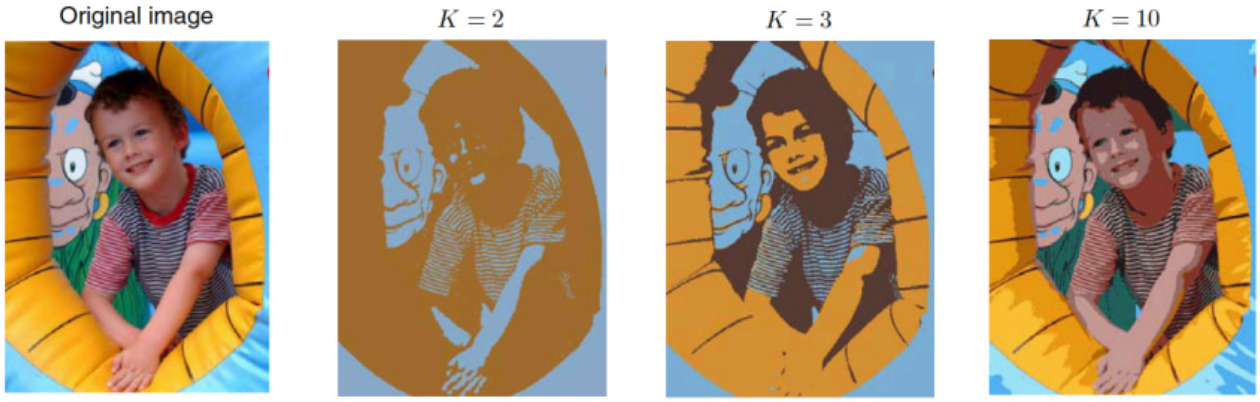
Elbow method for choosing K

Run K-means for increasing values of K and plot the average within-cluster distance (sum of squared distances from points to their assigned center) as a function of K. This curve typically decreases towards 0 (the extreme being one cluster per point). The **"elbow"** is the value of K beyond which the gain in within-cluster distance becomes marginal: it offers a reasonable trade-off between cluster compactness and interpretability.



3.5.4 Example: color-based image segmentation

A natural application of K-means is image segmentation by color: each pixel is treated as a data point $x_i \in \mathbb{R}^3$ encoded by its RGB values, and K-means groups pixels with similar colors. Increasing K yields finer segmentations.



3.5.5 The Euclidean distance is not always appropriate

The Euclidean distance works well when all dimensions are *homogeneous* (commensurate magnitudes, comparable semantics). This was the case in the previous examples: 2D toy points, RGB pixels in $[0, 255]^3$, or human pose vectors where every dimension represents the same type of quantity.

In practice this is not always the case. Different dimensions can have wildly different magnitudes *and* represent completely different kinds of information.

Example Consider the **Wine dataset** (UCI ML repository): 178 wines from 3 producers, each described by 13 attributes such as alcohol content, malic acid, magnesium. Two samples:

Sample 1: [14.37, 1.95, 2.5, 16.8, 113, 3.85, 3.49, 0.24, 2.18, 7.8, 0.86, 3.45, 1480]
Sample 2: [13.24, 2.59, 2.87, 21, 118, 2.8, 2.69, 0.39, 1.82, 4.32, 1.04, 2.93, 735]

The last dimension (proline content, in the thousands) will dominate the Euclidean distance and crush the contribution of all other features – although there is no reason to think it is more meaningful for the clustering.

Solution: *normalize* (e.g, scale each dimension to zero mean and unit variance) and/or use a different distance function. On the wine data this makes a large difference:

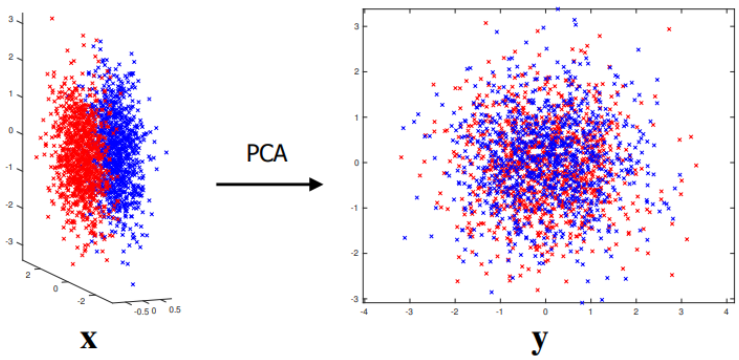


- Fig. 1: Euclidean distance, raw data: 70.3% accuracy.
- Fig. 2: Euclidean distance, scaled data: 93.2% accuracy.
- Fig. 3: L^1 distance, scaled data: 94.5% accuracy.

(Here *accuracy* means the percentage of samples assigned to the correct cluster, after matching each cluster to its majority producer.)

3.6 Fisher Linear Discriminant Analysis (LDA)

The dimensionality reduction methods we saw last week (PCA, kernel PCA, autoencoders) are **unsupervised**. As a result, they may project the data in directions that mix up classes – nothing tells PCA that some directions are more semantically meaningful than others.



If class labels *are* available, we can do better. **Fisher LDA** combines clustering and dimensionality reduction by looking for a projection in which

- samples from the *same* class cluster tightly,
- different classes are well *separated*.

Note that we are technically back to supervised learning, but the supervision is not used to predict $\{y_i\}$ directly: it only guides the projection. The intuition is the same as K-means – small within-cluster spread, large between-cluster spread – but applied *after* a linear projection.

3.6.1 Clustering within each class: within-class scatter

Consider a 1D projection given by a vector $w_{(1)} \in \mathbb{R}^D$, and C classes. Let $y_i = w_{(1)}^T x_i$ and let ν_c be the mean of the projections of class c. To cluster the samples of each class, we want **low variance within each class after projection**, i.e, minimize

$$E_W(w_{(1)}) = \sum_{c=1}^{C} \sum_{i \in c} (y_i - \nu_c)^2,$$

where ν_c is the mean of the samples in class c after projection, and $i \in c$ means "sample i belongs to class c". Both y_i and ν_c depend on $w_{(1)}$.

Since the projection is linear, the class-mean after projection is the projection of the class-mean before projection:

$$\nu_c = \frac{1}{N_c} \sum_{i \in c} y_i = \frac{1}{N_c} \sum_{i \in c} w_{(1)}^T x_i = w_{(1)}^T \mu_c, \qquad \text{so} \qquad y_i - \nu_c = w_{(1)}^T (x_i - \mu_c),$$

where μ_c is the mean of class c before projection.

As we did for PCA, the variance after projection equals the projection of a covariance-like matrix, which lets us rewrite this as

$$E_W(w_{(1)}) = w_{(1)}^T S_W \, w_{(1)}, \qquad S_W = \sum_{c=1}^{C} \sum_{i \in c} (x_i - \mu_c)(x_i - \mu_c)^T,$$

where μ_c is the mean of class c *before* projection. The matrix S_W is called the **within-class scatter matrix**.

3.6.2 Separating the classes: between-class scatter

To separate clusters, we also want to push the class means *away* from each other after projection. Letting $\bar{y}$ denote the mean of all projected samples and N_c the number of samples in class c, we want to *maximize*

$$E_B(w_{(1)}) = \sum_{c=1}^{C} N_c(\nu_c - \bar{y})^2.$$

By the same reasoning,

$$E_B(w_{(1)}) = w_{(1)}^T S_B\, w_{(1)}, \qquad S_B = \sum_{c=1}^{C} N_c\,(\mu_c - \bar{x})(\mu_c - \bar{x})^T,$$

where $\bar{x}$ is the mean of all samples. S_B is called the **between-class scatter matrix**.

3.6.3 Fisher LDA objective

We want simultaneously to:

- **Minimize** $E_W(w_{(1)})$ (tight classes),
- **Maximize** $E_B(w_{(1)})$ (well-separated classes).

A standard trick to do both at once is to maximize their *ratio*: minimizing $f(\cdot)$ is, in general, achieved by maximizing $1/f(\cdot)$. We therefore maximize

$$J(w_{(1)}) = \frac{E_B(w_{(1)})}{E_W(w_{(1)})} = \frac{w_{(1)}^T S_B\, w_{(1)}}{w_{(1)}^T S_W\, w_{(1)}}.$$

Scaling invariance and the constrained form

J is invariant to the scaling of $w_{(1)}$, i.e, $J(\alpha w_{(1)}) = J(w_{(1)})$ for any $\alpha \in \mathbb{R}$. We can therefore fix the scale by imposing $w_{(1)}^T S_W\, w_{(1)} = 1$, giving the Fisher LDA formulation

$$\max_{w_{(1)}} w_{(1)}^T S_B\, w_{(1)} \quad \text{s.t.} \quad w_{(1)}^T S_W\, w_{(1)} = 1.$$

Generalized eigenvalue problem

Forming the Lagrangian

$$\mathcal{L}(w_{(1)}, \lambda_1) = w_{(1)}^T S_B\, w_{(1)} + \lambda_1\big(1 - w_{(1)}^T S_W\, w_{(1)}\big)$$

and setting its gradient w.r.t. $w_{(1)}$ to zero yields

$$\boxed{S_B\, w_{(1)} = \lambda_1 S_W\, w_{(1)}}$$

which is a **generalized eigenvalue problem**. Left-multiplying by $w_{(1)}^T$ and dividing by $w_{(1)}^T S_W\, w_{(1)} = 1$ shows that the value of the objective J at the optimum is exactly λ_1, so we should pick the eigenvector $(w_{(1)})$ with the **largest eigenvalue**.

From 1D to d-dimensional projections

As for PCA, more output dimensions are obtained recursively: take the d eigenvectors of the generalized problem with the largest eigenvalues.

An important limitation: the rank of S_B is at most $C-1$ (it is a sum of C rank-1 matrices that are linearly dependent through the constraint that all class means average to $\bar{x}$). The remaining eigenvalues are exactly zero, so Fisher LDA can only project the data to at most $C-1$ dimensions. With 10 MNIST digit classes, this caps us at 9 LDA directions.

3.6.4 Example: visualizing MNIST

Projecting MNIST digits with Fisher LDA:

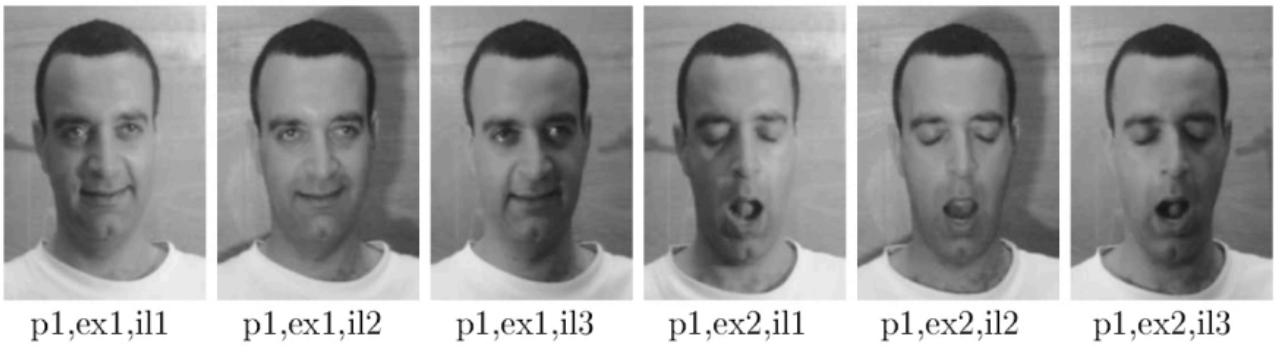


- in 2D, the classes form somewhat separated blobs, classification accuracy in the LDA space: 56%.
- in 3D, the separation improves significantly, classification accuracy: 74%.

3.6.5 Eigenfaces vs Fisherfaces

Consider a face dataset with 2 expressions (treated as 2 classes) and several illumination conditions. Applying PCA or Fisher LDA to vectorized face images yields eigenvectors that can themselves be visualized as images:

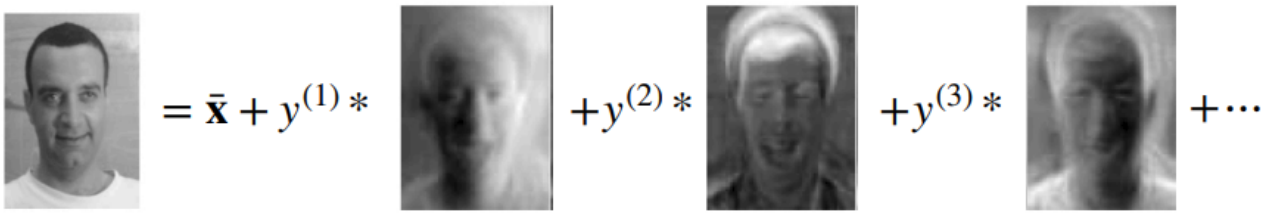
- the PCA directions are called **eigenfaces**,
- the LDA directions are called **fisherfaces**.



Eigenfaces minimize the reconstruction error. They therefore must capture whatever explains most of the variance in the data – and illumination changes induce huge pixel variations. Eigenfaces end up encoding a lot of illumination information.



Image reconstruction:



Fisherfaces are explicitly chosen to be discriminative of the class (here: expression). They therefore *discard* illumination, which is irrelevant to expression recognition, and emphasize features around the mouth and eyes.

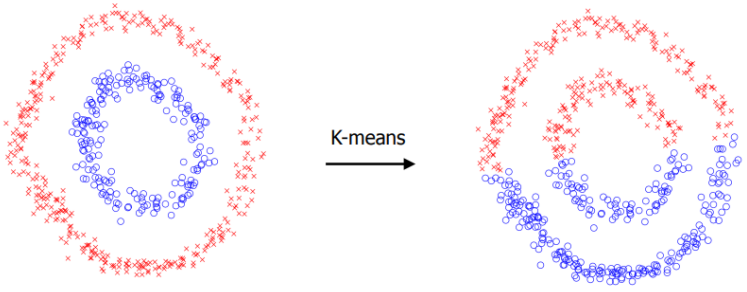


This is a nice illustration of how the choice of objective dictates which features survive projection: *maximum variance* (PCA) and *maximum discriminability* (LDA) can lead to very different bases.

3.7 Spectral clustering

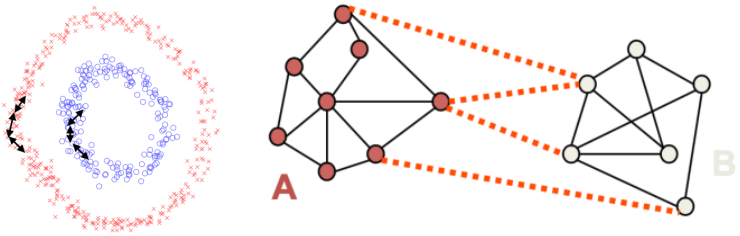
3.7.1 Why K-means and LDA can fail

K-means and Fisher LDA both rely on the notion of **compactness**: each cluster is expected to be a tight blob around its mean. This works well on data with compact samples but fails on data that is not compact. The classical pathological case is **two concentric circles**: there are clearly two natural clusters, but K-means – which only sees distances to centers – cuts the data in half through the middle.



3.7.2 Connectivity as the right notion

What separates the two circles is **connectivity**: every point on the inner circle has close neighbors that are themselves on the inner circle, and similarly for the outer circle, while no inner-circle point is close to any outer-circle point. We can model this with a **graph** whose nodes are the data points and whose edges encode local similarity (strong connections indicates nodes that should be clustered, weaker ones suggests that the graph can be cut into pieces/partitions).



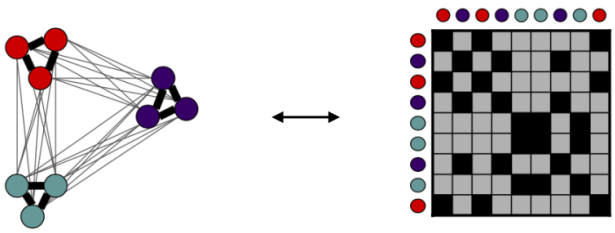
3.7.3 Building the graph

The similarity (connectivity) between points i and j is typically encoded by a positive weight W_{ij}. A standard choice is the Gaussian (RBF) similarity

$$W_{ij} = \exp\left(\frac{-\|x_i - x_j\|^2}{\sigma^2}\right),$$

where σ is a hyper-parameter controlling how quickly similarity decays with distance. This yields a **fully-connected** graph (every pair of points has an edge), but one typically restricts the graph to the K**-nearest neighbors** of each point to encode locality and keep the graph sparse.

A graph is equivalently described by its **similarity (affinity) matrix** $W \in \mathbb{R}^{N \times N}$.

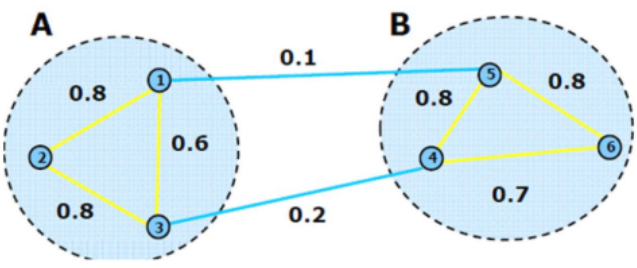


3.7.4 Graph cut

Consider a partition of the graph into two sets A and B. The **cut** between A and B is the total weight of the edges that connect them:

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} W_{ij}.$$

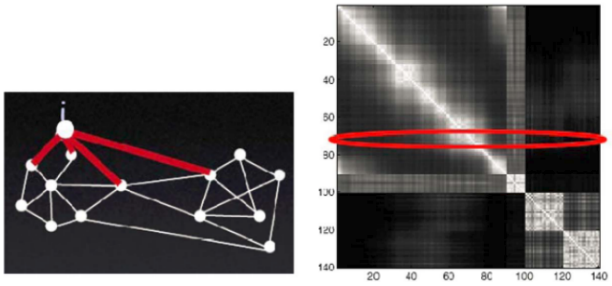
Intuitively, a good clustering minimizes the cut: we want few, weak edges across the boundary.



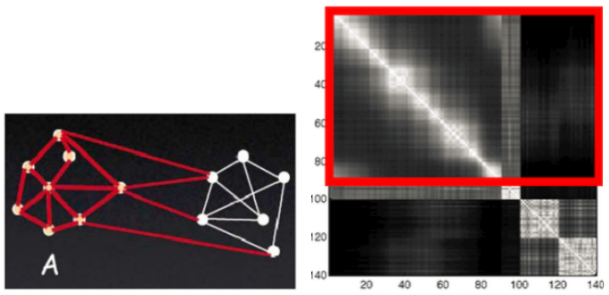
3.7.5 Graph terminology

We will also need:

- The **degree** of a node: $d_i = \sum_j W_{ij}$ (total weight incident on i).



- The **volume** of a set: $\text{vol}(A) = \sum_{i \in A} d_i$ (total degree of nodes in A).



3.7.6 Normalized cut

Minimizing $\text{cut}(A, B)$ alone is misleading: cutting a single isolated point off as "cluster A" often gives a very small cut while being a useless clustering. To prevent this, we use the **normalized cut**, which penalizes imbalanced partitions by dividing by the volume of each side:

$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{vol}(A)} + \frac{\text{cut}(A, B)}{\text{vol}(B)}.$$

If A contains only a tiny fraction of the graph, its volume is small ($\geq cut(A, B)$) and the first term could be close to 1. As $\frac{\text{cut}(A,B)}{\text{vol}(A)} \in [0, 1]$ a balanced partition is preferred.

3.7.7 Transformed formulation

Finding the partition that minimizes Ncut is **NP-hard** (we are choosing one of 2^N partitions). It can however be rewritten as the optimization problem

$$\min_y \frac{y^T(D - W)y}{y^T D y} \quad \text{s.t.} \quad y^T D\mathbf{1} = 0, \;\; y_i \in \{1, -b\} \;\; \forall\, 1 \leq i \leq N,$$

where:

- $W \in \mathbb{R}^{N \times N}$ is the similarity matrix between all pairs of points,
- $D \in \mathbb{R}^{N \times N}$ is the diagonal **degree matrix** with $D_{ii} = d_i$,
- $y \in \mathbb{R}^N$ indicates the partition assignment of each point.
- $b = \frac{vol(A)}{vol(B)}$

This is still NP-hard because of the discrete constraint $y_i \in \{1, -b\}$.

3.7.8 Spectral relaxation

The standard trick is to **relax** the discrete constraint and allow $y \in \mathbb{R}^N$:

$$\min_y \frac{y^T(D-W)y}{y^T D y} \quad \text{s.t.} \quad y^T D y = 1.$$

Using the Lagrangian, this turns into the generalized eigenvalue problem

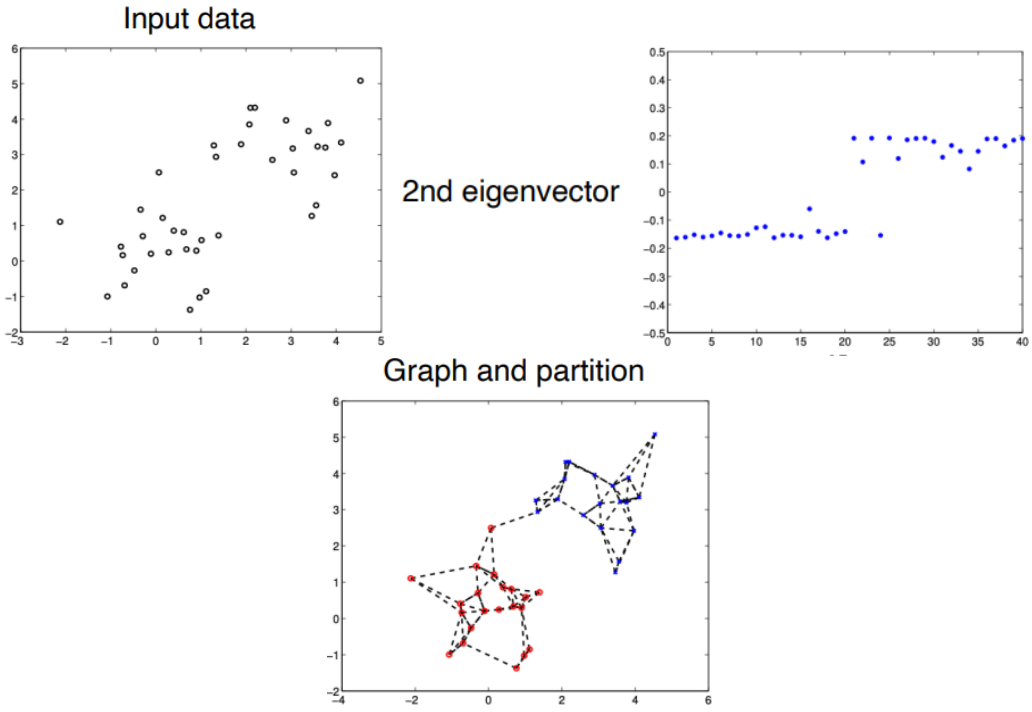
$$\boxed{(D-W)\,y = \lambda\, D\, y}$$

The matrix $L = D - W$ is called the **graph Laplacian**.

Which eigenvector?

The eigenvector with the **smallest** eigenvalue ($\lambda = 0$) is the trivial all-ones vector – it places every point in the same partition and is useless. The actual solution is the eigenvector with the **second smallest** eigenvalue (sometimes called the *Fiedler vector*).

In an ideal case, positive entries of this eigenvector would indicate one partition and negative entries the other. Because of the relaxation, the entries are continuous, so one has to **threshold** them. A common choice is to threshold at the median, which yields a balanced partition by construction.



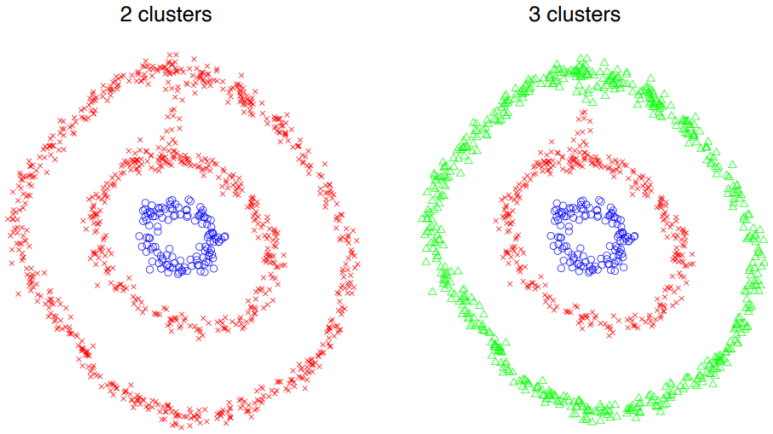
3.7.9 K-way partitioning

To obtain more than two clusters, two strategies exist:

1. **Recursive 2-way partitioning**: apply the 2-way spectral clustering, then recursively split each subset. This is inefficient and unstable – small changes early on can propagate dramatically.
2. **Use K eigenvectors at once**: take the K eigenvectors of the generalized problem with the smallest eigenvalues, and represent each point i by the K-dimensional vector formed by stacking the i-th entry of each eigenvector. Then run **K-means** on these K-dimensional vectors.

Interpretation: the second approach is dimensionality reduction (via the graph Laplacian) *followed by* K-means. Spectral clustering can therefore be seen as "K-means on a smarter feature space" – one where compactness aligns with the connectivity structure of the original data.

Example with $K = 2$ and $K = 3$



3.7.10 Beyond compactness and connectivity: density

For completeness, another paradigm for clustering is **density-based** clustering (e.g, DBSCAN), which defines clusters as regions of high point density separated by regions of low density. This is yet another way of escaping the compactness assumption of K-means.

Chapter 4
Review

This chapter ties together the whole course. The point is not to re-derive anything, but to see the **connections**: most of the supervised methods share a single linear model and differ only by their loss, kernelization is the same trick applied to many algorithms, and deep learning is the idea of learning the representation instead of designing it. The same backbone reappears everywhere.

4.1 The machine learning pipeline

Machine learning = Data → Algorithms → Insight.

- **Data** can be tabular attributes, text, speech/sound or images. In all algorithms seen, a sample is a vector $x_i \in \mathbb{R}^D$.
- **Insight** can be a category, a description, a vector-valued quantity or a different (lower-dimensional) representation.
- **Algorithms** split into **supervised learning** (relies on annotated data, the annotations being the desired insight) and **unsupervised learning** (analyzes or transforms unlabeled data).

4.2 Supervised learning

4.2.1 Setting

Supervised learning has two stages: **training**, where data with ground-truth labels is used to optimize the model parameters by minimizing a loss, and **testing**, where the fixed model predicts the output of a new sample. Two problem types:

- **Classification**: predict one discrete (unordered) label.
- **Regression**: predict continuous (ordered) value(s).

4.2.2 The linear model

The output varies linearly with the inputs. Absorbing the bias by appending a 1 to x (so $x \in \mathbb{R}^{D+1}$), a single-output model is

$$y = w^{(0)} + \sum_{j=1}^{D} w^{(j)} x^{(j)} = w^T x.$$

For C output dimensions we use a matrix $W \in \mathbb{R}^{(D+1)\times C}$ and

$$\hat{y} = W^T x,$$

where each column $w_{(k)}$ predicts one output dimension.

4.2.3 Linear regression

"Best fit" is encoded as minimizing the squared residuals:

$$\min_w \sum_{i=1}^{N} \left(x_i^T w - y_i\right)^2.$$

This is convex and has a **closed-form** solution (zero the gradient). Prediction for a new sample is $\hat{y} = (w^*)^T x$.

Overfitting and ridge regression

Increasing model complexity always lowers the training error but eventually raises the test error → **overfitting**. A standard remedy is to add a regularizer penalizing the weight norm:

$$\min_w \sum_{i=1}^{N} \left(x_i^T w - y_i\right)^2 + \lambda\, w^T w.$$

For $\lambda > 0$ this stays convex. The multi-output version uses the squared Frobenius norm:

$$\min_W \sum_{i=1}^{N} \|W^T x_i - y_i\|^2 + \lambda \|W\|_F^2.$$

4.2.4 Linear classification

Classification as regression, and its failure

A binary label can be predicted with a line, $\hat{y} = w^{(0)} + w^{(1)} x$, and a sample assigned by thresholding. Multiple classes are handled with one-hot labels and a multi-output least-square fit, assigning $k = \arg\max_j \hat{y}^{(j)}$. This **least-square classification** fails: a single far-away sample produces a huge residual that drags the line and misclassifies easy points. The cause is fitting a line to inherently nonlinear (0/1) data.

Logistic regression

We would like a discrete output; we approximate it by squashing the linear score through a **sigmoid**,

$$\hat{y}(x) = \sigma(w^T x) = \frac{1}{1 + \exp(-w^T x)}.$$

For C classes we use the **softmax**, giving a probability per class,

$$\hat{y}^{(k)} = \frac{\exp(w_{(k)}^T x)}{\sum_{j=1}^{C} \exp(w_{(j)}^T x)},$$

and train by minimizing the **cross-entropy** empirical risk

$$R(W) = -\sum_{i=1}^{N} \sum_{k=1}^{C} y_i^{(k)} \ln \hat{y}_i^{(k)},$$

via gradient descent (no closed form).

Margin and the SVM

Different classifiers give different decision boundaries, so what makes one good? The **margin** is the orthogonal distance between the boundary and the nearest sample; a good boundary maximizes it. The **Support Vector Machine** is a binary classifier on the same linear model $\hat{y} = w^T x$ that maximizes the margin while enforcing correct classification. It is written as the convex problem

$$\min_{w^{(0)},\,\tilde{w}} \ \tfrac{1}{2} C\|\tilde{w}\|^2 + \sum_{i=1}^{N} \max\Big(0,\, 1 - y_i\,(w^{(0)} + \tilde{w}^T x_i)\Big)\,,$$

where the $\max(\cdot)$ term is the **hinge loss** (encourages correct classification) and $\|\tilde{w}\|^2$ is the inverse margin (minimizing it maximizes the margin).

The key insight

Definition 9 (Same model, different loss)**.** Least-square classification, logistic regression and the SVM all rely on the *same* linear model. What distinguishes them is only the **loss function**: square, logistic and hinge respectively, each approximating the ideal but non-differentiable 0–1 loss. The choice of loss is what shapes the resulting algorithm.

4.2.5 Dealing with nonlinear data

Feature expansion

Linear models fail on nonlinear data, but a fixed nonlinear **feature map** $\phi(x)$ can make it separable (e.g. $x \to [x, x^2]$). The polynomial expansion stacks the constant, linear, quadratic, …, M-th power terms and cross-products, and the expanded features are then used in any algorithm, regression or classification. The drawback: one must pick the degree (and which functions), the number of choices grows combinatorially, and high degrees overfit.

Kernels

Many algorithms ultimately depend only on dot products $x_i^T x_j$, which encode similarity ($x_i^T x_j \propto \cos\angle(x_i, x_j)$). After expansion they become $\phi(x_i)^T \phi(x_j)$. Instead of choosing ϕ explicitly, we directly define a **kernel function**

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j),$$

which corresponds to *some* mapping $\phi(\cdot)$ that may be unknown.

Representer theorem and kernel ridge regression

The **representer theorem** states that the minimizer of a regularized empirical risk is a linear combination of the training points in feature space,

$$w^* = \sum_{i=1}^{N} \alpha_i \phi(x_i) = \Phi^T \alpha.$$

Substituting into ridge regression rewrites the risk purely in terms of the kernel,

$$R(\alpha) = \sum_{i=1}^{N} \big(\alpha^T k(X, x_i) - y_i\big)^2 + \lambda\, \alpha^T K \alpha,$$

and prediction for a new sample never needs Φ explicitly:

$$\hat{y} = y^T (K + \lambda I_N)^{-1} k(X, x).$$

The same **kernel trick** kernelizes the SVM (via its dual), logistic regression, and later PCA. Kernel methods handle nonlinear data with often nice (closed-form, convex) solutions, but depend on the choice of kernel and scale with the number of samples.

4.2.6 Neural networks

Rather than design ϕ by hand, neural networks **learn the representation jointly** with the final model. A layer applies a linear map followed by an elementwise nonlinear **activation** f (sigmoid, tanh, ReLU). With the convention that a is the **pre-activation** and z the **post-activation**, one hidden layer gives

$$z_{(1)} = f\left(W_{(1)}^T x\right), \qquad \hat{y} = f_{(2)}\left(W_{(2)}^T z_{(1)}\right).$$

Stacking layers yields a **multilayer perceptron (MLP)**; each layer has its own parameters. Training minimizes the square loss (regression) or cross-entropy (classification) by **stochastic gradient descent**, with gradients computed efficiently by **backpropagation**.

4.2.7 Convolutional neural networks

Treating an image as one big vector is wasteful: it gives a huge parameter count, ignores that nearby pixels share structure, and is not translation robust. **Convolutions** fix this by reducing parameters and adding translation robustness. A **CNN** stacks convolution and pooling blocks (varying the number of channels and filter sizes), then vectorizes the final feature map and appends fully-connected layer(s) to output a vector (class probabilities, joint coordinates, …). The weights are learned exactly as in an MLP.

4.2.8 Sequences and text

Words are mapped to dense **word embeddings**, real-valued vectors whose similarity reflects similarity of meaning. To model a whole sentence as a sequence:

- **RNNs** process the sequence step by step, carrying a hidden state h_t via feedback loops; unrolling them across time gives the full computation graph and lets them learn from history of any length.
- **Attention** lets the decoder focus on the relevant encoder outputs at each step instead of a single fixed summary vector.
- **Transformers** drop recurrence entirely and encode sequences with **multi-head self-attention** plus feedforward layers, in cascaded encoder and decoder blocks.
- The same idea returns to vision in the **Vision Transformer**, which treats image patches as a sequence of tokens.

4.2.9 k-Nearest Neighbors

A simple nonparametric baseline: compute the distance from the test sample x to all training samples, find the nearest x_{NN}, and predict its label $\hat{y} = y_{NN}$ (or a vote/average over the k nearest). No training, yet very competitive on tasks such as MNIST.

4.3 Unsupervised learning

Unsupervised learning is a single stage: analyze the whole dataset, or transform it for further analysis, without labels.

4.3.1 Clustering

K-means

Identify K groups so that within-cluster distances are small and across-cluster distances are large, encoded via distances to cluster centers $\{\mu_1, \ldots, \mu_K\}$. The algorithm alternates until convergence:

1. assign each point to the nearest center;
2. update each center as the mean of its assigned points.

Compactness vs. connectivity

K-means exploits **compactness** and fails on non-compact shapes (e.g. concentric rings). There, clusters are better defined by **connectivity**.

Spectral clustering

Encode the data as a graph with a similarity matrix $W \in \mathbb{R}^{N \times N}$ and a diagonal degree matrix D with $D_{ii} = \sum_j W_{ij}$. Strong edges should stay together, weak ones can be cut. The **normalized cut** is relaxed to a generalized eigenvalue problem

$$(D - W)\, y = \lambda D y,$$

and the partition is read off the eigenvector with the **second smallest** eigenvalue.

4.3.2 Dimensionality reduction

PCA

Real data lives on a low-dimensional manifold inside a huge ambient space. PCA projects

$$y_i = W^T(x_i - \bar{x}), \qquad \text{s.t. } W^T W = I_d,$$

keeping the "important" signal by finding directions of **large variance**. For a 1D output,

$$\max_{w_{(1)}} w_{(1)}^T C\, w_{(1)} \quad \text{s.t. } w_{(1)}^T w_{(1)} = 1,$$

solved by the eigenvector of the covariance C with the largest eigenvalue; more dimensions take the next-largest eigenvectors. PCA also gives a mapping **back** to the high-dimensional space, $\hat{x} = \bar{x} + Wy$, which constrains $\hat{x}$ to a subspace and thus regularizes.

Kernel PCA

Applying the kernel trick to PCA leads to the eigenvalue problem $Ka = \lambda_1 Na$, and the low-dimensional projections depend only on the kernel,

$$y_i = \sum_{j=1}^{N} a_j \, k(x_i, x_j),$$

so $w_{(1)}$ (which depends on the unknown ϕ) is never needed explicitly.

Fisher LDA

PCA is unsupervised and may discard class information. Using labels, Fisher **LDA** seeks a projection where samples of the same class cluster and different classes separate (the same intuition as K-means, but supervised). For a 1D projection,

$$\max_{w_{(1)}} w_{(1)}^T S_B \, w_{(1)} \quad \text{s.t. } w_{(1)}^T S_W \, w_{(1)} = 1,$$

with S_B the between-class and S_W the within-class scatter matrices; the solution is again a generalized eigenvalue problem.

Autoencoders

PCA's encode/decode pair ($y = W^T(x - \bar{x})$, $\hat{x} = \bar{x} + Wy$) generalizes to an **autoencoder**: there is no reason to keep the encoder and decoder linear or single-layer. Stacking nonlinear layers gives a **deep autoencoder**, and using transposed (upsampling) convolutions gives a **convolutional autoencoder**. It is trained by minimizing the reconstruction error

$$R(\{W_{(j)}\}) = \frac{1}{N} \sum_{i=1}^{N} \|\hat{x}_i(\{W_{(j)}\}) - x_i\|^2,$$

non-convex and optimized by SGD, just like supervised deep networks.

4.4 Cross-cutting concepts

4.4.1 Data representation

Real data is often large and heterogeneous (e.g. documents of different lengths). A good representation makes it compact and homogeneous, e.g. **bag-of-words** histograms of fixed length. Even then, data usually needs pre-processing: filling in missing values, cleaning noise, and **normalizing** features that have incommensurate magnitudes.

4.4.2 Cross-validation

Most algorithms have hyper-parameters; tuning them on the training loss favors complex models and overfits, yet the test set must not be touched. **Cross- validation** estimates generalization by holding out part of the training data. The trade-offs:

- **Validation set**: cheap, but an unreliable estimate.
- k**-fold** (e.g. 10-fold): only "wastes" $1/k$ of the data and is k times the cost of a single validation set; fewer folds (3-fold) are cheaper but noisier.
- **Leave-one-out** (N-fold): wastes no data but is expensive.

There is no ideal solution $\rightarrow$ the choice is a cost/reliability trade-off.